

- A custom ticker identifier created using `ticker.*()` functions. Ticker IDs constructed from these functions may contain additional settings for requesting data using non-standard chart calculations, alternative sessions, and other contexts. See the Custom contexts section for more information.

The `timeframe` value specifies the timeframe of the requested data. This parameter accepts “string” values in our timeframe specification format (e.g., a value of “1D” represents the daily timeframe). To request data from the same timeframe as the chart the script executes on, use the `timeframe.period` variable or an empty string.

The `expression` parameter of the `request.security()` function determines the data it retrieves from the specified context. This versatile parameter accepts “series” values of int, float, bool, color, string, and `chart.point` types. It can also accept tuples, collections, user-defined types, and the outputs of function and method calls. For more details on the data one can retrieve, see the Requestable data section below.

Timeframes

The `request.security()` function can request data from any available timeframe, regardless of the chart the script executes on. The timeframe of the data retrieved depends on the `timeframe` argument in the function call, which may represent a higher timeframe (e.g., using “1D” as the `timeframe` value while running the script on an intraday chart) or the chart’s timeframe (i.e., using `timeframe.period` or an empty string as the `timeframe` argument).

Scripts can also request *limited* data from lower timeframes with `request.security()` (e.g., using “1” as the `timeframe` argument while running the script on a 60-minute chart). However, we don’t typically recommend using this function for LTF data requests. The `request.security_lower_tf()` function is more optimal for such cases.

Higher timeframes Most use cases of `request.security()` involve requesting data from a timeframe higher than or the same as the chart timeframe. For example, this script retrieves the hl2 price from a requested `higherTimeframe`. It plots the resulting series on the chart alongside the current chart’s hl2 for comparison:



Figure 178: image

```
//@version=6
indicator("Higher timeframe security demo", overlay = true)

//@variable The higher timeframe to request data from.
string higherTimeframe = input.timeframe("240", "Higher timeframe")

//@variable The `hl2` value from the `higherTimeframe`. Combines lookahead with an offset to avoid repainting.
float htFPrice = request.security(syminfo.tickerid, higherTimeframe, hl2[1], lookahead = barmerge.lookahead_on)

// Plot the `hl2` from the chart timeframe and the `higherTimeframe`.
plot(hl2, "Current timeframe HL2", color.teal, 2)
plot(htFPrice, "Higher timeframe HL2", color.purple, 3)
```

Note that:

- We've included an offset to the `expression` argument and used `barmerge.lookahead_on` in `request.security()` to ensure the series returned behaves the same on historical and realtime bars. See the Avoiding repainting section for more information.

Notice that in the above example, it is possible to select a `higherTimeframe` value that actually represents a *lower timeframe* than the one the chart uses, as the code does not prevent it. When designing a script to work specifically with higher timeframes, we recommend including conditions to prevent it from accessing lower timeframes, especially if you intend to publish it.

Below, we've added an if structure to our previous example that raises a runtime error when the `higherTimeframe` input represents a timeframe smaller than the chart timeframe, effectively preventing the script from requesting LTF data:



Figure 179: image

```
//@version=6
indicator("Higher timeframe security demo", overlay = true)

//@variable The higher timeframe to request data from.
string higherTimeframe = input.timeframe("240", "Higher timeframe")

// Raise a runtime error when the `higherTimeframe` is smaller than the chart's timeframe.
if timeframe.in_seconds() > timeframe.in_seconds(higherTimeframe)
    runtime.error("The requested timeframe is smaller than the chart's timeframe. Select a higher timeframe.")

//@variable The `hl2` value from the `higherTimeframe`. Combines lookahead with an offset to avoid repainting.
float htfPrice = request.security(syminfo.tickerid, higherTimeframe, hl2[1], lookahead = barmerge.lookahead_on)

// Plot the `hl2` from the chart timeframe and the `higherTimeframe`.
plot(hl2, "Current timeframe HL2", color.teal, 2)
plot(htfPrice, "Higher timeframe HL2", color.purple, 3)
```

Lower timeframes Although the `request.security()` function is intended to operate on timeframes greater than or equal to the chart timeframe, it *can* request data from lower timeframes as well, with limitations. When calling this function to access a lower timeframe, it will evaluate the `expression` from the LTF context. However, it can only return the results from a *single* intrabar (LTF bar) on each chart bar.

The intrabar that the function returns data from on each historical chart bar depends on the `lookahead` value in the function call. When using `barmerge.lookahead_on`, it will return the *first* available intrabar from the chart period. When using `barmerge.lookahead_off`, it will return the *last* intrabar from the chart period. On realtime bars, it returns the last available value of the `expression` from the timeframe, regardless of the `lookahead` value, as the realtime intrabar information retrieved by the function is not yet sorted.

This script retrieves close data from the valid timeframe closest to a fourth of the size of the chart timeframe. It makes two calls to `request.security()` with different `lookahead` values. The first call uses `barmerge.lookahead_on` to access the first intrabar value in each chart bar. The second uses the default `lookahead` value (`barmerge.lookahead_off`), which requests the last intrabar value assigned to each chart bar. The script plots the outputs of both calls on the chart to compare the difference:

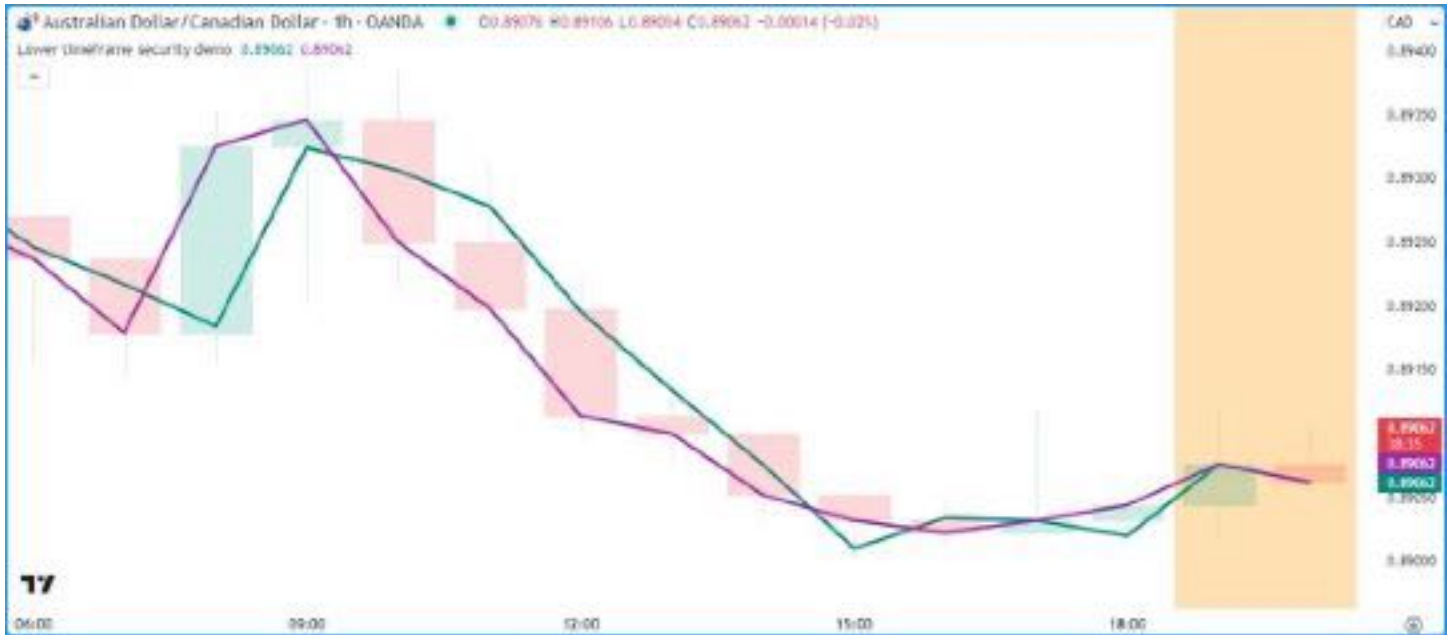


Figure 180: image

```
//@version=6
indicator("Lower timeframe security demo", overlay = true)

//@variable The valid timeframe closest to 1/4 the size of the chart timeframe.
string lowerTimeframe = timeframe.from_seconds(int(timeframe.in_seconds() / 4))

//@variable The `close` value on the `lowerTimeframe`. Represents the first intrabar value on each chart bar.
float firstLTFClose = request.security(syminfo.tickerid, lowerTimeframe, close, lookahead = barmerge.lookahead_on)
//@variable The `close` value on the `lowerTimeframe`. Represents the last intrabar value on each chart bar.
float lastLTFClose = request.security(syminfo.tickerid, lowerTimeframe, close)

// Plot the values.
plot(firstLTFClose, "First intrabar close", color.teal, 3)
plot(lastLTFClose, "Last intrabar close", color.purple, 3)
// Highlight the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 70) : na, title = "Realtime background highlight")
```

Note that:

- The script determines the value of the `lowerTimeframe` by calculating the number of seconds in the chart timeframe with `timeframe.in_seconds()`, then dividing by four and converting the result to a valid timeframe string via `timeframe.from_seconds()`.
- The plot of the series without lookahead (purple) aligns with the close value on the chart timeframe, as this is the last intrabar value in the chart bar.
- Both `request.security()` calls return the *same* value (the current close) on each realtime bar, as shown on the bars with the orange background.
- Scripts can retrieve up to 200,000 intrabars from a lower-timeframe context. The number of chart bars with available intrabar data varies with the requested lower timeframe, the `calc_bars_count` value, and the user's plan. See this section of the Limitations page.

Requestable data

The `request.security()` function is quite versatile, as it can retrieve values of any fundamental type (int, float, bool, color, or string). It can also request the IDs of data structures and built-in or user-defined types that reference fundamental types. The data this function requests depends on its **expression** parameter, which accepts any of the following arguments:

- Built-in variables and function calls
- Variables declared by the script
- Tuples
- Calls to user-defined functions
- Chart points
- Collections
- User-defined types

Built-in variables and functions A frequent use case of `request.security()` is requesting the output of a built-in variable or function/method call from another symbol or timeframe.

For example, suppose we want to calculate the 20-bar SMA of a symbol's ohlc4 price from the daily timeframe while on an intraday chart. We can accomplish this with a single line of code:

```
float ma = request.security(syminfo.tickerid, "1D", ta.sma(ohlc4, 20))
```

The above line calculates the value of `ta.sma(ohlc4, 20)` on the current symbol from the daily timeframe.

It's important to note that newcomers to Pine may sometimes confuse the above line of code as being equivalent to the following:

```
float ma = ta.sma(request.security(syminfo.tickerid, "1D", ohlc4), 20)
```

However, this line will return an entirely *different* result. Rather than requesting a 20-bar SMA from the daily timeframe, it requests the `ohlcv` price from the daily timeframe and calculates the `ta.sma()` of the results over 20 **chart bars**.

In essence, when the intention is to request the results of an expression from other contexts, pass the expression *directly* to the `expression` parameter in the `request.security()` call, as demonstrated in the initial example.

Let's expand on this concept. The script below calculates a multi-timeframe (MTF) ribbon of moving averages, where each moving average in the ribbon calculates over the same number of bars on its respective timeframe. Each `request.security()` call uses `ta.sma(close, length)` as its **expression** argument to return a **length**-bar SMA from the specified timeframe:

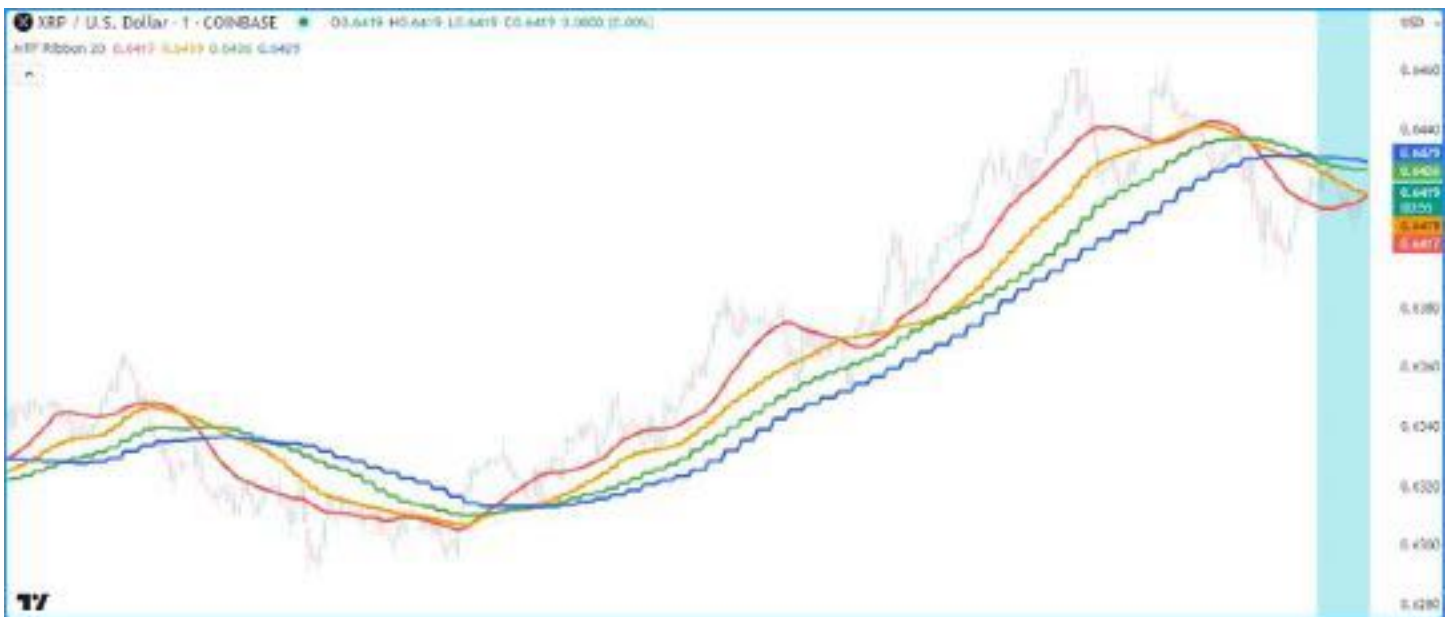


Figure 181: image

```
//@version=6
indicator("Requesting built-ins demo", "MTF Ribbon", true)

//@variable The length of each moving average.
```

```

int length = input.int(20, "Length", 1)

//@variable The number of seconds in the chart timeframe.
int chartSeconds = timeframe.in_seconds()

// Calculate the higher timeframes closest to 2, 3, and 4 times the size of the chart timeframe.
string htf1 = timeframe.from_seconds(chartSeconds * 2)
string htf2 = timeframe.from_seconds(chartSeconds * 3)
string htf3 = timeframe.from_seconds(chartSeconds * 4)

// Calculate the `length`-bar moving averages from each timeframe.
float chartAvg = ta.sma(ohlc4, length)
float htfAvg1 = request.security(syminfo.tickerid, htf1, ta.sma(ohlc4, length))
float htfAvg2 = request.security(syminfo.tickerid, htf2, ta.sma(ohlc4, length))
float htfAvg3 = request.security(syminfo.tickerid, htf3, ta.sma(ohlc4, length))

// Plot the results.
plot(chartAvg, "Chart timeframe SMA", color.red, 3)
plot(htfAvg1, "Double timeframe SMA", color.orange, 3)
plot(htfAvg2, "Triple timeframe SMA", color.green, 3)
plot(htfAvg3, "Quadruple timeframe SMA", color.blue, 3)

// Highlight the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.aqua, 70) : na, title = "Realtime highlight")

```

Note that:

- The script calculates the ribbon's higher timeframes by multiplying the chart's `timeframe.in_seconds()` value by 2, 3, and 4, then converting each result into a valid timeframe string using `timeframe.from_seconds()`.
- Instead of calling `ta.sma()` within each `request.security()` call, one could use the `chartAvg` variable as the **expression** in each call to achieve the same result. See the next section for more information.
- On realtime bars, this script also tracks *unconfirmed* SMA values from each higher timeframe. See the Historical and realtime behavior section to learn more.

Declared variables The `request.security()` function's **expression** parameter can accept declared variables that are accessible to the scope from which the function call executes. When using a declared variable as the **expression** argument, the function call *duplicates* all *preceding code* that determines the assigned value or reference. This duplication allows the function to evaluate necessary calculations and logic in the requested context without affecting the original variable.

For instance, this line of code declares a `priceReturn` variable that holds the current bar's arithmetic price return:

```
float priceReturn = (close - close[1]) / close[1]
```

We can evaluate the `priceReturn` variable's calculations in another context by using it as the **expression** in a `request.security()` call. The call below duplicates the variable's calculation and evaluates it across the data from another `symbol`, returning a *separate series* adapted to the chart's time axis:

```
float requestedReturn = request.security(symbol, timeframe.period, priceReturn)
```

This example script compares the price returns of the current chart's symbol and a user-specified symbol. It calculates the value of the `priceReturn` variable, then uses that variable as the **expression** in a `request.security()` call to evaluate the calculation on the input symbol's data. After the request, the script calculates the correlation between the `priceReturn` and `requestedReturn` and plots the result on the chart:

```

//@version=6
indicator("Requesting calculated variables demo", "Price return correlation")

//@variable The symbol to compare to the chart symbol.
string symbol = input.symbol("SPY", "Symbol to compare")
//@variable The number of bars in the calculation window.
int length = input.int(60, "Length", 1)

//@variable The close-to-close price return.
float priceReturn = (close - close[1]) / close[1]

```

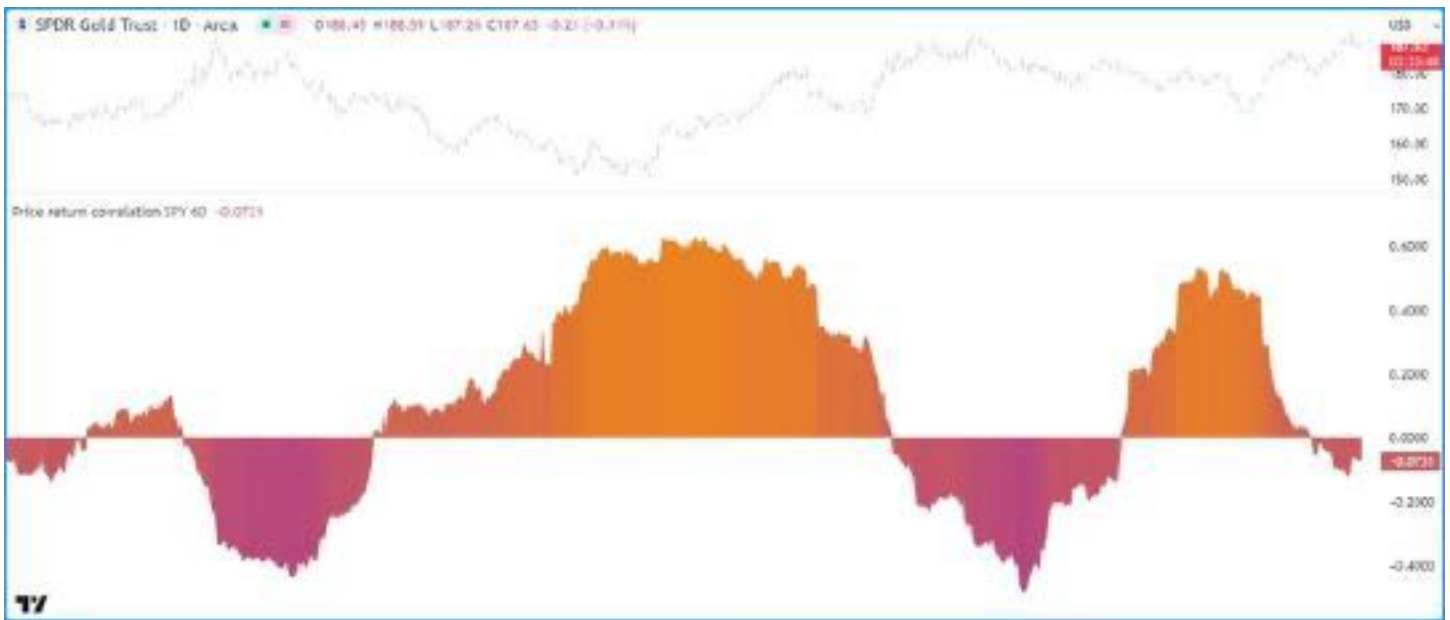


Figure 182: image

```
//@variable The close-to-close price return calculated on another `symbol`.
float requestedReturn = request.security(symbol, timeframe.period, priceReturn)

//@variable The correlation between the `priceReturn` and `requestedReturn` over `length` bars.
float correlation = ta.correlation(priceReturn, requestedReturn, length)
//@variable The color of the correlation plot.
color plotColor = color.from_gradient(correlation, -1, 1, color.purple, color.orange)

// Plot the correlation value.
plot(correlation, "Correlation", plotColor, style = plot.style_area)
```

Note that:

- The `request.security()` call executes the same calculation used in the `priceReturn` declaration, but the request's calculation operates on the close values from the specified symbol's data.
- The script colors the plot with a gradient based on the `correlation` value. See this section of the Colors page to learn more about color gradients.

When using a variable as the `expression` argument of a `request.*()` call, it's important to note that the function only duplicates code that affects the variable *before* the call. It *cannot* copy any subsequent code following the call. Consequently, if the script reassigns the variable or modifies its referenced data *after* calling `request.security()`, the code evaluated on the requested data **does not** include those additional operations.

For example, the following script declares a `counter` variable and calls `request.security()` to evaluate the variable from the same context as the chart. After the call, the script increments the `counter` value by one with the addition assignment operator (`+=`), then uses plots and Pine Logs to display the `counter` and `requestedCounter` values for comparison.

As shown below, the plots and logs of the two variables display *different* values. The `requestedCounter` variable has a consistent value of 0 because the `request.security()` call only evaluates the initial variable declaration. The request cannot evaluate the addition assignment operation because the script includes that code *after* the function call:

```
//@version=6
indicator("Modifying variables after requests demo")

//@variable A counter that starts at 0 and increments by 1 on each bar.
var int counter = 0

//@variable Holds a consistent value of 0.
//      `request.security()` cannot evaluate `counter += 1` in its requested context
//      because that modification occurs after the call.
```




Figure 183: image

```
int requestedCounter = request.security(syminfo.tickerid, timeframe.period, counter)

// Increment the `counter` by 1. This operation is *not* included in the `requestedCounter` calculation.
counter += 1

// Plot both variables for comparison.
plot(counter, "Original counter", color.purple, 3)
plot(requestedCounter, "Requested counter", color.red, 3)

// Log the values of both variables in the Pine Logs pane.
if barstate.isconfirmed
    log.info("counter: {0}, requestedCounter: {1}", counter, requestedCounter)
```

Tuples Tuples in Pine Script™ are comma-separated sets of expressions enclosed in brackets that can hold multiple values of any available type. We use tuples when creating functions or other local blocks that return more than one value.

The `request.security()` function can accept a tuple as its **expression** argument, allowing scripts to request multiple series of different types using a single function call. The expressions within requested tuples can be of any type outlined throughout the Requestable data section of this page, excluding other tuples.

Tuples are particularly handy when a script needs to retrieve more than one value from a specific context.

For example, this script calculates the percent rank of the close price over `length` bars and assigns the expression to the `rank` variable. It then calls `request.security()` to request a tuple containing the `rank`, `ta.crossover(rank, 50)`, and `ta.crossunder(rank, 50)` values from the specified `timeframe`. The script plots the `requestedRank` and uses the `crossOver` and `crossUnder` “bool” values within `bgcolor()` to conditionally highlight the chart pane’s background:

```
//@version=6
indicator("Requesting tuples demo", "Percent rank cross")

//@variable The timeframe of the request.
string timeframe = input.timeframe("240", "Timeframe")
//@variable The number of bars in the calculation.
int length = input.int(20, "Length")

//@variable The previous bar's percent rank of the `close` price over `length` bars.
float rank = ta.percentrank(close, length)[1]

// Request the `rank` value from another `timeframe`, and two "bool" values indicating the `rank` from the `ti
// crossed over or under 50.
[requestedRank, crossOver, crossUnder] = request.security(
```



Figure 184: image

```

syminfo.tickerid, timeframe, [rank, ta.crossover(rank, 50), ta.crossunder(rank, 50)],
lookahead = barmerge.lookahead_on
)

// Plot the `requestedRank` and create a horizontal line at 50.
plot(requestedRank, "Percent Rank", linewidth = 3)
hline(50, "Cross line", linewidth = 2)
// Highlight the background of all bars where the `timeframe`'s `crossOver` or `crossUnder` value is `true`.
bgcolor(crossOver ? color.new(color.green, 50) : crossUnder ? color.new(color.red, 50) : na)

```

Note that:

- We've offset the `rank` variable's expression by one bar using the history-referencing operator `[]` and included `barmerge.lookahead_on` in the `request.security()` call to ensure the values on realtime bars do not repaint after becoming historical bars. See the Avoiding repainting section for more information.
- The `request.security()` call returns a tuple, so we use a *tuple declaration* to declare the `requestedRank`, `crossOver`, and `crossUnder` variables. To learn more about using tuples, see this section of our User Manual's Type system page.

User-defined functions User-defined functions and methods are custom functions written by users. They allow users to define sequences of operations associated with an identifier that scripts can conveniently call throughout their executions (e.g., `myUDF()`).

The `request.security()` function can request the results of user-defined functions and methods whose scopes consist of any types outlined throughout this page's Requestable data section.

For example, this script contains a user-defined `weightedBB()` function that calculates Bollinger Bands with the basis average weighted by a specified `weight` series. The function returns a tuple of custom band values. The script calls the `weightedBB()` as the `expression` argument in `request.security()` to retrieve a tuple of band values calculated on the specified `timeframe` and plots the results on the chart:

```

//@version=6
indicator("Requesting user-defined functions demo", "Weighted Bollinger Bands", true)

//@variable The timeframe of the request.
string timeframe = input.timeframe("480", "Timeframe")

//@function    Calculates Bollinger Bands with a custom weighted basis.
//@param source The series of values to process.
//@param length The number of bars in the calculation.

```

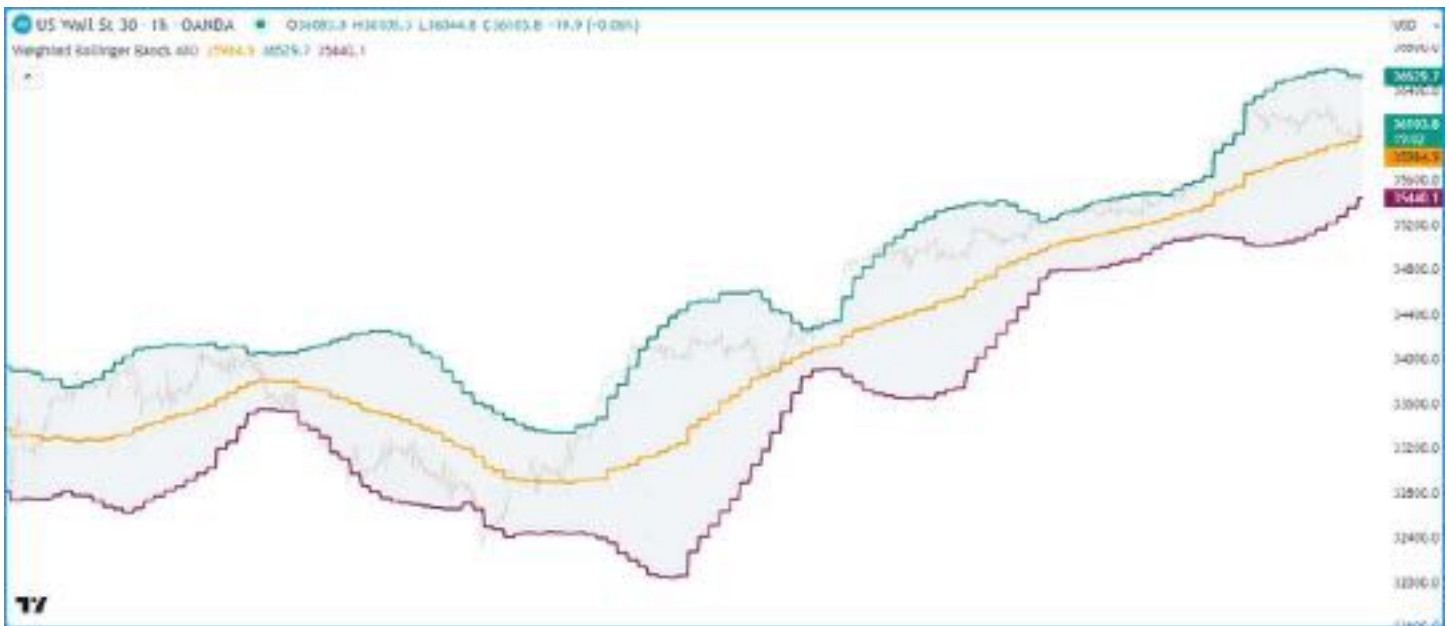



Figure 185: image

```
//@param mult The standard deviation multiplier.
//@param weight The series of weights corresponding to each `source` value.
//@returns A tuple containing the basis, upper band, and lower band respectively.
weightedBB(float source, int length, float mult = 2.0, float weight = 1.0) =>
    //@variable The basis of the bands.
    float ma = math.sum(source * weight, length) / math.sum(weight, length)
    //@variable The standard deviation from the `ma`.
    float dev = 0.0
    // Loop to accumulate squared error.
    for i = 0 to length - 1
        difference = source[i] - ma
        dev += difference * difference
    // Divide `dev` by the `length`, take the square root, and multiply by the `mult`.
    dev := math.sqrt(dev / length) * mult
    // Return the bands.
    [ma, ma + dev, ma - dev]

// Request weighted bands calculated on the chart symbol's prices over 20 bars from the
// last confirmed bar on the `timeframe`.
[basis, highBand, lowBand] = request.security(
    syminfo.tickerid, timeframe, weightedBB(close[1], 20, 2.0, (high - low)[1]), lookahead = barmerge.lookahead
)

// Plot the values.
basisPlot = plot(basis, "Basis", color.orange, 2)
upperPlot = plot(highBand, "Upper", color.teal, 2)
lowerPlot = plot(lowBand, "Lower", color.maroon, 2)
fill(upperPlot, lowerPlot, color.new(color.gray, 90), "Background")
```

Note that:

- We offset the `source` and `weight` arguments in the `weightedBB()` call used as the `expression` in `request.security()` and used `barmerge.lookahead_on` to ensure the requested results reflect the last confirmed values from the `timeframe` on realtime bars. See this section to learn more.

Chart points Chart points are reference types that represent coordinates on the chart. Lines, boxes, polylines, and labels use `chart.point` objects to set their display locations.

The `request.security()` function can use the ID of a `chart.point` instance in its `expression` argument, allowing scripts to retrieve chart coordinates from other contexts.

The example below requests a tuple of historical chart points from a higher timeframe and uses them to draw boxes on the chart. The script declares the `topLeft` and `bottomRight` variables that reference `chart.point` IDs from the last confirmed bar. It then uses `request.security()` to request a tuple containing the IDs of chart points representing the `topLeft` and `bottomRight` from a `higherTimeframe`.

When a new bar starts on the `higherTimeframe`, the script draws a new box using the `time` and `price` coordinates from the `requestedTopLeft` and `requestedBottomRight` chart points:

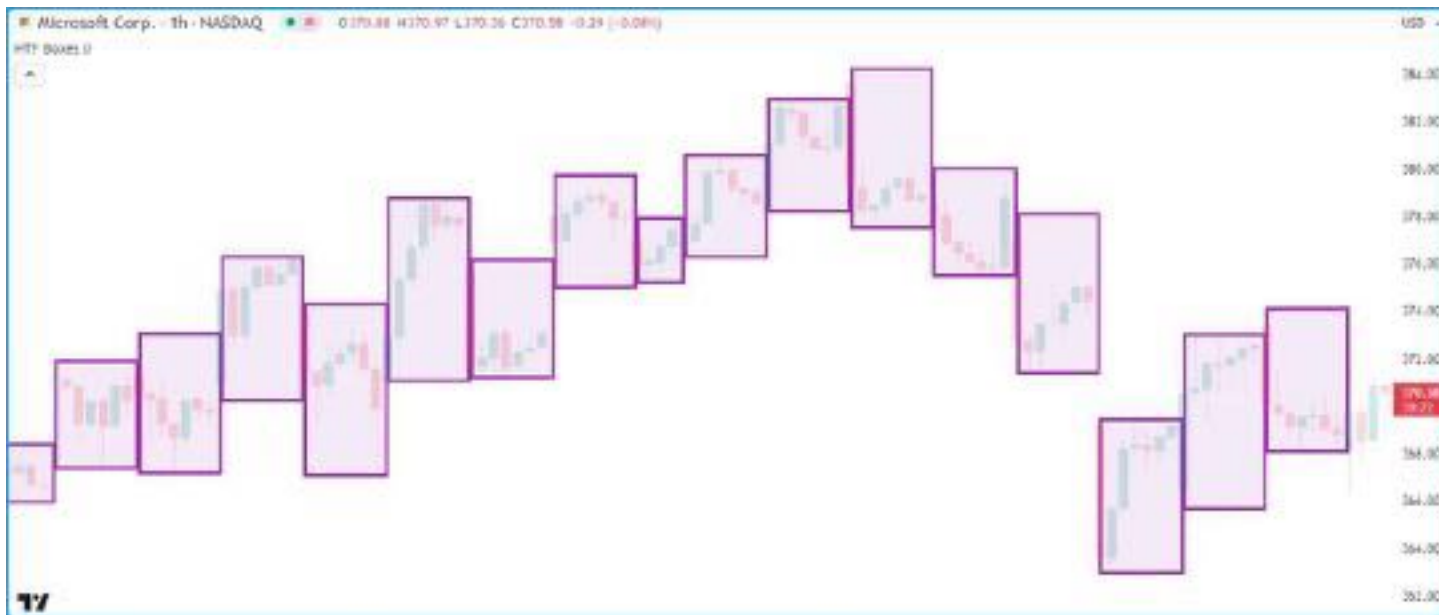


Figure 186: image

```
//@version=6
indicator("Requesting chart points demo", "HTF Boxes", true, max_boxes_count = 500)

//@variable The timeframe to request data from.
string higherTimeframe = input.timeframe("1D", "Timeframe")

// Raise a runtime error if the `higherTimeframe` is smaller than the chart's timeframe.
if timeframe.in_seconds(higherTimeframe) < timeframe.in_seconds(timeframe.period)
    runtime.error("The selected timeframe is too small. Choose a higher timeframe.")

//@variable A `chart.point` containing top-left coordinates from the last confirmed bar.
topLeft = chart.point.now(high)[1]
//@variable A `chart.point` containing bottom-right coordinates from the last confirmed bar.
bottomRight = chart.point.from_time(time_close, low)[1]

// Request the last confirmed `topLeft` and `bottomRight` chart points from the `higherTimeframe`.
[requestedTopLeft, requestedBottomRight] = request.security(
    syminfo.tickerid, higherTimeframe, [topLeft, bottomRight], lookahead = barmerge.lookahead_on
)

// Draw a new box when a new `higherTimeframe` bar starts.
// The box uses the `time` fields from the `requestedTopLeft` and `requestedBottomRight` as x-coordinates.
if timeframe.change(higherTimeframe)
    box.new(
        requestedTopLeft, requestedBottomRight, color.purple, 3,
        xloc = xloc.bar_time, bgcolor = color.new(color.purple, 90)
    )
```

Note that:

- Since this example is designed specifically for higher timeframes, we've included a custom runtime error that the script raises when the `timeframe.in_seconds()` of the `higherTimeframe` is smaller than that of the chart's timeframe.

Collections Pine Script™ *collections* (arrays, matrices, and maps) are data structures that contain an arbitrary number of elements with specified types. The `request.security()` function can retrieve the IDs of collections whose elements consist of:

- Fundamental types
- Chart points
- User-defined types that satisfy the criteria listed in the section below

This example calculates the ratio of a confirmed bar's high-low range to the range between the highest and lowest values over 10 bars from a specified `symbol` and `timeframe`. It uses maps to hold the values used in the calculations.

The script creates a `data` map with "string" keys and "float" values to hold high, low, highest, and lowest price values on each bar, which it uses as the `expression` in `request.security()` to calculate an `otherData` map representing the `data` from the specified context. It uses the values associated with the "High", "Low", "Highest", and "Lowest" keys of the `otherData` map to calculate the `ratio` that it plots in the chart pane:

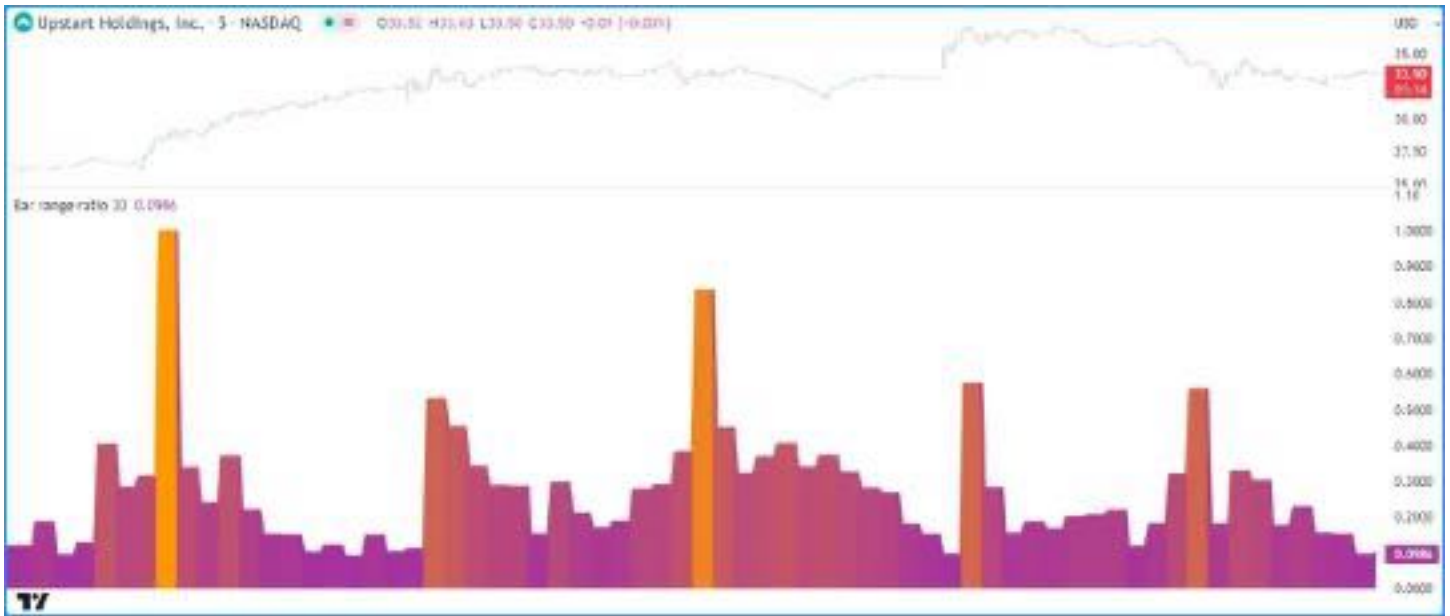


Figure 187: image

```
//@version=6
indicator("Requesting collections demo", "Bar range ratio")

//@variable The ticker ID to request data from.
string symbol = input.symbol("", "Symbol")
//@variable The timeframe of the request.
string timeframe = input.timeframe("30", "Timeframe")

//@variable A map with "string" keys and "float" values.
var map<string, float> data = map.new<string, float>()

// Put key-value pairs into the `data` map.
map.put(data, "High", high)
map.put(data, "Low", low)
map.put(data, "Highest", ta.highest(10))
map.put(data, "Lowest", ta.lowest(10))

//@variable A new `map` whose data is calculated from the last confirmed bar of the requested context.
map<string, float> otherData = request.security(symbol, timeframe, data[1], lookahead = barmerge.lookahead_on)

//@variable The ratio of the context's bar range to the max range over 10 bars. Returns `na` if no data is available
```

```

float ratio = na
if not na(otherData)
    ratio := (otherData.get("High") - otherData.get("Low")) / (otherData.get("Highest") - otherData.get("Lowest"))

//@variable A gradient color for the plot of the `ratio`.
color ratioColor = color.from_gradient(ratio, 0, 1, color.purple, color.orange)

// Plot the `ratio`.
plot(ratio, "Range Ratio", ratioColor, 3, plot.style_area)

```

Note that:

- The `request.security()` call in this script can return `na` if no data is available from the specified context. Since one cannot call methods on a map variable when its value is `na`, we've added an if structure to only calculate a new `ratio` value when `otherData` references a valid map ID.

User-defined types User-defined types (UDTs) are *composite types* containing an arbitrary number of *fields*, which can be of any available type, including other user-defined types.

The `request.security()` function can retrieve the IDs of objects produced by UDTs from other contexts if their fields consist of:

- Fundamental types
- Chart points
- Collections that satisfy the criteria listed in the section above
- Other UDTs whose fields consist of any of these types

The following example requests an object ID using a specified `symbol` and displays its field values on a chart pane.

The script contains a `TickerInfo` UDT with “string” fields for `syminfo.*` values, an array field to store recent “float” price data, and an “int” field to hold the requested ticker’s `bar_index` value. It assigns a new `TickerInfo` ID to an `info` variable on every bar and uses the variable as the `expression` in `request.security()` to retrieve the ID of an object representing the calculated `info` from the specified `symbol`.

The script displays the `requestedInfo` object’s `description`, `tickerType`, `currency`, and `barIndex` values in a label and uses `plotcandle()` to display the values from its `prices` array:



Figure 188: image

```

//@version=6
indicator("Requesting user-defined types demo", "Ticker info")

//@variable The symbol to request information from.

```

```

string symbol = input.symbol("NASDAQ:AAPL", "Symbol")

//@type          A custom type containing information about a ticker.
//@field description The symbol's description.
//@field tickerType The type of ticker.
//@field currency   The symbol's currency.
//@field prices     An array of the symbol's current prices.
//@field barIndex   The ticker's `bar_index`.
type TickerInfo
    string      description
    string      tickerType
    string      currency
    array<float> prices
    int         barIndex

//@variable A `TickerInfo` object containing current data.
info = TickerInfo.new(
    syminfo.description, syminfo.type, syminfo.currency, array.from(open, high, low, close), bar_index
)
//@variable The `info` requested from the specified `symbol`.
TickerInfo requestedInfo = request.security(symbol, timeframe.period, info)
// Assign a new `TickerInfo` instance to `requestedInfo` if one wasn't retrieved.
if na(requestedInfo)
    requestedInfo := TickerInfo.new(prices = array.new<float>(4))

//@variable A label displaying information from the `requestedInfo` object.
var infoLabel = label.new(
    na, na, "", color = color.purple, style = label.style_label_left, textcolor = color.white, size = size.la
)
//@variable The text to display inside the `infoLabel`.
string infoText = na(requestedInfo) ? "" : str.format(
    "{0}\nType: {1}\nCurrency: {2}\nBar Index: {3}",
    requestedInfo.description, requestedInfo.tickerType, requestedInfo.currency, requestedInfo.barIndex
)

// Set the `point` and `text` of the `infoLabel`.
label.set_point(infoLabel, chart.point.now(array.last(requestedInfo.prices)))
label.set_text(infoLabel, infoText)
// Plot candles using the values from the `prices` array of the `requestedInfo`.
plotcandle(
    requestedInfo.prices.get(0), requestedInfo.prices.get(1), requestedInfo.prices.get(2), requestedInfo.prices.get(3),
    "Requested Prices"
)

```

Note that:

- The `syminfo.*` variables used in this script all return “simple string” qualified types. However, objects in Pine are *always* qualified as “series”. Consequently, all values assigned to the `info` object’s fields automatically adopt the “series” qualifier.
- It is possible for the `request.security()` call to return `na` due to differences between the data requested from the `symbol` and the main chart. This script assigns a new `TickerInfo` object to the `requestedInfo` in that case to prevent runtime errors.

request.security_lower_tf()

The `request.security_lower_tf()` function is an alternative to `request.security()` designed for reliably requesting information from lower-timeframe (LTF) contexts.

While `request.security()` can retrieve data from a *single* intrabar (LTF bar) in each chart bar, `request.security_lower_tf()` retrieves data from *all* available intrabars in each chart bar, which the script can access and use in additional calculations. Each `request.security_lower_tf()` call can retrieve up to 200,000 intrabars from a lower timeframe, depending on the user’s plan. See this section of our Limitations page for more information.

Below is the function's signature, which is similar to `request.security()`:

```
request.security_lower_tf(symbol, timeframe, expression, ignore_invalid_symbol, currency, ignore_invalid_timeframe)
```

This function **only** requests data from timeframes less than or equal to the chart's timeframe. If the `timeframe` of the request represents a higher timeframe than the chart's timeframe, the function will either raise a runtime error or return `na` values depending on the `ignore_invalid_timeframe` argument in the call. The default value for this parameter is `false`, meaning it raises an error and halt the script's executions when attempting to request HTF data.

Requesting intrabar data

Intrabar data can provide a script with additional information that may not be obvious or accessible from solely analyzing data sampled on the chart's timeframe. The `request.security_lower_tf()` function can retrieve many data types from an intrabar context.

Before you venture further in this section, we recommend exploring the Requestable data portion of the `request.security()` section above, which provides foundational information about the types of data one can request. The `expression` parameter in `request.security_lower_tf()` accepts most of the same arguments discussed in that section, excluding direct references to collections and mutable variables declared in the script's main scope. Although it accepts many of the same types of arguments, this function returns array results, which comes with some differences in interpretation and handling, as explained below.

Intrabar data arrays

Lower timeframes contain more data points than higher timeframes, as new values come in at a *higher frequency*. For example, when comparing a 1-minute chart to an hourly chart, the 1-minute chart will have up to 60 times the number of bars per hour, depending on the available data.

To address the fact that multiple intrabars exist within a chart bar, `request.security_lower_tf()` always returns its results as arrays. The elements in the returned arrays represent the `expression` values retrieved from the lower timeframe sorted in ascending order based on each intrabar's timestamp.

The type template assigned to the returned arrays corresponds to the value types passed in the `request.security_lower_tf()` call. For example, using an `"int"` as the `expression` will produce an `array<int>` instance, a `"bool"` as the `expression` will produce an `array<bool>` instance, etc.

The following script uses intrabar information to decompose the chart's close-to-close price changes into positive and negative parts. It calls `request.security_lower_tf()` to fetch a `"float"` array of `ta.change(close)` values from the `lowerTimeframe` on each chart bar, then accesses all the array's elements using a `for...in` loop to accumulate `positiveChange` and `negativeChange` sums. The script adds the accumulated values to calculate the `netChange`, then plots the results on the chart alongside the `priceChange` for comparison:



Figure 189: image


```

//@version=6
indicator("Intrabar arrays demo", "Intrabar price changes")

//@variable The lower timeframe of the requested data.
string lowerTimeframe = input.timeframe("1", "Timeframe")

//@variable The close-to-close price change.
float priceChange = ta.change(close)

//@variable An array of `close` values from available intrabars on the `lowerTimeframe`.
array<float> intrabarChanges = request.security_lower_tf(syminfo.tickerid, lowerTimeframe, priceChange)

//@variable The total positive intrabar `close` movement on the chart bar.
float positiveChange = 0.0
//@variable The total negative intrabar `close` movement on the chart bar.
float negativeChange = 0.0

// Loop to calculate totals, starting from the chart bar's first available intrabar.
for change in intrabarChanges
    // Add the `change` to `positiveChange` if its sign is 1, and add to `negativeChange` if its sign is -1.
    switch math.sign(change)
        1 => positiveChange += change
        -1 => negativeChange += change

//@variable The sum of `positiveChange` and `negativeChange`. Equals the `priceChange` on bars with available
float netChange = positiveChange + negativeChange

// Plot the `positiveChange`, `negativeChange`, and `netChange`.
plot(positiveChange, "Positive intrabar change", color.teal, style = plot.style_area)
plot(negativeChange, "Negative intrabar change", color.maroon, style = plot.style_area)
plot(netChange, "Net intrabar change", color.yellow, 5)
// Plot the `priceChange` to compare.
plot(priceChange, "Chart price change", color.orange, 2)

```

Note that:

- The plots based on intrabar data may not appear on all available chart bars, as `request.security_lower_tf()` can only access up to the most recent 200,000 intrabars available from the requested context. When executing this function on a chart bar that doesn't have accessible intrabar data, it will return an *empty array*.
- The number of intrabars per chart bar may vary depending on the data available from the context and the chart the script executes on. For example, a provider's 1-minute data feed may not include data for every minute within the 60-minute timeframe due to a lack of trading activity over some 1-minute intervals. To check the number of intrabars retrieved for a chart bar, one can use `array.size()` on the resulting array.
- If the `lowerTimeframe` value is greater than the chart's timeframe, the script will raise a *runtime error*, as we have not supplied an `ignore_invalid_timeframe` argument in the `request.security_lower_tf()` call.

Tuples of intrabar data

When passing a tuple or a function call that returns a tuple as the **expression** argument in `request.security_lower_tf()`, the result is a tuple of arrays with type templates corresponding to the types within the argument. For example, using a `[float, string, color]` tuple as the **expression** will result in `[array<float>, array<string>, array<color>]` data returned by the function. Using a tuple **expression** allows a script to fetch several arrays of intrabar data with a single `request.security_lower_tf()` function call.

The following example requests OHLC data from a lower timeframe and visualizes the current bar's intrabars on the chart using lines and boxes. The script calls `request.security_lower_tf()` with the `[open, high, low, close]` tuple as its **expression** to retrieve a tuple of arrays representing OHLC information from a calculated `lowerTimeframe`. It then uses a for loop to set line coordinates with the retrieved data and current bar indices to display the results next to the current chart bar, providing a "magnified view" of the price movement within the latest candle. It also draws a box around the lines to indicate the chart region occupied by intrabar drawings:

```

//@version=6

```



Figure 190: image

```

indicator("Tuples of intrabar data demo", "Candle magnifier", max_lines_count = 500)

//@variable The maximum number of intrabars to display.
int maxIntrabars = input.int(20, "Max intrabars", 1, 250)
//@variable The width of the drawn candle bodies.
int candleWidth = input.int(20, "Candle width", 2)

//@variable The largest valid timeframe closest to `maxIntrabars` times smaller than the chart timeframe.
string lowerTimeframe = timeframe.from_seconds(math.ceil(timeframe.in_seconds() / maxIntrabars))

//@variable An array of lines to represent intrabar wicks.
var array<line> wicks = array.new<line>()
//@variable An array of lines to represent intrabar bodies.
var array<line> bodies = array.new<line>()
//@variable A box that surrounds the displayed intrabars.
var box magnifierBox = box.new(na, na, na, na, bgcolor = na)

// Fill the `wicks` and `bodies` arrays with blank lines on the first bar.
if barstate.isfirst
    for i = 1 to maxIntrabars
        array.push(wicks, line.new(na, na, na, na, color = color.gray))
        array.push(bodies, line.new(na, na, na, na, width = candleWidth))

//@variable A tuple of "float" arrays containing `open`, `high`, `low`, and `close` prices from the `lowerTimeframe`.
[oData, hData, lData, cData] = request.security_lower_tf(syminfo.tickerid, lowerTimeframe, [open, high, low, close])
//@variable The number of intrabars retrieved from the `lowerTimeframe` on the chart bar.
int numIntrabars = array.size(oData)

if numIntrabars > 0
    // Define the start and end bar index values for intrabar display.
    int startIndex = bar_index + 2
    int endIndex = startIndex + numIntrabars
    // Loop to update lines.
    for i = 0 to maxIntrabars - 1
        line wickLine = array.get(wicks, i)
        line bodyLine = array.get(bodies, i)
        if i < numIntrabars

```

```

//@variable The `bar_index` of the drawing.
int candleIndex = startIndex + i
// Update the properties of the `wickLine` and `bodyLine`.
line.set_xy1(wickLine, startIndex + i, array.get(hData, i))
line.set_xy2(wickLine, startIndex + i, array.get(lData, i))
line.set_xy1(bodyLine, startIndex + i, array.get(oData, i))
line.set_xy2(bodyLine, startIndex + i, array.get(cData, i))
line.set_color(bodyLine, bodyLine.get_y2() > bodyLine.get_y1() ? color.teal : color.maroon)
continue
// Set the coordinates of the `wickLine` and `bodyLine` to `na` if no intrabar data is available at th
line.set_xy1(wickLine, na, na)
line.set_xy2(wickLine, na, na)
line.set_xy1(bodyLine, na, na)
line.set_xy2(bodyLine, na, na)
// Set the coordinates of the `magnifierBox`.
box.set_lefttop(magnifierBox, startIndex - 1, array.max(hData))
box.set_rightbottom(magnifierBox, endIndex, array.min(lData))

```

Note that:

- The script draws each candle using two lines: one to represent wicks and the other to represent the body. Since the script can display up to 500 lines on the chart, we've limited the `maxIntrabars` input to 250.
- The `lowerTimeframe` value is the result of calculating the `math.ceil()` of the `timeframe.in_seconds()` divided by the `maxIntrabars` and converting to a valid timeframe string with `timeframe.from_seconds()`.
- The script sets the top of the box drawing using the `array.max()` of the requested `hData` array, and it sets the box's bottom using the `array.min()` of the requested `lData` array. As we see on the chart, these values correspond to the high and low of the chart bar.

Requesting collections

In some cases, a script may need to request the IDs of collections from an intrabar context. However, unlike `request.security()`, one cannot pass collections or calls to functions that return them as the `expression` argument in a `request.security_lower_tf()` call, as arrays cannot directly reference other collections.

Despite these limitations, it is possible to request collections from lower timeframes, if needed, with the help of *wrapper* types.

To make collections requestable with `request.security_lower_tf()`, we must create a UDT with a field to reference a collection ID. This step is necessary since arrays cannot reference other collections directly but *can* reference UDTs with collection fields:

```

//@type A "wrapper" type to hold an `array<float>` instance.
type Wrapper
    array<float> collection

```

With our `Wrapper` UDT defined, we can now pass the IDs of objects of the UDT to the `expression` parameter in `request.security_lower_tf()`.

A straightforward approach is to call the built-in `*.new()` function as the `expression`. For example, this line of code calls `Wrapper.new()` with `array.from(close)` as its `collection` within `request.security_lower_tf()`:

```

//@variable An array of `Wrapper` IDs requested from the 1-minute timeframe.
array<Wrapper> wrappers = request.security_lower_tf(syminfo.tickerid, "1", Wrapper.new(array.from(close)))

```

Alternatively, we can create a user-defined function or method that returns an object of the UDT and call that function within `request.security_lower_tf()`. For instance, this code calls a custom `newWrapper()` function that returns a `Wrapper` ID as the `expression` argument:

```

//@function Creates a new `Wrapper` instance to wrap the specified `collection`.
newWrapper(array<float> collection) =>
    Wrapper.new(collection)

```

```

//@variable An array of `Wrapper` IDs requested from the 1-minute timeframe.
array<Wrapper> wrappers = request.security_lower_tf(syminfo.tickerid, "1", newWrapper(array.from(close)))

```

The result with either of the above is an array containing `Wrapper` IDs from all available intrabars in the chart bar, which the script can use to reference `Wrapper` instances from specific intrabars and use their `collection` fields in additional operations.

The script below utilizes this approach to collect arrays of intrabar data from a `lowerTimeframe` and uses them to display data from a specific intrabar. Its custom `Prices` type contains a single `data` field to reference `array<float>` instances that hold price data, and the user-defined `newPrices()` function returns the ID of a `Prices` object.

The script calls `request.security_lower_tf()` with a `newPrices()` call as its `expression` argument to retrieve an array of `Prices` IDs from each intrabar in the chart bar, then uses `array.get()` to get the ID from a specified available intrabar, if it exists. Lastly, it uses `array.get()` on the `data` array assigned to that instance and calls `plotcandle()` to display its values on the chart:



Figure 191: image

```
//@version=6
indicator("Requesting LTF collections demo", "Intrabar viewer", true)

//@variable The timeframe of the LTF data request.
string lowerTimeframe = input.timeframe("1", "Timeframe")
//@variable The index of the intrabar to show on each chart bar. 0 is the first available intrabar.
int intrabarIndex = input.int(0, "Intrabar to show", 0)

//@variable A custom type to hold an array of price `data`.
type Prices
    array<float> data

//@function Returns a new `Prices` instance containing current `open`, `high`, `low`, and `close` prices.
newPrices() =>
    Prices.new(array.from(open, high, low, close))

//@variable An array of `Prices` requested from the `lowerTimeframe`.
array<Prices> requestedPrices = request.security_lower_tf(syminfo.tickerid, lowerTimeframe, newPrices())

//@variable The `Prices` ID from the `requestedPrices` array at the `intrabarIndex`, or `na` if not available.
Prices intrabarPrices = array.size(requestedPrices) > intrabarIndex ? array.get(requestedPrices, intrabarIndex) : na
//@variable The `data` array from the `intrabarPrices`, or an array of `na` values if `intrabarPrices` is `na`
array<float> intrabarData = na(intrabarPrices) ? array.new<float>(4, na) : intrabarPrices.data

// Plot the `intrabarData` values as candles.
plotcandle(intrabarData.get(0), intrabarData.get(1), intrabarData.get(2), intrabarData.get(3))
```

Note that:

- The `intrabarPrices` variable only references a `Prices` ID if the size of the `requestedPrices` array is greater than the

`intrabarIndex`, as attempting to use `array.get()` to get an element that doesn't exist will result in an out of bounds error.

- The `intrabarData` variable only references the `data` field from `intrabarPrices` if a valid `Prices` ID exists since a script cannot reference fields of an `na` value.
- The process used in this example is *not* necessary to achieve the intended result. We could instead avoid using UDTs and pass an `[open, high, low, close]` tuple to the `expression` parameter to retrieve a tuple of arrays for further operations, as explained in the previous section.

Custom contexts

Pine Script™ includes multiple `ticker.*()` functions that allow scripts to construct *custom* ticker IDs that specify additional settings for data requests when used as a `symbol` argument in `request.security()` and `request.security_lower_tf()`:

- `ticker.new()` constructs a custom ticker ID from a specified `prefix` and `ticker` with additional `session` and `adjustment` settings.
- `ticker.modify()` constructs a modified form of a specified `tickerid` with additional `session` and `adjustment` settings.
- `ticker.heikinashi()`, `ticker.renko()`, `ticker.pointfigure()`, `ticker.kagi()`, and `ticker.linebreak()` construct a modified form a `symbol` with non-standard chart settings.
- `ticker.inherit()` constructs a new ticker ID for a `symbol` with additional parameters inherited from the `from_tickerid` specified in the function call, allowing scripts to request the `symbol` data with the same modifiers as the `from_tickerid`, including session, dividend adjustment, currency conversion, non-standard chart type, back-adjustment, settlement-as-close, etc.
- `ticker.standard()` constructs a standard ticker ID representing the `symbol` *without* additional modifiers.

Let's explore some practical examples of applying `ticker.*()` functions to request data from custom contexts.

Suppose we want to include dividend adjustment in a stock symbol's prices without enabling the “Adjust data for dividends” option in the “Symbol” section of the chart's settings. We can achieve this in a script by constructing a custom ticker ID for the instrument using `ticker.new()` or `ticker.modify()` with an `adjustment` value of `adjustment.dividends`.

This script creates an `adjustedTickerID` using `ticker.modify()`, uses that ticker ID as the `symbol` in `request.security()` to retrieve a tuple of adjusted price values, then plots the result as candles on the chart. It also highlights the background when the requested prices differ from the prices without dividend adjustment.

As we see on the “NYSE:XOM” chart below, enabling dividend adjustment results in different historical values before the date of the latest dividend:



Figure 192: image

```
//@version=6
indicator("Custom contexts demo 1", "Adjusted prices", true)
```

```
//@variable A custom ticker ID representing the chart's symbol with the dividend adjustment modifier.
string adjustedTickerID = ticker.modify(syminfo.tickerid, adjustment = adjustment.dividends)

// Request the adjusted prices for the chart's symbol.
[o, h, l, c] = request.security(adjustedTickerID, timeframe.period, [open, high, low, close])

//@variable The color of the candles on the chart.
color candleColor = c > o ? color.teal : color.maroon

// Plot the adjusted prices.
plotcandle(o, h, l, c, "Adjusted Prices", candleColor)
// Highlight the background when `c` is different from `close`.
bgcolor(c != close ? color.new(color.orange, 80) : na)
```

Note that:

- If a modifier included in a constructed ticker ID does not apply to the symbol, the script will *ignore* that modifier when requesting data. For instance, this script will display the same values as the main chart on forex symbols such as “EURUSD”.

While the example above demonstrates a simple way to modify the chart’s symbol, a more frequent use case for `ticker.*()` functions is applying custom modifiers to another symbol while requesting data. If a ticker ID referenced in a script already has the modifiers one would like to apply (e.g., adjustment settings, session type, etc.), they can use `ticker.inherit()` to quickly and efficiently add those modifiers to another symbol.

In the example below, we’ve edited the previous script to request data for a `symbolInput` using modifiers inherited from the `adjustedTickerID`. This script calls `ticker.inherit()` to construct an `inheritedTickerID` and uses that ticker ID in a `request.security()` call. It also requests data for the `symbolInput` without additional modifiers and plots candles for both ticker IDs in a separate chart pane to compare the difference.

As shown on the chart, the data requested using the `inheritedTickerID` includes dividend adjustment, whereas the data requested using the `symbolInput` directly does not:



Figure 193: image

```
//@version=6
indicator("Custom contexts demo 2", "Inherited adjustment")

//@variable The symbol to request data from.
string symbolInput = input.symbol("NYSE:PFE", "Symbol")

//@variable A custom ticker ID representing the chart's symbol with the dividend adjustment modifier.
```



```

string adjustedTickerID = ticker.modify(syminfo.tickerid, adjustment = adjustment.dividends)
//@variable A custom ticker ID representing the `symbolInput` with modifiers inherited from the `adjustedTickerID`
string inheritedTickerID = ticker.inherit(adjustedTickerID, symbolInput)

// Request prices using the `symbolInput`.
[o1, h1, l1, c1] = request.security(symbolInput, timeframe.period, [open, high, low, close])
// Request prices using the `inheritedTickerID`.
[o2, h2, l2, c2] = request.security(inheritedTickerID, timeframe.period, [open, high, low, close])

//@variable The color of the candles that use the `inheritedTickerID` prices.
color candleColor = c2 > o2 ? color.teal : color.maroon

// Plot the `symbol` prices.
plotcandle(o1, h1, l1, c1, "Symbol", color.gray, color.gray, bordercolor = color.gray)
// Plot the `inheritedTickerID` prices.
plotcandle(o2, h2, l2, c2, "Symbol With Modifiers", candleColor)
// Highlight the background when `c1` is different from `c2`.
bgcolor(c1 != c2 ? color.new(color.orange, 80) : na)

```

Note that:

- Since the `adjustedTickerID` represents a modified form of the `syminfo.tickerid`, if we modify the chart's context in other ways, such as changing the chart type or enabling extended trading hours in the chart's settings, those modifiers will also apply to the `adjustedTickerID` and `inheritedTickerID`. However, they will *not* apply to the `symbolInput` since it represents a *standard* ticker ID.

Another frequent use case for requesting custom contexts is retrieving data that uses non-standard chart calculations. For example, suppose we want to use Renko price values to calculate trade signals in a `strategy()` script. If we simply change the chart type to “Renko” to get the prices, the strategy will also simulate its trades based on those synthetic prices, producing misleading results:

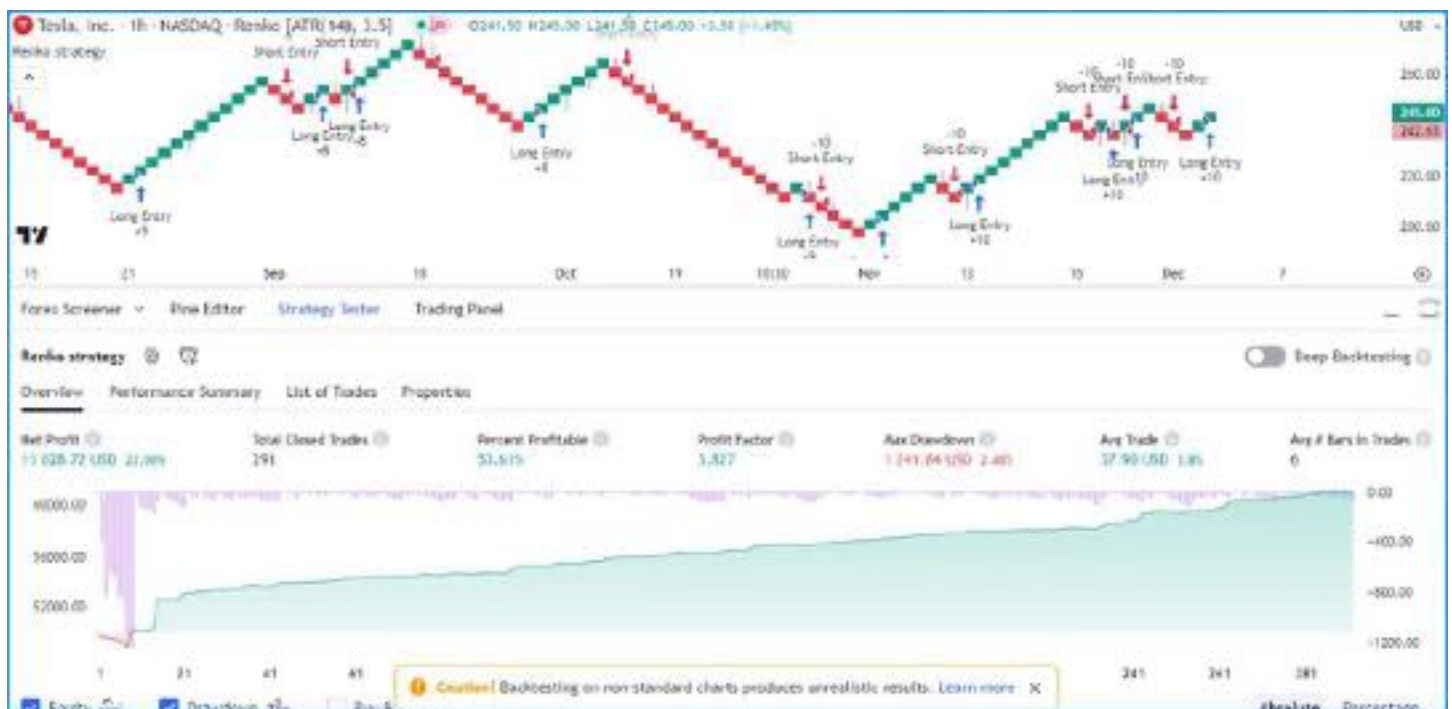


Figure 194: image

```

//@version=6
strategy(
    "Custom contexts demo 3", "Renko strategy", true, default_qty_type = strategy.percent_of_equity,
    default_qty_value = 2, initial_capital = 50000, slippage = 2,
    commission_type = strategy.commission.cash_per_contract, commission_value = 1, margin_long = 100,
    margin_short = 100

```

```

)

//@variable When `true`, the strategy places a long market order.
bool longEntry = ta.crossover(close, open)
//@variable When `true`, the strategy places a short market order.
bool shortEntry = ta.crossunder(close, open)

if longEntry
    strategy.entry("Long Entry", strategy.long)
if shortEntry
    strategy.entry("Short Entry", strategy.short)

```

To ensure our strategy shows results based on *actual* prices, we can create a Renko ticker ID using `ticker.renko()` while keeping the chart on a *standard type*, allowing the script to request and use Renko prices to calculate its signals without calculating the strategy results on them:



Figure 195: image

```

//@version=6
strategy(
    "Custom contexts demo 3", "Renko strategy", true, default_qty_type = strategy.percent_of_equity,
    default_qty_value = 2, initial_capital = 50000, slippage = 1,
    commission_type = strategy.commission.cash_per_contract, commission_value = 1, margin_long = 100,
    margin_short = 100
)

//@variable A Renko ticker ID.
string renkoTickerID = ticker.renko(syminfo.tickerid, "ATR", 14)
// Request the `open` and `close` prices using the `renkoTickerID`.
[renkoOpen, renkoClose] = request.security(renkoTickerID, timeframe.period, [open, close])

//@variable When `true`, the strategy places a long market order.
bool longEntry = ta.crossover(renkoClose, renkoOpen)
//@variable When `true`, the strategy places a short market order.
bool shortEntry = ta.crossunder(renkoClose, renkoOpen)

if longEntry
    strategy.entry("Long Entry", strategy.long)

```

```

if shortEntry
    strategy.entry("Short Entry", strategy.short)

plot(renkoOpen)
plot(renkoClose)

```

Historical and realtime behavior

Functions in the `request.*()` namespace can behave differently on historical and realtime bars. This behavior is closely related to Pine's Execution model.

Consider how a script behaves within the main context. Throughout the chart's history, the script calculates its required values once and *commits* them to that bar so their states are accessible on subsequent executions. On an unconfirmed bar, however, the script recalculates its values on *each update* to the bar's data to align with realtime changes. Before recalculating the values on that bar, it reverts calculated values to their last committed states, otherwise known as *rollback*, and it only commits values to that bar once the bar closes.

Now consider the behavior of data requests from other contexts with `request.security()`. As when evaluating historical bars in the main context, `request.security()` only returns new historical values when it confirms a bar in its specified context. When executing on realtime bars, it returns recalculated values on each chart bar, similar to how a script recalculates values in the main context on the open chart bar.

However, the function only *confirms* the requested values when a bar from its context closes. When the script restarts, what were previously *realtime* bars become *historical* bars. Therefore, `request.security()` only returns the values it confirmed on those bars. In essence, this behavior means that requested data may *repaint* when its values fluctuate on realtime bars without confirmation from the context.

In most circumstances where a script requests data from a broader context, one will typically require confirmed, stable values that *do not* fluctuate on realtime bars. The section below explains how to achieve such a result and avoid repainting data requests.

Avoiding Repainting

Higher-timeframe data When requesting values from a higher timeframe, they are subject to repainting since realtime bars can contain *unconfirmed* information from developing HTF bars, and the script may adjust the times that new values come in on historical bars. To avoid repainting HTF data, one must ensure that the function only returns confirmed values with consistent timing on all bars, regardless of bar state.

The most reliable approach to achieve non-repainting results is to use an **expression** argument that only references past bars (e.g., `close[1]`) while using `barmerge.lookahead_on` as the `lookahead` value.

Using `barmerge.lookahead_on` with non-offset HTF data requests is discouraged since it prompts `request.security()` to "look ahead" to the final values of an HTF bar, retrieving confirmed values *before* they're actually available in the script's history. However, if the values used in the **expression** are offset by at least one bar, the "future" data the function retrieves is no longer from the future. Instead, the data represents confirmed values from established, *available* HTF bars. In other words, applying an offset to the **expression** effectively prevents the requested data from repainting when the script restarts its executions and eliminates lookahead bias in the historical series.

The following example demonstrates a repainting HTF data request. The script uses `request.security()` without offset modifications or additional arguments to retrieve the results of a `ta.wma()` call from a higher timeframe. It also highlights the background to indicate which bars were in a realtime state during its calculations.

As shown on the chart below, the plot of the requested WMA only changes on historical bars when HTF bars close, whereas it fluctuates on all realtime bars since the data includes unconfirmed values from the higher timeframe:

```

//@version=6
indicator("Avoiding HTF repainting demo", overlay = true)

//@variable The multiplier applied to the chart's timeframe.
int tfMultiplier = input.int(10, "Timeframe multiplier", 1)
//@variable The number of bars in the moving average.
int length = input.int(5, "WMA smoothing length")

//@variable The valid timeframe string closest to `tfMultiplier` times larger than the chart timeframe.
string timeframe = timeframe.from_seconds(timeframe.in_seconds() * tfMultiplier)

```



Figure 196: image

```
//@variable The weighted MA of `close` prices over `length` bars on the `timeframe`.
//          This request repaints because it includes unconfirmed HTF data on realtime bars and it may offset
//          times of its historical results.
float requestedWMA = request.security(syminfo.tickerid, timeframe, ta.wma(close, length))
```

```
// Plot the requested series.
plot(requestedWMA, "HTF WMA", color.purple, 3)
// Highlight the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 70) : na, title = "Realtime bar highlight")
```

To avoid repainting in this script, we can add `lookahead = barmerge.lookahead_on` to the `request.security()` call and offset the call history of `ta.wma()` by one bar with the history-referencing operator `[]`, ensuring the request always retrieves the last confirmed HTF bar's WMA at the start of each new `timeframe`. Unlike the previous script, this version has consistent behavior on historical and realtime bar states, as we see below:



Figure 197: image

```
//@version=6
indicator("Avoiding HTF repainting demo", overlay = true)

//@variable The multiplier applied to the chart's timeframe.
int tfMultiplier = input.int(10, "Timeframe multiplier", 1)
//@variable The number of bars in the moving average.
int length = input.int(5, "WMA smoothing length")

//@variable The valid timeframe string closest to `tfMultiplier` times larger than the chart timeframe.
string timeframe = timeframe.from_seconds(timeframe.in_seconds() * tfMultiplier)

//@variable The weighted MA of `close` prices over `length` bars on the `timeframe`.
//      This request does not repaint, as it always references the last confirmed WMA value on all bars.
float requestedWMA = request.security(
    syminfo.tickerid, timeframe, ta.wma(close, length)[1], lookahead = barmerge.lookahead_on
)

// Plot the requested value.
plot(requestedWMA, "HTF WMA", color.purple, 3)
// Highlight the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 70) : na, title = "Realtime bar highlight")
```

Lower-timeframe data The `request.security()` and `request.security_lower_tf()` functions can retrieve data from lower-timeframe contexts. The `request.security()` function can only retrieve data from a *single* intrabar in each chart bar, and `request.security_lower_tf()` retrieves data from *all* available intrabars.

When using these functions to retrieve intrabar data, it's important to note that such requests are **not** immune to repainting behavior. Historical and realtime series often rely on *separate* data feeds. Data providers may retroactively modify realtime data, and it's possible for races to occur in realtime data feeds, as explained in the Data feeds section of this page. Either case may result in intrabar data retrieved on realtime bars repainting after the script restarts its executions.

Additionally, a particular case that *will* cause repainting LTF requests is using `request.security()` with `barmerge.lookahead_on` to retrieve data from the first intrabar in each chart bar. While it will generally work as expected on historical bars, it will track only the most recent intrabar on realtime bars, as `request.security()` does not retain all intrabar information, and the intrabars the function retrieves on realtime bars are unsorted until restarting the script:



Figure 198: image

```
//@version=6
indicator("Avoiding LTF repainting demo", overlay = true)
```

```
//@variable The lower timeframe of the requested data.
string lowerTimeframe = input.timeframe("1", "Timeframe")

//@variable The first intrabar `close` requested from the `lowerTimeframe` on each bar.
//          Only works as intended on historical bars.
float requestedClose = request.security(syminfo.tickerid, lowerTimeframe, close, lookahead = barmerge.lookahead)

// Plot the `requestedClose`.
plot(requestedClose, "First intrabar close", linewidth = 3)
// Highlight the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 60) : na, title = "Realtime bar Highlight")
```

One can mitigate this behavior and track the values from the first intrabar, or any available intrabar in the chart bar, by using `request.security_lower_tf()` since it maintains an array of intrabar values ordered by the times they come in. Here, we call `array.first()` on a requested array of intrabar data to retrieve the close price from the first available intrabar in each chart bar:

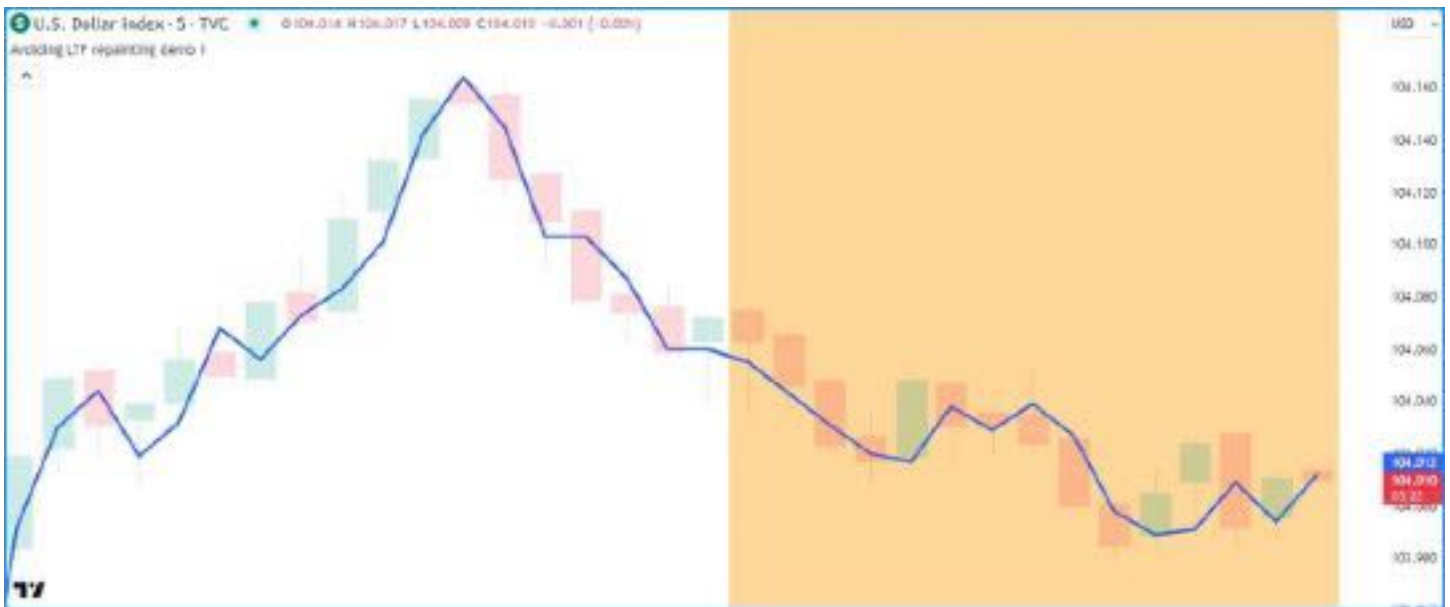


Figure 199: image

```
//@version=6
indicator("Avoiding LTF repainting demo", overlay = true)

//@variable The lower timeframe of the requested data.
string lowerTimeframe = input.timeframe("1", "Timeframe")

//@variable An array of intrabar `close` values requested from the `lowerTimeframe` on each bar.
array<float> requestedCloses = request.security_lower_tf(syminfo.tickerid, lowerTimeframe, close)

//@variable The first intrabar `close` on each bar with available data.
float firstClose = requestedCloses.size() > 0 ? requestedCloses.first() : na

// Plot the `firstClose`.
plot(firstClose, "First intrabar close", linewidth = 3)
// Highlight the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 60) : na, title = "Realtime bar Highlight")
```

Note that:

- While `request.security_lower_tf()` is more optimized for handling historical and realtime intrabars, it's still possible in some cases for minor repainting to occur due to data differences from the provider, as outlined above.
- This code may not show intrabar data on all available chart bars, depending on how many intrabars each chart bar contains, as `request.*()` functions can retrieve up to 200,000 intrabars from an LTF context. The maximum number

of requestable intrabars depends on the user's plan. See this section of the Limitations page for more information.

request.currency_rate()

When a script needs to convert values expressed in one currency to another, one can use `request.currency_rate()`. This function requests a *daily rate* for currency conversion calculations based on currency pair or spread data from the most popular exchanges, providing a simpler alternative to fetching specific pairs or spreads with `request.security()`.

While one can use `request.security()` to retrieve daily currency rates, its use case is more involved than `request.currency_rate()`, as one needs to supply a valid *ticker ID* for a currency pair or spread to request the rate. Additionally, a historical offset and `barmerge.lookahead_on` are necessary to prevent the results from repainting, as explained in this section.

The `request.currency_rate()` function, on the other hand, only requires *currency codes*. No ticker ID is needed when requesting rates with this function, and it ensures non-repainting results without requiring additional specification.

The function's signature is as follows:

```
request.currency_rate(from, to, ignore_invalid_currency) → series float
```

The `from` parameter specifies the currency to convert, and the `to` parameter specifies the target currency. Both parameters accept "string" values representing valid currency codes (e.g., "USD") or any built-in `currency.*` variable (e.g., `currency.USD`).

When the function cannot calculate a valid conversion rate between the specified `from` and `to` currencies, programmers can decide whether it raises a runtime error or returns `na` via the `ignore_invalid_currency` parameter. The default value is `false`, meaning the function raises a runtime error and halts the script's executions.

The following example demonstrates a simple use case for `request.currency_rate()`. Suppose we want to convert values expressed in Turkish lira (`currency.TRY`) to South Korean won (`currency.KRW`) using a daily conversion rate. If we use `request.security()` to retrieve the rate, we must supply a valid ticker ID and request the last confirmed close from the previous day.

In this case, no valid symbol exists that would allow us to retrieve a conversion rate directly with `request.security()`. Therefore, we first need a ticker ID for a spread that converts TRY to an intermediate currency, such as USD, then converts the intermediate currency to KRW. We can then use that ticker ID within `request.security()` with `close[1]` as the `expression` and `barmerge.lookahead_on` as the `lookahead` value to request a non-repainting daily rate.

Alternatively, we can achieve the same result more simply by calling `request.currency_rate()`. This function does all the heavy lifting for us, only requiring `from` and `to` currency arguments to perform its calculation.

As we see below, both approaches return the same daily rate:

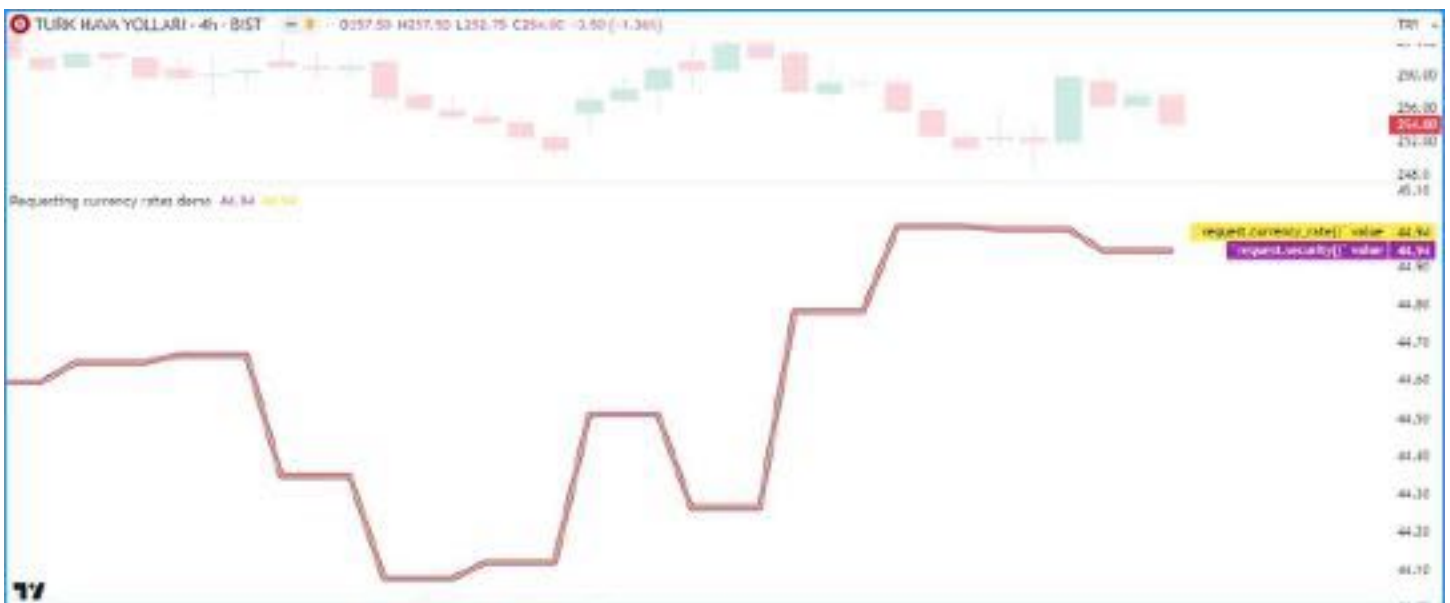


Figure 200: image

```
//@version=6
indicator("Requesting currency rates demo")
```

```

//@variable The currency to convert.
simple string fromCurrency = currency.TRY
//@variable The resulting currency.
simple string toCurrency = currency.KRW

//@variable The spread symbol to request. Required in `request.security()` because no direct symbol exists.
simple string spreadSymbol = str.format("{0}{2} * {2}{1}", fromCurrency, toCurrency, currency.USD)

//@variable The non-repainting conversion rate from `request.security()` using the `spreadSymbol`.
float securityRequestedRate = request.security(spreadSymbol, "1D", close[1], lookahead = barmerge.lookahead_on)
//@variable The non-repainting conversion rate from `request.currency_rate()`.
float nonSecurityRequestedRate = request.currency_rate(fromCurrency, toCurrency)

// Plot the requested rates. We can multiply TRY values by these rates to convert them to KRW.
plot(securityRequestedRate, "`request.security()` value", color.purple, 5)
plot(nonSecurityRequestedRate, "`request.currency_rate()` value", color.yellow, 2)

```

request.dividends(), request.splits(), and request.earnings()

Analyzing a stock's earnings data and corporate actions provides helpful insights into its underlying financial strength. Pine Script™ provides the ability to retrieve essential information about applicable stocks via `request.dividends()`, `request.splits()`, and `request.earnings()`.

These are the functions' signatures:

```

request.dividends(ticker, field, gaps, lookahead, ignore_invalid_symbol, currency) → series float
request.splits(ticker, field, gaps, lookahead, ignore_invalid_symbol) → series float
request.earnings(ticker, field, gaps, lookahead, ignore_invalid_symbol, currency) → series float

```

Each function has the same parameters in its signature, with the exception of `request.splits()`, which doesn't have a `currency` parameter.

Note that unlike the `symbol` parameter in other `request.*()` functions, the `ticker` parameter in these functions only accepts an *“Exchange:Symbol”* pair, such as *“NASDAQ:AAPL”*. The built-in `syminfo.ticker` variable does not work with these functions since it does not contain exchange information. Instead, one must use `syminfo.tickerid` for such cases.

The `field` parameter determines the data the function will retrieve. Each of these functions accepts different built-in variables as the `field` argument since each requests different information about a stock:

- The `request.dividends()` function retrieves current dividend information for a stock, i.e., the amount per share the issuing company paid out to investors who purchased shares before the ex-dividend date. Passing the built-in `dividends.gross` or `dividends.net` variables to the `field` parameter specifies whether the returned value represents dividends before or after factoring in expenses the company deducts from its payouts.
- The `request.splits()` function retrieves current split and reverse split information for a stock. A split occurs when a company increases its outstanding shares to promote liquidity. A reverse split occurs when a company consolidates its shares and offers them at a higher price to attract specific investors or maintain their listing on a market that has a minimum per-share price. Companies express their split information as *ratios*. For example, a 5:1 split means the company issued additional shares to its shareholders so that they have five times the number of shares they had before the split, and the raw price of each share becomes one-fifth of the previous price. Passing `splits.numerator` or `splits.denominator` to the `field` parameter of `request.splits()` determines whether it returns the numerator or denominator of the split ratio.
- The `request.earnings()` function retrieves the earnings per share (EPS) information for a stock `ticker`'s issuing company. The EPS value is the ratio of a company's net income to the number of outstanding stock shares, which investors consider an indicator of the company's profitability. Passing `earnings.actual`, `earnings.estimate`, or `earnings.standardized` as the `field` argument in `request.earnings()` respectively determines whether the function requests the actual, estimated, or standardized EPS value.

For a detailed explanation of the `gaps`, `lookahead`, and `ignore_invalid_symbol` parameters of these functions, see the Common characteristics section at the top of this page.

It's important to note that the values returned by these functions reflect the data available as it comes in. This behavior differs from financial data originating from a `request.financial()` call in that the underlying data from such calls becomes available according to a company's fiscal reporting period.

Here, we've included an example that displays a handy table containing the most recent dividend, split, and EPS data. The script calls the `request.*()` functions discussed in this section to retrieve the data, then converts the values to "strings" with `str.*()` functions and displays the results in the `infoTable` with `table.cell()`:



Figure 201: image

```
//@version=6
indicator("Dividends, splits, and earnings demo", overlay = true)

//@variable The size of the table's text.
string tableSize = input.string(
    size.large, "Table size", [size.auto, size.tiny, size.small, size.normal, size.large, size.huge]
)

//@variable The color of the table's text and frame.
var color tableColor = chart.fg_color
//@variable A `table` displaying the latest dividend, split, and EPS information.
var table infoTable = table.new(position.top_right, 3, 4, frame_color = tableColor, frame_width = 1)

// Add header cells on the first bar.
if barstate.isfirst
    table.cell(infoTable, 0, 0, "Field", text_color = tableColor, text_size = tableSize)
    table.cell(infoTable, 1, 0, "Value", text_color = tableColor, text_size = tableSize)
    table.cell(infoTable, 2, 0, "Date", text_color = tableColor, text_size = tableSize)
    table.cell(infoTable, 0, 1, "Dividend", text_color = tableColor, text_size = tableSize)
    table.cell(infoTable, 0, 2, "Split", text_color = tableColor, text_size = tableSize)
    table.cell(infoTable, 0, 3, "EPS", text_color = tableColor, text_size = tableSize)

//@variable The amount of the last reported dividend as of the current bar.
float latestDividend = request.dividends(syminfo.tickerid, dividends.gross, barmerge.gaps_on)
//@variable The numerator of that last reported split ratio as of the current bar.
float latestSplitNum = request.splits(syminfo.tickerid, splits.numerator, barmerge.gaps_on)
//@variable The denominator of the last reported split ratio as of the current bar.
float latestSplitDen = request.splits(syminfo.tickerid, splits.denominator, barmerge.gaps_on)
//@variable The last reported earnings per share as of the current bar.
float latestEPS = request.earnings(syminfo.tickerid, earnings.actual, barmerge.gaps_on)

// Update the "Value" and "Date" columns when new values come in.
if not na(latestDividend)
    table.cell(
```

```

        infoTable, 1, 1, str.tostring(math.round(latestDividend, 3)), text_color = tableColor, text_size = tableSize
    )
    table.cell(infoTable, 2, 1, str.format_time(time, "yyyy-MM-dd"), text_color = tableColor, text_size = tableSize)
    if not na(latestSplitNum)
        table.cell(
            infoTable, 1, 2, str.format("{0}-for-{1}", latestSplitNum, latestSplitDen), text_color = tableColor,
            text_size = tableSize
        )
    table.cell(infoTable, 2, 2, str.format_time(time, "yyyy-MM-dd"), text_color = tableColor, text_size = tableSize)
    if not na(latestEPS)
        table.cell(infoTable, 1, 3, str.tostring(latestEPS), text_color = tableColor, text_size = tableSize)
        table.cell(infoTable, 2, 3, str.format_time(time, "yyyy-MM-dd"), text_color = tableColor, text_size = tableSize)

```

Note that:

- We’ve included `barmerge.gaps_on` in the `request.*()` calls, so they only return values when new data is available. Otherwise, they return `na`.
- The script assigns a table ID to the `infoTable` variable on the first chart bar. On subsequent bars, it updates necessary cells with new information whenever data is available.
- If no information is available from any of the `request.*()` calls throughout the chart’s history (e.g., if the `ticker` has no dividend information), the script does not initialize the corresponding cells since it’s unnecessary.

request.financial()

Financial metrics provide investors with insights about a company’s economic and financial health that are not tangible from solely analyzing its stock prices. TradingView offers a wide variety of financial metrics from FactSet that traders can access via the “Financials” tab in the “Indicators” menu of the chart. Scripts can access available metrics for an instrument directly via the `request.financial()` function.

This is the function’s signature:

```
request.financial(symbol, financial_id, period, gaps, ignore_invalid_symbol, currency) → series float
```

As with the first parameter in `request.dividends()`, `request.splits()`, and `request.earnings()`, the `symbol` parameter in `request.financial()` requires an “*Exchange:Symbol*” pair. To request financial information for the chart’s ticker ID, use `syminfo.tickerid`, as `syminfo.ticker` will not work.

The `financial_id` parameter accepts a “string” value representing the ID of the requested financial metric. TradingView has numerous financial metrics to choose from. See the Financial IDs section below for an overview of all accessible metrics and their “string” identifiers.

The `period` parameter specifies the fiscal period for which new requested data comes in. It accepts one of the following “string” arguments: “**FQ**” (**quarterly**), “**FH**” (**semiannual**), “**FY**” (**annual**), or “**TTM**” (**trailing twelve months**). Not all fiscal periods are available for all metrics or instruments. To confirm which periods are available for specific metrics, see the second column of the tables in the Financial IDs section.

See this page’s Common characteristics section for a detailed explanation of this function’s `gaps`, `ignore_invalid_symbol`, and `currency` parameters.

It’s important to note that the data retrieved from this function comes in at a *fixed frequency*, independent of the precise date on which the data is made available within a fiscal period. For a company’s dividends, splits, and earnings per share (EPS) information, one can request data reported on exact dates via `request.dividends()`, `request.splits()`, and `request.earnings()`.

This script uses `request.financial()` to retrieve information about the income and expenses of a stock’s issuing company and visualize the profitability of its typical business operations. It requests the “`OPER_INCOME`”, “`TOTAL_REVENUE`”, and “`TOTAL_OPER_EXPENSE`” financial IDs for the `syminfo.tickerid` over the latest `fiscalPeriod`, then plots the results on the chart:

```

//@version=6
indicator("Requesting financial data demo", format = format.volume)

//@variable The size of the fiscal reporting period. Some options may not be available, depending on the instrument
string fiscalPeriod = input.string("FQ", "Period", ["FQ", "FH", "FY", "TTM"])

//@variable The operating income after expenses reported for the stock's issuing company.
float operatingIncome = request.financial(syminfo.tickerid, "OPER_INCOME", fiscalPeriod)

```

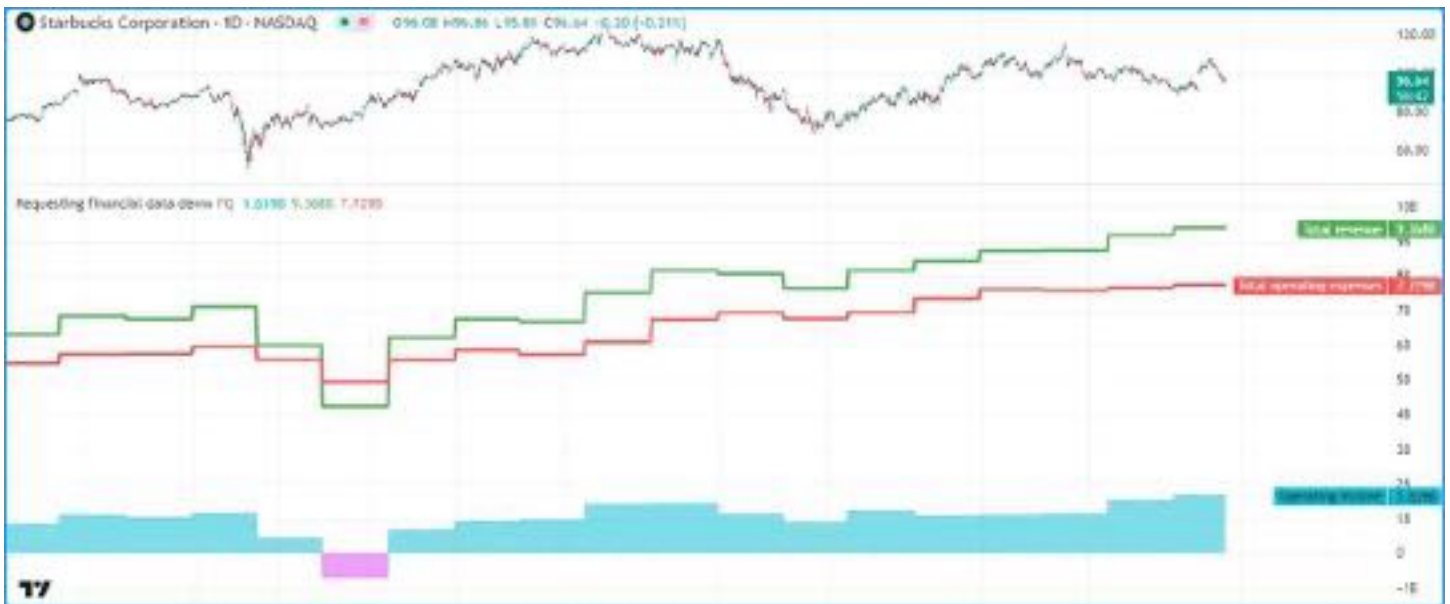


Figure 202: image

```
//@variable The total revenue reported for the stock's issuing company.
float totalRevenue = request.financial(syminfo.tickerid, "TOTAL_REVENUE", fiscalPeriod)
//@variable The total operating expenses reported for the stock's issuing company.
float totalExpenses = request.financial(syminfo.tickerid, "TOTAL_OPER_EXPENSE", fiscalPeriod)

//@variable Is aqua when the `totalRevenue` exceeds the `totalExpenses`, fuchsia otherwise.
color incomeColor = operatingIncome > 0 ? color.new(color.aqua, 50) : color.new(color.fuchsia, 50)

// Display the requested data.
plot(operatingIncome, "Operating income", incomeColor, 1, plot.style_area)
plot(totalRevenue, "Total revenue", color.green, 3)
plot(totalExpenses, "Total operating expenses", color.red, 3)
```

Note that:

- Not all `fiscalPeriod` options are available for every ticker ID. For example, companies in the US typically publish *quarterly* reports, whereas many European companies publish *semiannual* reports. See this page in our Help Center for more information.

Calculating financial metrics

The `request.financial()` function can provide scripts with numerous useful financial metrics that don't require additional calculations. However, some commonly used financial estimates require combining an instrument's current market price with requested financial data. Such is the case for:

- Market Capitalization (market price * total shares outstanding)
- Earnings Yield (12-month EPS / market price)
- Price-to-Book Ratio (market price / BVPS)
- Price-to-Earnings Ratio (market price / EPS)
- Price-to-Sales Ratio (market cap / 12-month total revenue)

The following script contains user-defined functions that calculate the above financial metrics for the `syminfo.tickerid`. We've created these functions so users can easily copy them into their scripts. This example uses them within a `str.format()` call to construct a `tooltipText`, which it displays in tooltips on the chart using labels. Hovering over any bar's label will expose the tooltip containing the metrics calculated on that bar:

```
//@version=6
indicator("Calculating financial metrics demo", overlay = true, max_labels_count = 500)

//@function Calculates the market capitalization (market cap) for the chart's symbol.
```



Figure 203: image

```

marketCap() =>
    //@variable The most recent number of outstanding shares reported for the symbol.
    float totalSharesOutstanding = request.financial(syminfo.tickerid, "TOTAL_SHARES_OUTSTANDING", "FQ")
    // Return the market cap value.
    totalSharesOutstanding * close

//@function Calculates the Earnings Yield for the chart's symbol.
earningsYield() =>
    //@variable The most recent 12-month earnings per share reported for the symbol.
    float eps = request.financial(syminfo.tickerid, "EARNINGS_PER_SHARE", "TTM")
    //Return the Earnings Yield percentage.
    100.0 * eps / close

//@function Calculates the Price-to-Book (P/B) ratio for the chart's symbol.
priceBookRatio() =>
    //@variable The most recent Book Value Per Share (BVPS) reported for the symbol.
    float bookValuePerShare = request.financial(syminfo.tickerid, "BOOK_VALUE_PER_SHARE", "FQ")
    // Return the P/B ratio.
    close / bookValuePerShare

//@function Calculates the Price-to-Earnings (P/E) ratio for the chart's symbol.
priceEarningsRatio() =>
    //@variable The most recent 12-month earnings per share reported for the symbol.
    float eps = request.financial(syminfo.tickerid, "EARNINGS_PER_SHARE", "TTM")
    // Return the P/E ratio.
    close / eps

//@function Calculates the Price-to-Sales (P/S) ratio for the chart's symbol.
priceSalesRatio() =>
    //@variable The most recent number of outstanding shares reported for the symbol.
    float totalSharesOutstanding = request.financial(syminfo.tickerid, "TOTAL_SHARES_OUTSTANDING", "FQ")
    //@variable The most recent 12-month total revenue reported for the symbol.
    float totalRevenue = request.financial(syminfo.tickerid, "TOTAL_REVENUE", "TTM")
    // Return the P/S ratio.
    totalSharesOutstanding * close / totalRevenue

```



```
//@variable The text to display in label tooltips.
string tooltipText = str.format(
    "Market Cap: {0} {1}\nEarnings Yield: {2}%\nP/B Ratio: {3}\nP/E Ratio: {4}\nP/S Ratio: {5}",
    str.toString(marketCap(), format.volume), syminfo.currency, earningsYield(), priceBookRatio(),
    priceEarningsRatio(), priceSalesRatio()
)

//@variable Displays a blank label with a tooltip containing the `tooltipText`.
label info = label.new(chart.point.now(high), tooltip = tooltipText)
```

Note that:

- Since not all companies publish quarterly financial reports, one may need to change the “FQ” in these functions to match the minimum reporting period for a specific company, as the `request.financial()` calls will return na when “FQ” data isn’t available.

Financial IDs

Below is an overview of all financial metrics one can request via `request.financial()`, along with the periods in which reports may be available. We’ve divided this information into four tables corresponding to the categories displayed in the “Financials” section of the “Indicators” menu:

- Income statements
- Balance sheet
- Cash flow
- Statistics

Each table has the following three columns:

- The first column contains descriptions of each metric with links to Help Center pages for additional information.
- The second column lists the possible `period` arguments allowed for the metric. Note that all available values may not be compatible with specific ticker IDs, e.g., while “FQ” may be a possible argument, it will not work if the issuing company does not publish quarterly data.
- The third column lists the “string” IDs for the `financial_id` argument in `request.financial()`.

Income statements This table lists the available metrics that provide information about a company’s income, costs, profits and losses.

Click to show/hide
`Financialperiod` `financial_id`
 After tax other income/expenseFQ, FH, FY, TTMAFTER_TAX_OTHER_INCOME
 basic shares outstandingFQ, FH, FYBASIC_SHARES_OUTSTANDINGBasic earnings per share (Basic EPS)FQ, FH, FY, TTMEARNINGS_PER_SHARE_BASICCost of goods soldFQ, FH, FY, TTMCOST_OF_GOODSDepreciation and amortizationFQ, FH, FY, TTMDEP_AMORT_EXP_INCOME_SDI
 diluted earnings per share (Diluted EPS)FQ, FH, FY, TTMEARNINGS_PER_SHARE_DILUTEDDiluted net income available to common stockholdersFQ, FH, FY, TTMDILUTED_NET_INCOMEDiluted shares outstandingFQ, FH, FYDILUTED_SHARES_OUTSTANDINGDilution adjustmentFQ, FH, FY, TTMDILUTION_ADJUSTMENTDiscontinued operationsFQ, FH, FY, TTMDISCONTINUED_OPERATIONSEBITFQ, FH, FY, TTMEBITEBITDAFQ, FH, FY, TTMEBITDAEEquity in earningsFQ, FH, FY, TTMEQUITY_IN_EARNINGS
 Gross profitFQ, FH, FY, TTMGROSS_PROFITInterest capitalizedFQ, FH, FY, TTMINTEREST_CAPITALIZEDInterest expense on debtFQ, FH, FY, TTMINTEREST_EXPENSE_ON_DEBTInterest expense, net of interest capitalizedFQ, FH, FY, TTMNON_OPER_INTEREST_EXPMiscellaneous non-operating expenseFQ, FH, FY, TTMOTHER_INCOMENet incomeFQ, FH, FY, TTMNET_INCOMENet income before discontinued operationsFQ, FH, FY, TTMNET_INCOME_BEFO
 DISCONTINUED OPERATIONSNon-controlling/minority interestFQ, FH, FY, TTMMINORITY_INTEREST_EXPNNon-operating income, excl. interest expensesFQ, FH, FY, TTMNON_OPER_INCOMENon-operating income, totalFQ, FH, FY, TTMTOTAL_NON_OPER_INCOMENon-operating interest incomeFQ, FH, FY, TTMNON_OPER_INTEREST_INCOMEOperating expenses (excl. COGS)FQ, FH, FY, TTMO
 PERATING_EXPENSESOperating incomeFQ, FH, FY, TTMOPER_INCOMEOther cost of goods soldFQ, FH, FY, TTMCOST_OF_GOODS_EXCL_DEP_AMORTOther operating expenses, totalFQ, FH, FY, TTMO
 THER_OPER_EXPENSE_TOTALPreferred dividendsFQ, FH, FY, TTMPREFERRED_DIVIDENDSPretax equity in earningsFQ, FH, FY, TTMPRETAX_EQUITY_IN_EARNINGS
 Pretax incomeFQ, FH, FY, TTMPRETAX_INCOMEResearch & developmentFQ, FH, FY, TTMRESEARCH_AND_DEV
 Selling/general/admin expenses, otherFQ, FH, FY, TTMSELL_GEN_ADMIN_EXP_OTHERSelling/general/admin expenses, totalFQ, FH, FY, TTMSELL_GEN_ADMIN_EXP_TOTAL
 TaxesFQ, FH, FY, TTMINCOME_TAXTotal operating expensesFQ, FH, FY, TTMTOTAL_OPER_EXPENSETotal revenueFQ, FH, FY, TTMTOTAL_REVENUEUnusual income/expenseFQ, FH, FY, TTMUNUSUAL_EXPENSE_INC##### Balance sheet

This table lists the metrics that provide information about a company's capital structure.

Click to show/hideFinancialperiod`financial_idAccounts payableFQ, FH, FYACCOUNTS_PAYABLEAccounts receivable - trade, netFQ, FH, FYACCOUNTS_RECEIVABLES_NETAccrued payrollFQ, FH, FYACCRUED_PAYROLLAccumulated depreciation, totalFQ, FH, FYACCUM_DEPREC_TOTALAdditional paid-in capital/Capital surplusFQ, FH, FYADDITIONAL_PAID_IN_CAPITALBook value per shareFQ, FH, FYBOOK_VALUE_PER_SHARECapital and operating lease obligationsFQ, FH, FYCAPITAL_OPERATING_LEASE_OBLIGATIONSCapitalized lease obligationsFQ, FH, FYCAPITAL_LEASE_OBLIGATIONSCash & equivalentsFQ, FH, FYCASH_N_EQUIVALENTSCash and short term investmentsFQ, FH, FYCASH_N_SHORT_TERM_INVESTCommon equity, totalFQ, FH, FYCOMMON_EQUITY_TOTALCommon stock par/Carrying valueFQ, FH, FYCOMMON_STOCK_PARCCurrent portion of LT debt and capital leasesFQ, FH, FYCURRENT_PORT_DEBT_CAPITAL_LEASESDeferred income, currentFQ, FH, FYDEFERRED_INCOME_CURRENTDeferred income, non-currentFQ, FH, FYDEFERRED_INCOME_NON_CURRENTDeferred tax assetsFQ, FH, FYDEFERRED_TAX_ASSESTSDDeferred tax liabilitiesFQ, FH, FYDEFERRED_TAX_LIABILITIESDividends payableFYDIVIDENDS_PAYABLEGoodwill, netFQ, FH, FYGOODWILLGross property/plant/equipmentFQ, FH, FYPPE_TOTAL_GROSSIncome tax payableFQ, FH, FYINCOME_TAX_PAYABLEInventories - finished goodsFQ, FH, FYINVENTORY_FINISHED_GOODSInventories - progress payments & otherFQ, FH, FYINVENTORY_PROGRESS_PAYMENTInventories - raw materialsFQ, FH, FYINVENTORY_RAW_MATERIALSInventories - work in progressFQ, FH, FYINVENTORY_WORK_IN_PROGRESSInvestments in unconsolidated subsidiariesFQ, FH, FYINVESTMENTS_IN_UNCONCONSOLIDATED term debtFQ, FH, FYLONG_TERM_DEBTLong term debt excl. lease liabilitiesFQ, FH, FYLONG_TERM_DEBT_EXCL_CAPITAL term investmentsFQ, FH, FYLONG_TERM_INVESTMENTSMinority interestFQ, FH, FYMINORITY_INTERESTNet debtFQ, FH, FYNET_DEBTNet intangible assetsFQ, FH, FYINTANGIBLES_NETNet property/plant/equipmentFQ, FH, FYPPE_TOTAL_NETNote receivable - long termFQ, FH, FYLONG_TERM_NOTE_RECEIVABLENotes payableFYNOTES_PAYABLE_SHORT_TERM_DEBTOperating lease liabilitiesFQ, FH, FYOPERATING_LEASE_LIABILITIESOther common equityFQ, FH, FYOTHER_COMMON_EQUITYOther current assets, totalFQ, FH, FYOTHER_CURRENT_ASSETS_TOTOther current liabilitiesFQ, FH, FYOTHER_CURRENT_LIABILITIESOther intangibles, netFQ, FH, FYOTHER_INTANGIBLES_NETOther investmentsFQ, FH, FYOTHER_INVESTMENTSOther long term assets, totalFQ, FH, FYLONG_TERM_OTHER_ASSETS_TOTOther non-current liabilities, totalFQ, FH, FYOTHER_LIABILITIES_TOTALOther receivablesFQ, FH, FYOTHER_RECEIVABLESOther short term debtFYOTHER_SHORT_TERM_DEBTPaid in capitalFQ, FH, FYPAID_IN_CAPITALPreferred stock, carrying valueFQ, FH, FYPREFERRED_STOCK_CARRYING_VALUEPrepaid expensesFQ, FH, FYPREPAID_EXPENSESProvision for risks & chargeFQ, FH, FYPROVISION_F_RISKSRetained earningsFQ, FH, FYRETAINED_EARNINGSShareholders' equityFQ, FH, FYSHRHLDRS_EQUITYShort term debtFQ, FH, FYSHORT_TERM_DEBTShort term debt excl. current portion of LT debtFQ, FH, FYSHORT_TERM_DEBT_EXCL_CURRENT_PORTShort term investmentsFQ, FH, FYSHORT_TERM_INVESTTangible book value per shareFQ, FH, FYBOOK_TANGIBLE_PER_SHARETotal assetsFQ, FH, FYTOTAL_ASSETSTotal current assetsFQ, FH, FYTOTAL_CURRENT_ASSETSTotal current liabilitiesFQ, FH, FYTOTAL_CURRENT_LIABILITIESTotal debtFQ, FH, FYTOTAL_DEBTTotal equityFQ, FH, FYTOTAL_EQUITYTotal inventoryFQ, FH, FYTOTAL_INVENTORYTotal liabilitiesFQ, FH, FYTOTAL_LIABILITIESTotal liabilities & share- holders' equitiesFQ, FH, FYTOTAL_LIABILITIES_SHRHLDRS_EQUITYTotal non-current assetsFQ, FH, FYTOTAL_NON_CURRENT_ASSETSTotal non-current liabilitiesFQ, FH, FYTOTAL_NON_CURRENT_LIABILITIESTotal receivables, netFQ, FH, FYTOTAL_RECEIVABLES_NETTreasury stock - commonFQ, FH, FYTREASURY_STOCK_COMMONCash flow

This table lists the available metrics that provide information about how cash flows through a company.

Click to show/hideFinancialperiod`financial_idAmortizationFQ, FH, FY, TTMAMORTIZATIONCapital expendituresFQ, FH, FY, TTMCAPITAL_EXPENDITURESCapital expenditures - fixed assetsFQ, FH, FY, TTMCAPITAL_EXPENDITURES_FIXED_ASSETSCapital expenditures - other assetsFQ, FH, FY, TTMCAPITAL_EXPENDITURES_OTHER from financing activitiesFQ, FH, FY, TTMCASH_F_FINANCING_ACTIVITIESCash from investing activitiesFQ, FH, FY, TTMCASH_F_INVESTING_ACTIVITIESCash from operating activitiesFQ, FH, FY, TTMCASH_F_OPERATING_ACTIVITIESChange in accounts payableFQ, FH, FY, TTMCHANGE_IN_ACCOUNTS_PAYABLEChange in accounts receivableFQ, FH, FY, TTMCHANGE_IN_ACCOUNTS_RECEIVABLEChange in accrued expensesFQ, FH, FY, TTMCHANGE_IN_ACCRUED_EXPENSESChange in inventoriesFQ, FH, FY, TTMCHANGE_IN_INVENTORIESChange in other assets/liabilitiesFQ, FH, FY, TTMCHANGE_IN_OTHER_ASSETSCChange in taxes payableFQ, FH, FY, TTMCHANGE_IN_TAXES_PAYABLEChanges in working capitalFQ, FH, FY, TTMCHANGES_IN_WORKING_CAPITALCommon dividends paidFQ, FH, FY, TTMCOMMON_DIVIDENDS_CASH_FLOWDeferred taxes (cash flow)FQ, FH, FY, TTMCASH_FLOW_DEFERRED_TAXESDepreciation & amortization (cash flow)FQ, FH, FY, TTMCASH_FLOW_DEPRECIATION_N_AMORTIZATIONDepreciation/depletionFQ, FH, FY, TTMDEPRECIATION_DEPLETIONFinancing activities - other sourcesFQ, FH, FY, TTMOTHER_FINANCING_CASH_FLOWFinancing activities - other usesFQ, FH, FY, TTMOTHER_FINANCING_CASH_FLOW_USESFree cash flowFQ, FH, FY, TTMFREE_CASH_FLOWFunds from operationsFQ, FH, FY, TTMFUNDS_F_OPERATIONSInvesting activities - other sourcesFQ, FH, FY, TTMOTHER_INVESTING_CASH_FLOW_SOURCESInvesting activities - other usesFQ, FH, FY, FYOTHER_INVESTING_CASH_FLOW_USESIssuance of long term debtFQ, FH, FY, TTMSUPPLYING_OF_LONG_TERM_DEBTIssuance/retirement of debt, netFQ, FH, FY, TTMISSUANCE_OF_DEBT_NETIssuance/retirement of debt, netFQ, FH, FY, TTMISSUANCE_OF_DEBT_NET

of long term debtFQ, FH, FY, TTMISUANCE_OF_LONG_TERM_DEBTIssuance/retirement of other debtFQ, FH, FY, TTMISUANCE_OF_OTHER_DEBTIssuance/retirement of short term debtFQ, FH, FY, TTMISUANCE_OF_SHORT_TERM
of stock, netFQ, FH, FY, TTMISUANCE_OF_STOCK_NETNet income (cash flow)FQ, FH, FY, TTMNET_INCOME_STARTIN
cash itemsFQ, FH, FY, TTMNON_CASH_ITEMSOther financing cash flow items, totalFQ, FH, FY, TTMOTHER_FINANCING_C
investing cash flow items, totalFQ, FH, FYOTHER_INVESTING_CASH_FLOW_ITEMS_TOTALPreferred divi-
dends paidFQ, FH, FYPREFERRED_DIVIDENDS_CASH_FLOWPurchase of investmentsFQ, FH, FY, TTMPUR-
CHASE_OF_INVESTMENTSPurchase/acquisition of businessFQ, FH, FY, TTMPURCHASE_OF_BUSINESSPurchase/sale
of business, netFQ, FH, FYPURCHASE_SALE_BUSINESSPurchase/sale of investments, netFQ, FH, FY, TTMPUR-
CHASE_SALE_INVESTMENTSReduction of long term debtFQ, FH, FY, TTMREDUCTION_OF_LONG_TERM_DEBTRepurc
of common & preferred stockFQ, FH, FY, TTMPURCHASE_OF_STOCKSale of common & preferred stockFQ, FH, FY,
TTMSALE_OF_STOCKSale of fixed assets & businessesFQ, FH, FY, TTMSALES_OF_BUSINESSSale/maturity of invest-
mentsFQ, FH, FYSALES_OF_INVESTMENTSTotal cash dividends paidFQ, FH, FY, TMTOTAL_CASH_DIVIDENDS_PAID#
Statistics

This table contains a variety of statistical metrics, including commonly used financial ratios.

Click to show/hideFinancialperiod`financial_idAccrualsFQ, FH, FYACCRUALS_RATIOAltman Z-scoreFQ, FH, FYALTMAN_Z_SCOREAsset turnoverFQ, FH, FYASSET_TURNOVERBeneish M-scoreFQ, FH, FYBENEISH_M_SCOREBuyRet
yield %FQ, FH, FYBUYBACK_YIELDCOGS to revenue ratioFQ, FH, FYCOGS_TO_REVENUECash conversion cy-
cleFQ, FYCASH_CONVERSION_CYCLECash to debt ratioFQ, FH, FYCASH_TO_DEBTCurrent ratioFQ, FH, FYCURRENT_RATIODays inventoryFQ, FYDAYS_INVENTDays payableFQ, FYDAYS_PAYDays sales outstand-
ingFQ, FYDAY_SALES_OUTDebt to EBITDA ratioFQ, FH, FYDEBT_TO_EBITDADebt to assets ratioFQ, FH, FYDEBT_TO_ASSETDebt to equity ratioFQ, FH, FYDEBT_TO_EQUITYDebt to revenue ratioFQ, FH, FY-
DEBT_TO_REVENUEDividend payout ratio %FQ, FH, FY, TTMDIVIDEND_PAYOUT_RATIODividend yield %FQ, FH, FYDIVIDENDS_YIELDDividends per share - common stock primary issueFQ, FH, FY, TTMDPS_COMMON_STOCK_PRIM
margin %FQ, FH, FY, TTMEBITDA_MARGINEPS basic one year growthFQ, FH, FY, TTMEARNINGS_PER_SHARE_BASIC_C
diluted one year growthFQ, FH, FYEARNINGS_PER_SHARE_DILUTED_ONE_YEAR_GROWTHEPS estimatesFQ, FH, FYEARNINGS_ESTIMATEEEffective interest rate on debt %FQ, FH, FYEFFECTIVE_INTEREST_RATE_ON_DEBTEnterpr
valueFQ, FH, FYENTERPRISE_VALUEEnterprise value to EBIT ratioFQ, FH, FYEV_EBITEnterprise value to EBITDA
ratioFQ, FH, FYENTERPRISE_VALUE_EBITDAEnterprise value to revenue ratioFQ, FH, FYEV_REVENUEEEquity
to assets ratioFQ, FH, FYEQUITY_TO_ASSETFloat shares outstandingFYFLOAT_SHARES_OUTSTANDINGFree
cash flow margin %FQ, FH, FYFREE_CASH_FLOW_MARGINFulmer H factorFQ, FYFULMER_H_FACTORGoodwill
to assets ratioFQ, FH, FYGOODWILL_TO_ASSETGraham's numberFQ, FYGRAHAM_NUMBERSGross margin
%FQ, FH, FY, TTMGROSS_MARGINGross profit to assets ratioFQ, FYGROSS_PROFIT_TO_ASSETInterest
coverageFQ, FH, FYINTERST_COVERInventory to revenue ratioFQ, FH, FYINVENT_TO_REVENUEInventory
turnoverFQ, FH, FYINVENT_TURNOVERKZ indexFYKZ_INDEXLong term debt to total assets ratioFQ, FH, FYLONG_TERM_DEBT_TO_ASSETSNet current asset value per shareFQ, FYNCAVPS_RATIONet income per
employeeFYNET_INCOME_PER_EMPLOYEEENet margin %FQ, FH, FY, TTMNET_MARGINNumber of employ-
eesFYNUMBER_OF_EMPLOYEEESOperating earnings yield %FQ, FH, FYOPERATING_EARNINGS_YIELDOperating
margin %FQ, FH, FYOPERATING_MARGINPEG ratioFQ, FYPEG_RATIO Piotroski F-scoreFQ, FH, FYPI-
OTOSKI_F_SCOREPrice earnings ratio forwardFQ, FYPRICE_EARNINGS_FORWARDPrice sales ratio forwardFQ, FYPRICE_SALES_FORWARDQuality ratioFQ, FH, FYQUALITY_RATIOQuick ratioFQ, FH, FYQUICK_RATIOResearch
& development to revenue ratioFQ, FH, FYRESEARCH_AND_DEVELOP_TO_REVENUEReturn on assets %FQ, FH, FYRETURN_ON_ASSETSReturn on common equity %FQ, FH, FYRETURN_ON_COMMON_EQUITYReturn
on equity %FQ, FH, FYRETURN_ON_EQUITYReturn on equity adjusted to book value %FQ, FH, FYRE-
TURN_ON_EQUITY_ADJUST_TO_BOOKReturn on invested capital %FQ, FH, FYRETURN_ON_INVESTED_CAPITALRet
on tangible assets %FQ, FH, FYRETURN_ON_TANG_ASSETSReturn on tangible equity %FQ, FH, FYRE-
TURN_ON_TANG_EQUITYRevenue estimatesFQ, FH, FYSALES_ESTIMATESRevenue one year growthFQ, FH, FY, TTMREVENUE_ONE_YEAR_GROWTHRevenue per employeeFYREVENUE_PER_EMPLOYEEShares
buyback ratio %FQ, FH, FYSHARE_BUYBACK_RATIO Sloan ratio %FQ, FH, FYSLOAN_RATIO Springate
scoreFQ, FYSRINGATE_SCORESustainable growth rateFQ, FYSUSTAINABLE_GROWTH_RATE Tangible com-
mon equity ratioFQ, FH, FYTANGIBLE_COMMON_EQUITY_RATIO Tobin's Q (approximate)FQ, FH, FYTO-
BIN_Q_RATIO Total common shares outstandingFQ, FH, FYTOTAL_SHARES_OUTSTANDINGZmijewski scoreFQ, FYZMIJEWSKI_SCORE## request.economic()

The request.economic() function provides scripts with the ability to retrieve economic data for a specified country or region, including information about the state of the economy (GDP, inflation rate, etc.) or of a particular industry (steel production, ICU beds, etc.).

Below is the signature for this function:

```
request.economic(country_code, field, gaps, ignore_invalid_symbol) → series float
```

The `country_code` parameter accepts a “string” value representing the identifier of the country or region to request economic data for (e.g., “US”, “EU”, etc.). See the Country/region codes section for a complete list of codes this function supports. Note that the economic metrics available depend on the country or region specified in the function call.

The `field` parameter accepts a “string” specifying the metric that the function requests. The Field codes section covers all accessible metrics and the countries/regions they’re available for.

For a detailed explanation on the last two parameters of this function, see the Common characteristics section at the top of this page.

This simple example requests the growth rate of the Gross Domestic Product (“GDPQQ”) for the United States (“US”) using `request.economic()`, then plots its value on the chart with a gradient color:



Figure 204: image

```
//@version=6
indicator("Requesting economic data demo")

//@variable The GDP growth rate for the US economy.
float gdpqq = request.economic("US", "GDPQQ")

//@variable The all-time maximum growth rate.
float maxRate = ta.max(gdpqq)
//@variable The all-time minimum growth rate.
float minRate = ta.min(gdpqq)

//@variable The color of the `gdpqq` plot.
color rateColor = switch
    gdpqq >= 0 => color.from_gradient(gdpqq, 0, maxRate, color.purple, color.blue)
    => color.from_gradient(gdpqq, minRate, 0, color.red, color.purple)

// Plot the results.
plot(gdpqq, "US GDP Growth Rate", rateColor, style = plot.style_area)
```

Note that:

- This example does not include a `gaps` argument in the `request.economic()` call, so the function uses the default `barmerge.gaps_off`. In other words, it returns the last retrieved value when new data isn’t yet available.

Country/region codes

The table in this section lists all country/region codes available for use with `request.economic()`. The first column of the table contains the “string” values that represent the country or region code, and the second column contains the corresponding country/region names.

It’s important to note that the value used as the `country_code` argument determines which field codes are accessible to the function.

Click to show/hide `country_code` Country/region name
AFAfghanistan ALAlbania DZAlgeria ADAndorra AOAngola AGAntigua and Barbuda ARArgentina AMArmenia AWAruba AUAustralia ATAustria AZAzerbaijan BSBahamas BHBahrain BDBangladesh BB and Herzegovina BWBotswana BRBrazil BNBrunei BGBulgaria BFBurkina Faso BIBurundi KHCambodia CMCameroon CACanada CVCape Verde KYCayman Islands CFCentral African Republic TDChad CLChile CNChina COColombia KMComoros CGCongo CRCosta Rica HRCroatia CUCuba CYCyprus CZCzech Republic DKDenmark DJDjibouti DMDominica DODominican Republic TLEast Timor ECEcuador EGEgypt SVEL Salvador GQEquatorial Guinea EREritrea EEESTonia ETEThiopia EUEuro area FOFaroe Islands FJFiji FIFinland FRFrance GAGabon GMGambia GEGeorgia DEGermany GHGhana GRGreece GLGreenland Bissau GYGuyana HTHaiti HNHonduras HKHong Kong HUHungary ISICeland INIndia IDIndonesia IRIran IQIraq IEIreland IMIsle of Man ILIsrael ITItaly CIIvory Coast JMJamaica JPPakistan JOJordan KZKazakhstan KEKenya KIKiribati XKKosovo KWKuwait KGKyrgyzstan Caledonia NZNew Zealand NINicaragua NENiger NGNigeria KPNorth Korea NONorway OMOman PKPakistan PSPalestine PAPANama PGPapua New Guinea PYParaguay PEPPeru PHPhilippines PLPoland PTPortugal PRPuerto Rico QAQatar CDRRepublic of the Congo RORomania RURussia RWRwanda WSSamoa SMSan Marino STSao Tome and Principe SASaudi Arabia SNSenegal RSSerbia SCSEychelles SLSierra Leone SSSingapore SKSlovakia SISlovenia SBSolomon Islands SOSomalia ZASouth Africa KRSouth Korea SSSouth Sudan ESSpain LKSri Lanka LCSt Lucia VCSt Vincent and the Grenadines SDSudan SRSuriname SZSwaziland SESweden CHSwitzerland SYSyria TWTaiwan TJTajikistan TZTanzania THThailand TGTogo TOTonga TTTrinidad and Tobago TNTunisia TRTurkey TMTurkmenistan UGUganda UAUkraine AEUnited Arab Emirates GBUnited Kingdom USUnited States UYUruguay UZUzbekistan VUVanuatu VEVenezuela VNVietnam YEYemen ZMZambia ZWZimbabwe ### Field codes

The table in this section lists the field codes available for use with `request.economic()`. The first column contains the “string” values used as the `field` argument, and the second column contains names of each metric and links to our Help Center with additional information, including the countries/regions they’re available for.

Click to show/hide `field` Metric
AAAsylum Applications ACRAPI Crude Runs AEAuto Exports AHEAverage Hourly Earnings AHOAPI Heating Oil AWHAverage Weekly Hours BBSBanks Balance Sheet BCLIBusiness Climate Indicator BCOIBusiness Confidence Index BIBusiness Inventories BLRBank Lending Rate BOINFIB Business Optimism Index BOTBalance Of Trade BPBuilding Permits BRBankruptcies CACurrent Account CAGCurrent Account To GDP CAPCar Production CARCar Registrations CBBSCentral Bank Balance Sheet CCCClaimant Count Change CCICConsumer Confidence Index CCOSCushing Crude Oil Stocks CCPCore Consumer Prices CCPICore CPI CCPTConsumer Confidence Price Trends CCRConsumer Credit CCSCredit Card Spending CEPCEment Production CFCapital Flows CFNAIChicago Fed National Activity Index CIAPI Crude Imports CINDCoincident Index CIRCore Inflation Rate, YoY CJCCContinuing Jobless Claims CNAPI Cushing Number COICrude Oil Imports COIRCruide Oil Imports from Russia CONSTSConstruction Spending COPCrude Oil Production CORCrude Oil Rigs CORDConstruction Orders, YoY CORPICorruption Index CORRCorruption Rank COSCCrude Oil Stocks Change COUTConstruction Output, YoY CPCopper Production CPCEPICore PCE Price Index CPIConsumer Price Index CPIHUCPI Housing Utilities CPIMCPI Median CPITCPI Transportation CPITMCPI Trimmed Mean CPMIChicago PMICPPICore Producer Price Index CPRCorporate Profits CRLPICereals Price Index CRRCash Reserve Ratio CSConsumer Spending CSCAPI Crude Oil Stock Change CSHPICase Shiller Home Price Index CSHPIIMCase Shiller Home Price Index, MoM CSHPIYYCase Shiller Home Price Index, YoY CSSChain Store Sales CTRCorporate Tax Rate CUCapacity Utilization DFMIDallas Fed Manufacturing Index DFPDistillate Fuel Production DFSDistillate Stocks DFSIDallas Fed Services Index DFSRIDallas Fed Services Revenues Index DGDDeposit Growth DGODurable Goods Orders DGOEDDurable Goods Orders Excluding Defense DGOETDurable Goods Orders Excluding Transportation DIRDeposit Interest Rate DPIDisposable Personal Income DRPIDairy Price Index DSAPI Distillate Stocks DTCBI Distributive Trades ECADP Employment Change EDEExternal Debt EDBREase Of Doing Business Ranking EHSExisting Home Sales ELPElectricity Production EMCCEmployment Change EMCIEmployment Cost Index EMPEmployed Persons EMREmployment Rate EOIEconomic Optimism Index EPEExport Prices ESIZEW Economic Sentiment Index EWSEconomy Watchers Survey EXPEExports EXPYYExports, YoY FAIFixed Asset Investment FBIForeign Bond Investment FDIForeign Direct Investment FEFiscal Expenditure FERForeign Exchange Reserves FIFood Inflation, YoY FOFactory Orders FOETFFactory Orders Excluding Transportation FPIFood Price Index FSIForeign Stock Investment FTEFull Time Employment FYGDPGFull Year GDP Growth GASPGasoline Prices GBPGovernment Budget GBVGGovernment Budget Value GCICompetitiveness Index GCRCompetitiveness Rank GDGovernment Debt GDGGovernment Debt To GDP GDPGDP Gross Domestic Product GDPAGDP From Agriculture GDPCGDP From Construction GDPCPGDP Constant Prices GDPDGDGDP Deflator GDPGAGDP Growth Annualized GDPMANGDP From Manufacturing GDPMINGDP From Mining GDPPAGDP From Public Administration GDPPCGDP Per Capita GDPPCPGDP Per Capita, PPP GDPQQGDP Growth Rate GDPSGDP From Services GDPSAGDP Sales GDPTGDP From Transport GDPUGDP From Utilities GDPYYGDP,

YoYGDTPIGlobal Dairy Trade Price IndexGFCFGross Fixed Capital FormationGNPGross National ProductGPGGold ProductionGPAGovernment PayrollsGPROGasoline ProductionGRGovernment RevenuesGRESGold ReservesGSAPI Gasoline StocksGSCGrain Stocks CornGSCHGasoline Stocks ChangeGSGGovernment Spending To GDPGSPGovernment SpendingGSSGrain Stocks SoyGSWGrain Stocks WheatGTBGoods Trade BalanceHBHospital BedsHDGHouseholds Debt To GDPHDIHouseholds Debt To IncomeHICPHarmonised Index of Consumer PricesHIRMMHarmonised Inflation Rate, MoMHIRYYHarmonised Inflation Rate, YoYHMINAHB Housing Market IndexHORHome Ownership RateHOSHHeating Oil StocksHOSPHospitalsHPIHouse Price IndexHPIMMHouse Price Index, MoMHPIYYHouse Price Index, YoYHSHome LoansHSPHousehold SpendingHSTHousing StartsICChanges In InventoriesICUBICU BedsIEInflation ExpectationsIFOCCIFO Assessment Of The Business SituationIFOEIFO Business Developments ExpectationsIJCInitial Jobless ClaimsIMPImportsIMPYYImports, YoYINBRInterbank RateINTRInterest RateIPAIP AddressesIPMMIndustrial Production, MoMIPRIImport PricesIPYYIndustrial Production, YoYIRMMInflation Rate, MoMIRYYInflation Rate, YoYISIndustrial SentimentISPInternet SpeedJAJob AdvertisementsJARJobs To Applications RatioJCChallenger Job CutsJC4WJobless Claims, 4-Week AverageJOJob OffersJVJob VacanciesKFMIKansas Fed Manufacturing IndexLBLoans To BanksLCLabor CostsLEILEading Economic IndexLFPRLabor Force Participation RateLGLLoan Growth, YoYLIVRRRLiquidity Injections Via Reverse RepoLMICLMI Logistics Managers Index CurrentLMICILMI Inventory CostsLMIFLMI Logistics Managers Index FutureLMITPLMI Transportation PricesLMIWPLMI Warehouse PricesLPSLoans To Private SectorLRCentral Bank Lending RateLTURLong Term Unemployment RateLWFLiving Wage FamilyLWILiving Wage IndividualM0Money Supply M0M1Money Supply M1M2Money Supply M2M3Money Supply M3MAMortgage ApprovalsMAPLMortgage ApplicationsMCEMMichigan Consumer ExpectationsMCECMMichigan Current Economic ConditionsMDMedical DoctorsMEMilitary ExpenditureMGDPYYMonthly GDP, YoYMIE1YMichigan Inflation ExpectationsMIE5YMichigan 5 Year Inflation ExpectationsMIPMining Production, YoYMMIMBA Mortgage Market IndexMOMachinery OrdersMPManufacturing PayrollsMPIMEat Price IndexMPRMMManufacturing Production, MoMMPRYYYManufacturing Production, YoYMRMortgage RateMRIMBA Mortgage Refinance IndexMSManufacturing SalesMTOMachine Tool OrdersMWMinimum WagesNDCGOEAOrders For Non-defense Capital Goods Excluding AircraftNEGTBGoods Trade Deficit With Non-EU CountriesNFPNonfarm PayrollsNGINatural Gas ImportsNGIRNatural Gas Imports from RussiaNGSCNatural Gas Stocks ChangeNHPINationwide House Price IndexNHSNew Home SalesNHSMMNew Home Sales, MoMNMMPMINon-Manufacturing PMINONew OrdersNODXMMNon-Oil Domestic Exports, MoMNODXYNon-Oil Domestic Exports, YoYNOENon-Oil ExportsNPPNonfarm Payrolls PrivateNURSNursesNYESMINY Empire State Manufacturing IndexOEOil ExportsOPIOils Price IndexPCEPIPCE Price IndexPDGPrivate Debt To GDP PPFMIPhiladelphia Fed Manufacturing IndexPHSIMMPending Home Sales Index, MoMPHSIYYPending Home Sales Index, YoYIPIPersonal IncomePINPrivate InvestmentPINDMBA Purchase IndexPITRPersonal Income Tax RatePOPPopulationPPIProducer Price IndexPPIIProducer Price Index InputPPIMMProducer Price Inflation, MoMPPIYYProducer Prices Index, YoYPRIAPI Product ImportsPRODProductivityPSPPersonal SavingsPSCPrivate Sector CreditPSPPersonal SpendingPTEPart Time EmploymentPUACPandemic Unemployment Assistance ClaimsRAMRetirement Age MenRAWRetirement Age WomenRCRRefinery Crude RunsREMRRemittancesRFMIRichmond Fed Manufacturing IndexRFMSIRichmond Fed Manufacturing Shipments IndexRFSIRichmond Fed Services IndexRIRedbook IndexRIEARetail Inventories Excluding AutosRPIRetail Price IndexRRRRepo RateRRRReverse Repo RateRSEARetail Sales Excluding AutosRSEFRetail Sales Excluding FuelRSMMRetail Sales, MoMRSYYYRetail Sales, YoYRTIREuters Tankan IndexSBSISSmall Business Sentiment IndexSFHPSingle Family Home PricesSPSteel ProductionSPISugar Price IndexSSServices SentimentSSRSocial Security RateSSRCSocial Security Rate For CompaniesSSRESocial Security Rate For EmployeesSTRSales Tax RateTATourist ArrivalsTAXRTax RevenueTCBTreasury Cash BalanceTCPITokyo CPITITerrorism IndexTITertiary Industry IndexTOTTerms Of TradeTRTourism RevenuesTVSTotal Vehicle SalesUCUnemployment ChangeUPUnemployed PersonsURUnemployment RateWAGWagesWESWeapons SalesWGWage Growth, YoYWHSWages High SkilledWIWholesale InventoriesWLSWages Low SkilledWMWages In ManufacturingWPIWholesale Price IndexWSWholesale SalesYURYouth Unemployment RateZCCZEW Current Conditions## request.seed()

TradingView aggregates a vast amount of data from its many providers, including price and volume information on tradable instruments, financials, economic data, and more, which users can retrieve in Pine Script™ using the functions discussed in the sections above, as well as multiple built-in variables.

To further expand the horizons of possible data one can analyze on TradingView, we have Pine Seeds, which allows users to supply custom *user-maintained* EOD data feeds via GitHub for use on TradingView charts and within Pine Script™ code.

To retrieve data from a Pine Seeds data feed within a script, use the `request.seed()` function. Below is the function's signature:

```
request.seed(source, symbol, expression, ignore_invalid_symbol, calc_bars_count) → series <type>
```

The **source** parameter specifies the unique name of the user-maintained GitHub repository that contains the data feed.

The **symbol** parameter represents the file name from the “data/” directory of the **source** repository, excluding the “.csv” file extension. See this page for information about the structure of the data stored in repositories.

The **expression** parameter is the series to evaluate using data extracted from the requested context. It is similar to the

equivalent in `request.security()` and `request.security_lower_tf()`. Data feeds stored in user-maintained repos contain time, open, high, low, close, and volume information, meaning the `expression` argument can use the corresponding built-in variables, including variables derived from them (e.g., `bar_index`, `ohlc4`, etc.) to request their values from the context of the custom data.

The script below visualizes sample data from the `seed_crypto_sentiment` demo repository. It uses two calls to `request.seed()` to retrieve the close values from the repository's `BTC_SENTIMENT_POSITIVE_TOTAL` and `BTC_SENTIMENT_NEGATIVE_TOTAL` data feeds and plots the results on the chart as step lines:

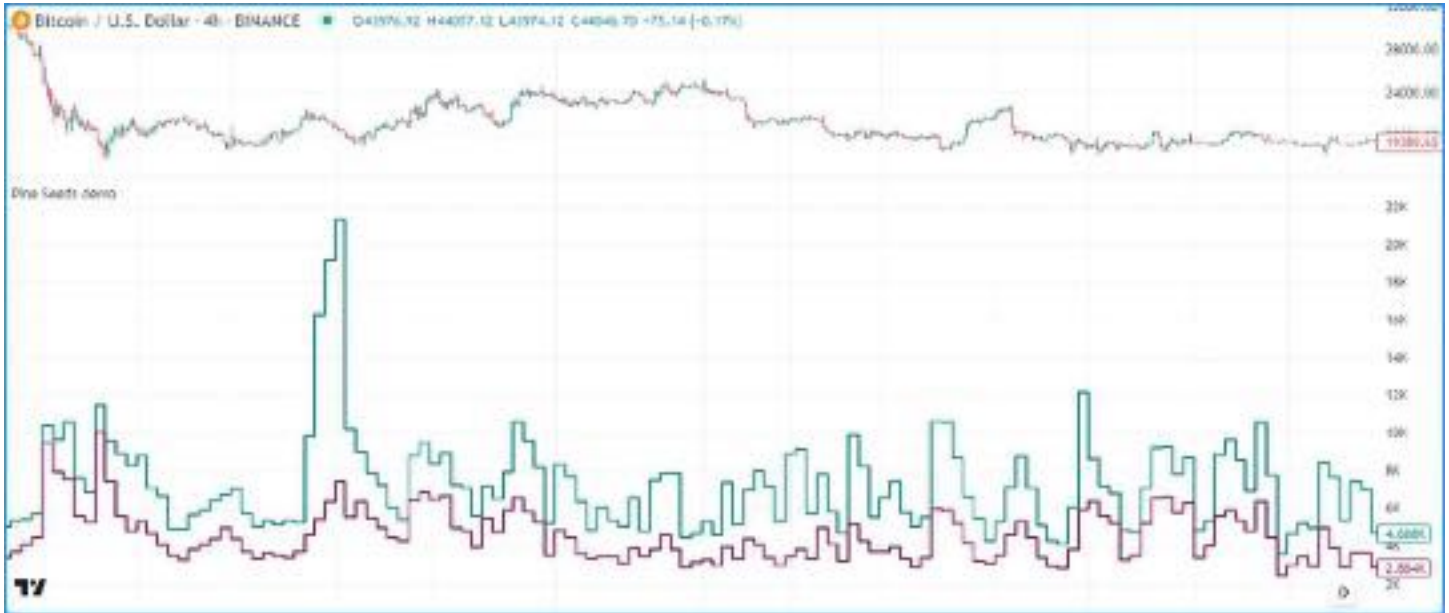


Figure 205: image

```
//@version=6
indicator("Pine Seeds demo", format=format.volume)

//@variable The total positive sentiment for BTC extracted from the "seed_crypto_sentiment" repository.
float positiveTotal = request.seed("seed_crypto_sentiment", "BTC_SENTIMENT_POSITIVE_TOTAL", close)
//@variable The total negative sentiment for BTC extracted from the "seed_crypto_sentiment" repository.
float negativeTotal = request.seed("seed_crypto_sentiment", "BTC_SENTIMENT_NEGATIVE_TOTAL", close)

// Plot the data.
plot(positiveTotal, "Positive sentiment", color.teal, 2, plot.style_stepline)
plot(negativeTotal, "Negative sentiment", color.maroon, 2, plot.style_stepline)
```

Note that:

- This example requests data from the repository highlighted in the Pine Seeds documentation. It exists solely for example purposes, and its data *does not* update on a regular basis.
- Unlike most other `request.*()` functions, `request.seed()` does not have a `gaps` parameter. It always returns na values when no new data exists.
- Pine Seeds data is searchable from the chart's symbol search bar. To load a data feed on the chart, enter the “*Repo:File*” pair, similar to searching for an “Exchange:Symbol” pair.

[Previous

Non-standard charts data](#non-standard-charts-data)[Next

Plots](#plots) User Manual/Concepts/Plots

Plots

Introduction

The `plot()` function is the most frequently used function used to display information calculated using Pine scripts. It is versatile and can plot different styles of lines, histograms, areas, columns (like volume columns), fills, circles or crosses.

The use of `plot()` to create fills is explained in the page on Fills.

This script showcases a few different uses of `plot()` in an overlay script:

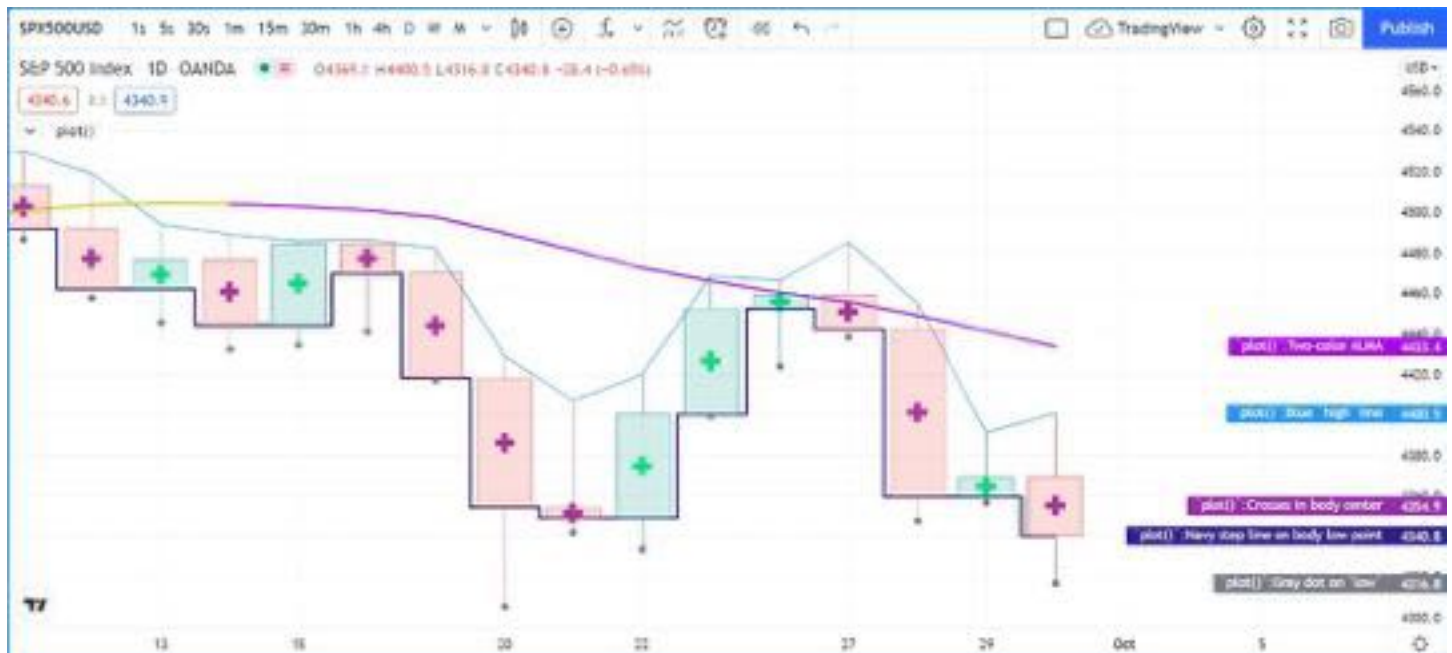


Figure 206: image

```
//@version=6
indicator("`plot()`", "", true)
plot(high, "Blue `high` line")
plot(math.avg(close, open), "Crosses in body center", close > open ? color.lime : color.purple, 6, plot.style_circles)
plot(math.min(open, close), "Navy step line on body low point", color.navy, 3, plot.style_stepline)
plot(low, "Gray dot on `low`", color.gray, 3, plot.style_circles)

color VIOLET = #AA00FF
color GOLD = #CCCC00
ma = ta.alma(hl2, 40, 0.85, 6)
var almaColor = color.silver
almaColor := ma > ma[2] ? GOLD : ma < ma[2] ? VIOLET : almaColor
plot(ma, "Two-color ALMA", almaColor, 2)
```

Note that:

- The first `plot()` call plots a 1-pixel blue line across the bar highs.
- The second plots crosses at the mid-point of bodies. The crosses are colored lime when the bar is up and purple when it is down. The argument used for `linewidth` is 6 but it is not a pixel value; just a relative size.
- The third call plots a 3-pixel wide step line following the low point of bodies.
- The fourth call plot a gray circle at the bars' low.
- The last plot requires some preparation. We first define our bull/bear colors, calculate an Arnaud Legoux Moving Average, then make our color calculations. We initialize our color variable on bar zero only, using `var`. We initialize it to `color.silver`, so on the dataset's first bars, until one of our conditions causes the color to change, the line will be silver. The conditions that change the color of the line require it to be higher/lower than its value two bars ago. This makes for less noisy color transitions than if we merely looked for a higher/lower value than the previous one.

This script shows other uses of `plot()` in a pane:



Figure 207: image

```
//@version=6
indicator("Volume change", format = format.volume)

color GREEN          = #008000
color GREEN_LIGHT    = color.new(GREEN, 50)
color GREEN_LIGHTER  = color.new(GREEN, 85)
color PINK           = #FF0080
color PINK_LIGHT     = color.new(PINK, 50)
color PINK_LIGHTER   = color.new(PINK, 90)

bool  barUp = ta.rising(close, 1)
bool  barDn = ta.falling(close, 1)
float volumeChange = ta.change(volume)

volumeColor = barUp ? GREEN_LIGHTER : barDn ? PINK_LIGHTER : color.gray
plot(volume, "Volume columns", volumeColor, style = plot.style_columns)

volumeChangeColor = barUp ? volumeChange > 0 ? GREEN : GREEN_LIGHT : volumeChange < 0 ? PINK : PINK_LIGHT
plot(volumeChange, "Volume change columns", volumeChangeColor, 12, plot.style_histogram)

plot(0, "Zero line", color.gray)
```

Note that:

- We are plotting normal volume values as wide columns above the zero line (see the `style = plot.style_columns` in our `plot()` call).
- Before plotting the columns we calculate our `volumeColor` by using the values of the `barUp` and `barDn` boolean variables. They become respectively `true` when the current bar's close is higher/lower than the previous one. Note that the "Volume" built-in does not use the same condition; it identifies an up bar with `close > open`. We use the `GREEN_LIGHTER` and `PINK_LIGHTER` colors for the volume columns.
- Because the first plot plots columns, we do not use the `linewidth` parameter, as it has no effect on columns.
- Our script's second plot is the **change** in volume, which we have calculated earlier using `ta.change(volume)`. This value is plotted as a histogram, for which the `linewidth` parameter controls the width of the column. We make this width 12 so that histogram elements are thinner than the columns of the first plot. Positive/negative `volumeChange` values plot above/below the zero line; no manipulation is required to achieve this effect.
- Before plotting the histogram of `volumeChange` values, we calculate its color value, which can be one of four different colors. We use the bright `GREEN` or `PINK` colors when the bar is up/down AND the volume has increased since the

last bar (`volumeChange > 0`). Because `volumeChange` is positive in this case, the histogram's element will be plotted above the zero line. We use the bright `GREEN_LIGHT` or `PINK_LIGHT` colors when the bar is up/down AND the volume has NOT increased since the last bar. Because `volumeChange` is negative in this case, the histogram's element will be plotted below the zero line.

- Finally, we plot a zero line. We could just as well have used `hline(0)` there.
- We use `format = format.volume` in our `indicator()` call so that large values displayed for this script are abbreviated like those of the built-in "Volume" indicator.

`plot()` calls must always be placed in a line's first position, which entails they are always in the script's global scope. They can't be placed in user-defined functions or structures like `if`, `for`, etc. Calls to `plot()` can, however, be designed to plot conditionally in two ways, which we cover in the Plotting conditionally section of this page.

A script can only plot in its own visual space, whether it is in a pane or on the chart as an overlay. Scripts running in a pane can only color bars in the chart area.

`plot()` parameters

The `plot()` function has the following signature:

```
plot(series, title, color, linewidth, style, trackprice, histbase, offset, join, editable, show_last, display,
```

The parameters of `plot()` are:

`series`

It is the only mandatory parameter. Its argument must be of "series int/float" type. Note that because the auto-casting rules in Pine Script™ convert in the `int float bool` direction, a "bool" type variable cannot be used as is; it must be converted to an "int" or a "float" for use as an argument. For example, if `newDay` is of "bool" type, then `newDay ? 1 : 0` can be used to plot 1 when the variable is `true`, and zero when it is `false`.

`title`

Requires a "const string" argument, so it must be known at compile time. The string appears:

- In the script's scale when the "Chart settings/Scales/Indicator Name Label" field is checked.
- In the Data Window.
- In the "Settings/Style" tab.
- In the dropdown of `input.source()` fields.
- In the "Condition" field of the "Create Alert" dialog box, when the script is selected.
- As the column header when exporting chart data to a CSV file.

`color`

Accepts "series color", so can be calculated on the fly, bar by bar. Plotting with `na` as the color, or any color with a transparency of 100, is one way to hide plots when they are not needed.

`linewidth`

Is the plotted element's size, but it does not apply to all styles. When a line is plotted, the unit is pixels. It has no impact when `plot.style_columns` is used.

`style`

The available arguments are:

- `plot.style_line` (the default): It plots a continuous line using the `linewidth` argument in pixels for its width. `na` values will not plot as a line, but they will be bridged when a value that is not `na` comes in. Non-`na` values are only bridged if they are visible on the chart.
- `plot.style_linebr`: Allows the plotting of discontinuous lines by not plotting on `na` values, and not joining gaps, i.e., bridging over `na` values.
- `plot.style_stepline`: Plots using a staircase effect. Transitions between changes in values are done using a vertical line drawn in middle of bars, as opposed to a point-to-point diagonal joining the midpoints of bars. Can also be used to achieve an effect similar to that of `plot.style_linebr`, but only if care is taken to plot no color on `na` values.
- `plot.style_area`: plots a line of `linewidth` width, filling the area between the line and the `histbase`. The `color` argument is used for both the line and the fill. You can make the line a different color by using another `plot()` call. Positive values are plotted above the `histbase`, negative values below it.
- `plot.style_areabr`: This is similar to `plot.style_area` but it doesn't bridge over `na` values. Another difference is how the indicator's scale is calculated. Only the plotted values serve in the calculation of the `y` range of the script's visual space.

If only high values situated far away from the **histbase** are plotted, for example, those values will be used to calculate the *y* scale of the script's visual space. Positive values are plotted above the **histbase**, negative values below it.

- **plot.style_columns**: Plots columns similar to those of the “Volume” built-in indicator. The **linewidth** value does **not** affect the width of the columns. Positive values are plotted above the **histbase**, negative values below it. Always includes the value of **histbase** in the *y* scale of the script's visual space.
- **plot.style_histogram**: Plots columns similar to those of the “Volume” built-in indicator, except that the **linewidth** value is used to determine the width of the histogram's bars in pixels. Note that since **linewidth** requires an “input int” value, the width of the histogram's bars cannot vary bar to bar. Positive values are plotted above the **histbase**, negative values below it. Always includes the value of **histbase** in the *y* scale of the script's visual space.
- **plot.style_circles** and **plot.style_cross**: These plot a shape that is not joined across bars unless **join = true** is also used. For these styles, the **linewidth** argument becomes a relative sizing measure — its units are not pixels.

trackprice

The default value of this is **false**. When it is **true**, a dotted line made up of small squares will be plotted the full width of the script's visual space. It is often used in conjunction with **show_last = 1**, **offset = -99999** to hide the actual plot and only leave the residual dotted line.

histbase

It is the reference point used with **plot.style_area**, **plot.style_columns** and **plot.style_histogram**. It determines the level separating positive and negative values of the **series** argument. It cannot be calculated dynamically, as an “input int/float” is required.

offset

This allows shifting the plot in the past/future using a negative/positive offset in bars. The value cannot change during the script's execution.

join

This only affect styles **plot.style_circles** or **plot.style_cross**. When **true**, the shapes are joined by a one-pixel line.

editable

This boolean parameter controls whether or not the plot's properties can be edited in the “Settings/Style” tab. Its default value is **true**.

show_last

Allows control over how many of the last bars the plotted values are visible. An “input int” argument is required, so it cannot be calculated dynamically.

display

The default is **display.all**. When it is set to **display.none**, plotted values will not affect the scale of the script's visual space. The plot will be invisible and will not appear in indicator values or the Data Window. It can be useful in plots destined for use as external inputs for other scripts, or for plots used with the `{{plot("[plot_title]")}}` placeholder in **alertcondition()** calls, e.g.:

```
//@version=6
indicator("")
r = ta.rsi(close, 14)
xUp = ta.crossover(r, 50)
plot(r, "RSI", display = display.none)
alertcondition(xUp, "xUp alert", message = 'RSI is bullish at: {{plot("RSI")}}')
```

force_overlay

If **true**, the plotted results will display on the main chart pane, even when the script occupies a separate pane. Optional. The default is **false**.

Plotting conditionally

plot() calls cannot be used in conditional structures such as **if**, but they can be controlled by varying their plotted values, or their color. When no plot is required, you can either plot **na** values, or plot values using **na** color or any color with 100 transparency (which also makes it invisible).

Value control

One way to control the display of plots is to plot na values when no plot is needed. Sometimes, values returned by functions such as `request.security()` will return na values, when `gaps = barmerge.gaps_on` is used, for example. In both these cases it is sometimes useful to plot discontinuous lines. This script shows a few ways to do it:



Figure 208: image

```
//@version=6
indicator("Discontinuous plots", "", true)
bool plotValues = bar_index % 3 == 0
plot(plotValues ? high : na, color = color.fuchsia, linewidth = 6, style = plot.style_linebr)
plot(plotValues ? high : na)
plot(plotValues ? math.max(open, close) : na, color = color.navy, linewidth = 6, style = plot.style_cross)
plot(plotValues ? math.min(open, close) : na, color = color.navy, linewidth = 6, style = plot.style_circles)
plot(plotValues ? low : na, color = plotValues ? color.green : na, linewidth = 6, style = plot.style_stepline)
```

Note that:

- We define the condition determining when we plot using `bar_index % 3 == 0`, which becomes `true` when the remainder of the division of the bar index by 3 is zero. This will happen every three bars.
- In the first plot, we use `plot.style_linebr`, which plots the fuchsia line on highs. It is centered on the bar's horizontal midpoint.
- The second plot shows the result of plotting the same values, but without using special care to break the line. What's happening here is that the thin blue line of the plain `plot()` call is automatically bridged over na values (or *gaps*), so the plot does not interrupt.
- We then plot navy blue crosses and circles on the body tops and bottoms. The `plot.style_circles` and `plot.style_cross` style are a simple way to plot discontinuous values, e.g., for stop or take profit levels, or support & resistance levels.
- The last plot in green on the bar lows is done using `plot.style_stepline`. Note how its segments are wider than the fuchsia line segments plotted with `plot.style_linebr`. Also note how on the last bar, it only plots halfway until the next bar comes in.
- The plotting order of each plot is controlled by their order of appearance in the script. See

This script shows how you can restrict plotting to bars after a user-defined date. We use the `input.time()` function to create an input widget allowing script users to select a date and time, using Jan 1st 2021 as its default value:

```
//@version=6
indicator("", "", true)
startInput = input.time(timestamp("2021-01-01"))
plot(time > startInput ? close : na)
```


Color control

The Conditional coloring section of the page on colors discusses color control for plots. We'll look here at a few examples.

The value of the `color` parameter in `plot()` can be a constant, such as one of the built-in constant colors or a color literal. In Pine Script™, the qualified type of such colors is called “**const color**” (see the Type system page). They are known at compile time:

```
//@version=6
indicator("", "", true)
plot(close, color = color.gray)
```

The color of a plot can also be determined using information that is only known when the script begins execution on the first historical bar of a chart (bar zero, i.e., `bar_index == 0` or `barstate.isfirst == true`), as will be the case when the information needed to determine a color depends on the chart the script is running on. Here, we calculate a plot color using the `syminfo.type` built-in variable, which returns the type of the chart's symbol. The qualified type of `plotColor` in this case will be “**simple color**”:

```
//@version=6
indicator("", "", true)
plotColor = switch syminfo.type
    "stock"      => color.purple
    "futures"    => color.red
    "index"      => color.gray
    "forex"      => color.fuchsia
    "crypto"     => color.lime
    "fund"       => color.orange
    "dr"         => color.aqua
    "cfd"        => color.blue
plot(close, color = plotColor)
printTable(txt) => var table t = table.new(position.middle_right, 1, 1), table.cell(t, 0, 0, txt, bgcolor = co
printTable(syminfo.type)
```

Plot colors can also be chosen through a script's inputs. In this case, the `lineColorInput` variable is of the “**input color**” type:

```
//@version=6
indicator("", "", true)
color lineColorInput = input(#1848CC, "Line color")
plot(close, color = lineColorInput)
```

Finally, plot colors can also be *dynamic* values, i.e., calculated values that can change on each bar. These values are of the “**series color**” type:

```
//@version=6
indicator("", "", true)
plotColor = close >= open ? color.lime : color.red
plot(close, color = plotColor)
```

When plotting pivot levels, one common requirement is to avoid plotting level transitions. Using lines is one alternative, but you can also use `plot()` like this:

```
//@version=6
indicator("Pivot plots", "", true)
pivotHigh = fixnan(ta.pivohigh(3,3))
plot(pivotHigh, "High pivot", ta.change(pivotHigh) != 0 ? na : color.olive, 3)
plotchar(ta.change(pivotHigh), "ta.change(pivotHigh)", "•", location.top, size = size.small)
```

Note that:

- We use `pivotHigh = fixnan(ta.pivohigh(3,3))` to hold our pivot values. Because `ta.pivohigh()` only returns a value when a new pivot is found, we use `fixnan()` to fill the gaps with the last pivot value returned. The gaps here refer to the `na` values `ta.pivohigh()` returns when no new pivot is found.
- Our pivots are detected three bars after they occur because we use the argument `3` for both the `leftbars` and `rightbars` parameters in our `ta.pivohigh()` call.



Figure 209: image

- The last plot is plotting a continuous value, but it is setting the plot's color to na when the pivot's value changes, so the plot isn't visible then. Because of this, a visible plot will only appear on the bar following the one where we plotted using na color.
- The blue dot indicates when a new high pivot is detected and no plot is drawn between the preceding bar and that one. Note how the pivot on the bar indicated by the arrow has just been detected in the realtime bar, three bars later, and how no plot is drawn. The plot will only appear on the next bar, making the plot visible **four bars** after the actual pivot.

Levels

Pine Script™ has an `hline()` function to plot horizontal lines (see the page on Levels). `hline()` is useful because it has some line styles unavailable with `plot()`, but it also has some limitations, namely that it does not accept “series color”, and that its price parameter requires an “input int/float”, so cannot vary during the script's execution.

You can plot levels with `plot()` in a few different ways. This shows a CCI indicator with levels plotted using `plot()`:

```
//@version=6
indicator("CCI levels with `plot()`")
plot(ta.cci(close, 20))
plot(0, "Zero", color.gray, 1, plot.style_circles)
plot(bar_index % 2 == 0 ? 100 : na, "100", color.lime, 1, plot.style_linebr)
plot(bar_index % 2 == 0 ? -100 : na, "-100", color.fuchsia, 1, plot.style_linebr)
plot(200, "200", color.green, 2, trackprice = true, show_last = 1, offset = -99999)
plot(-200, "-200", color.red, 2, trackprice = true, show_last = 1, offset = -99999)
plot(300, "300", color.new(color.green, 50), 1)
plot(-300, "-300", color.new(color.red, 50), 1)
```

Note that:

- The zero level is plotted using `plot.style_circles`.
- The 100 levels are plotted using a conditional value that only plots every second bar. In order to prevent the na values from being bridged, we use the `plot.style_linebr` line style.
- The 200 levels are plotted using `trackprice = true` to plot a distinct pattern of small squares that extends the full width of the script's visual space. The `show_last = 1` in there displays only the last plotted value, which would appear as a one-bar straight line if the next trick wasn't also used: the `offset = -99999` pushes that one-bar segment far away in the past so that it is never visible.
- The 300 levels are plotted using a continuous line, but a lighter transparency is used to make them less prominent.

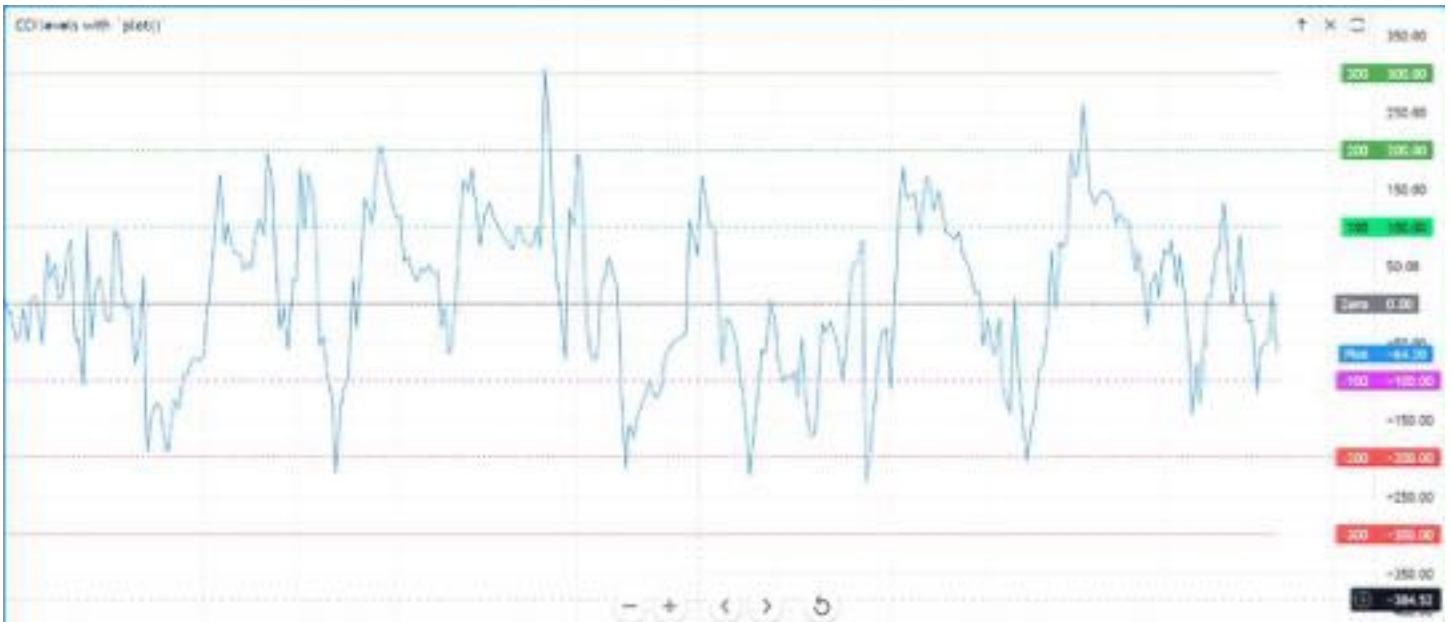


Figure 210: image

Offsets

The `offset` parameter specifies the shift used when the line is plotted (negative values shift in the past, positive values shift into the future). For example:

```
//@version=6
indicator("", "", true)
plot(close, color = color.red, offset = -5)
plot(close, color = color.lime, offset = 5)
```



Figure 211: image

As can be seen in the screenshot, the *red* series has been shifted to the left (since the argument's value is negative), while the *green* series has been shifted to the right (its value is positive).

Plot count limit

Each script is limited to a maximum plot count of 64. All `plot*()` calls and `alertcondition()` calls count in the plot count of a script. Some types of calls count for more than one in the total plot count.

`plot()` calls count for one in the total plot count if they use a "const color" argument for the `color` parameter, which means it is known at compile time, e.g.:

```
plot(close, color = color.green)
```

When they use another qualified type, such as any one of these, they will count for two in the total plot count:

```
plot(close, color = syminfo.mintick > 0.0001 ? color.green : color.red) // "simple color"
plot(close, color = input.color(color.purple)) // "input color"
plot(close, color = close > open ? color.green : color.red) // "series color"
plot(close, color = color.new(color.silver, close > open ? 40 : 0)) // "series color"
```

Scale

Not all values can be plotted everywhere. Your script's visual space is always bound by upper and lower limits that are dynamically adjusted with the values plotted. An RSI indicator will plot values between 0 and 100, which is why it is usually displayed in a distinct *pane* — or area — above or below the chart. If RSI values were plotted as an overlay on the chart, the effect would be to distort the symbol's normal price scale, unless it just happened to be close to RSI's 0 to 100 range. This shows an RSI signal line and a centerline at the 50 level, with the script running in a separate pane:

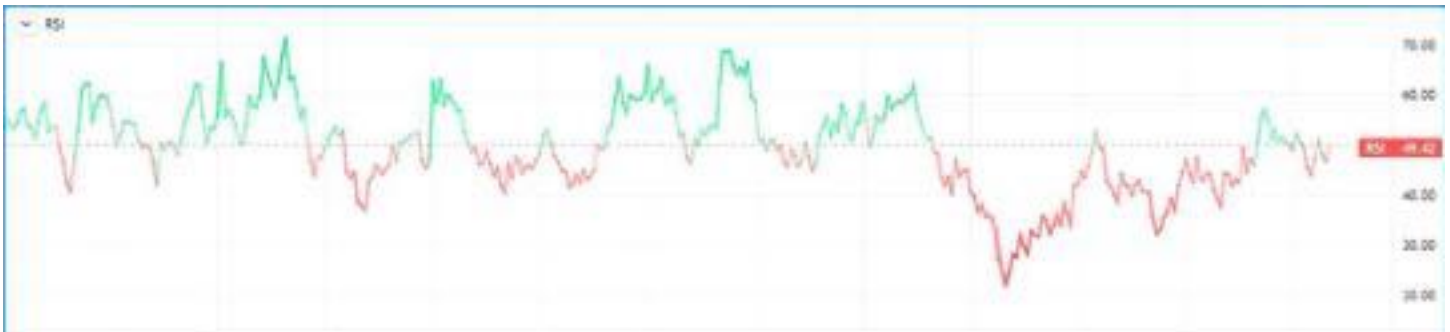


Figure 212: image

```
//@version=6
indicator("RSI")
myRSI = ta.rsi(close, 20)
bullColor = color.from_gradient(myRSI, 50, 80, color.new(color.lime, 70), color.new(color.lime, 0))
bearColor = color.from_gradient(myRSI, 20, 50, color.new(color.red, 0), color.new(color.red, 70))
myRSIColor = myRSI > 50 ? bullColor : bearColor
plot(myRSI, "RSI", myRSIColor, 3)
hline(50)
```

Note that the *y* axis of our script's visual space is automatically sized using the range of values plotted, i.e., the values of RSI. See the page on Colors for more information on the `color.from_gradient()` function used in the script.

If we try to plot the symbol's close values in the same space by adding the following line to our script:

```
plot(close)
```

This is what happens:



Figure 213: image

The chart is on the BTCUSD symbol, whose close prices are around 40000 during this period. Plotting values in the 40000 range makes our RSI plots in the 0 to 100 range indiscernible. The same distorted plots would occur if we placed the RSI indicator on the chart as an overlay.

Merging two indicators

If you are planning to merge two signals in one script, first consider the scale of each. It is impossible, for example, to correctly plot an RSI and a MACD in the same script's visual space because RSI has a fixed range (0 to 100) while MACD doesn't, as it plots moving averages calculated on price.

If both your indicators used fixed ranges, you can shift the values of one of them so they do not overlap. We could, for example, plot both RSI (0 to 100) and the True Strength Indicator (TSI) (-100 to +100) by displacing one of them. Our strategy here will be to compress and shift the TSI values so they plot over RSI:

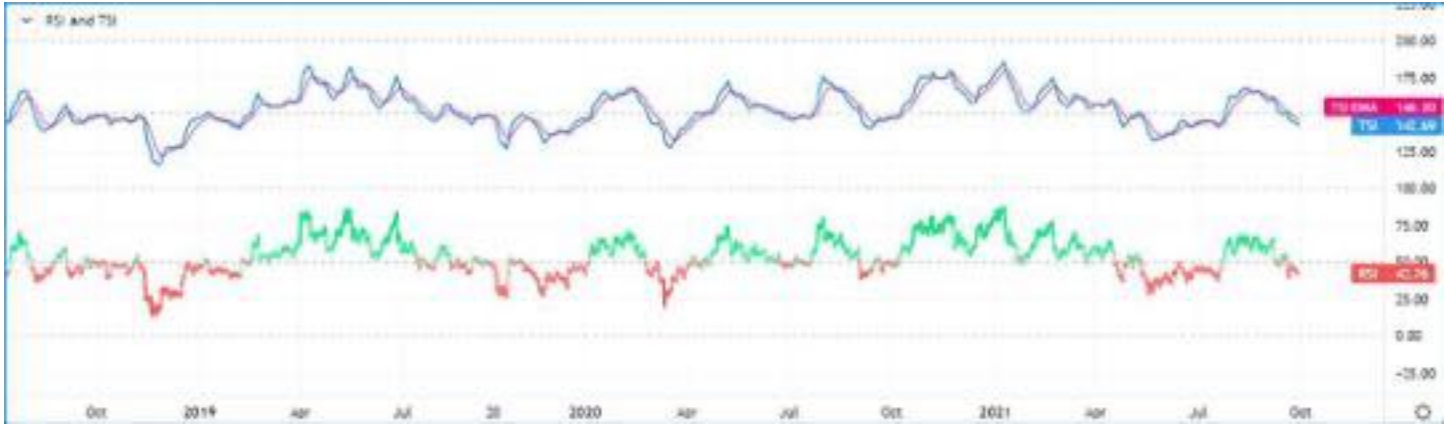


Figure 214: image

```
//@version=6
indicator("RSI and TSI")
myRSI = ta.rsi(close, 20)
bullColor = color.from_gradient(myRSI, 50, 80, color.new(color.lime, 70), color.new(color.lime, 0))
bearColor = color.from_gradient(myRSI, 20, 50, color.new(color.red, 0), color.new(color.red, 70))
myRSIColor = myRSI > 50 ? bullColor : bearColor
plot(myRSI, "RSI", myRSIColor, 3)
hline(100)
hline(50)
hline(0)

// 1. Compress TSI's range from -100/100 to -50/50.
// 2. Shift it higher by 150, so its -50 min value becomes 100.
myTSI = 150 + (100 * ta.tsi(close, 13, 25) / 2)
plot(myTSI, "TSI", color.blue, 2)
plot(ta.ema(myTSI, 13), "TSI EMA", #FF006E)
hline(200)
hline(150)
```

Note that:

- We have added levels using hline to situate both signals.
- In order for both signal lines to oscillate on the same range of 100, we divide the TSI value by 2 because it has a 200 range (-100 to +100). We then shift this value up by 150 so it oscillates between 100 and 200, making 150 its centerline.
- The manipulations we make here are typical of the compromises required to bring two indicators with different scales in the same visual space, even when their values, contrary to MACD, are bounded in a fixed range.

[Previous

Other timeframes and data](#other-timeframes-and-data)[Next

Repainting](#repainting) User Manual/Concepts/Repainting

Repainting

Introduction

We define repainting as: **script behavior causing historical vs realtime calculations or plots to behave differently.**

Repainting behavior is widespread and many factors can cause it. Following our definition, our estimate is that more than 95% of indicators in existence exhibit some form of repainting behavior. Commonly used indicators such as MACD and RSI, for example, show confirmed values on historical bars, but will fluctuate on a realtime, unconfirmed chart bar until it closes. Therefore, they behave *differently* in historical and realtime states.

Not all repainting behavior is inherently useless or misleading, nor does such behavior prevent knowledgeable traders from using indicators with such behavior. For example, who would think of discrediting a volume profile indicator solely because it updates its values on realtime bars?

One may encounter any of the following forms of repainting in the scripts they use, depending on what a script's calculations entail:

- **Widespread but often acceptable:** A script may use values that update with realtime price changes on the unconfirmed bar. For example, if one uses the close variable in calculations performed on an open chart bar, its values will reflect the most recent price in the bar. However, the script will only commit a new data point to its historical series once the bar closes. Another common case is using `request.security()` to fetch higher-timeframe data on realtime bars, as explained in the Historical and realtime behavior section of the Other timeframes and data page. As with the unconfirmed chart bar in the chart's timeframe, `request.security()` can track unconfirmed values from a higher-timeframe context on realtime bars, which can lead to repainting after the script restarts its execution. There is often nothing wrong with using such scripts, provided you understand how they work. When electing to use such scripts to issue alerts or trade orders, however, it's important to understand the difference between their realtime and historical behavior and decide for yourself whether it provides utility for your needs.
- **Potentially misleading:** Scripts that plot values into the past, calculate results on realtime bars that one cannot replicate on historical bars, or relocate past events are potentially misleading. For example, Ichimoku, most scripts based on pivots, most strategies using `calc_on_every_tick = true`, scripts using `request.security()` when it behaves differently on realtime bars, many scripts using `varip`, many scripts using `timenow`, and some scripts that use `barstate.*` variables can exhibit misleading repainting behavior.
- **Unacceptable:** Scripts that leak future information into the past, strategies that execute on non-standard charts, and scripts using realtime intrabars to generate alerts or orders, are examples that can produce heavily misleading repainting behavior.
- **Unavoidable:** Revisions of the data feed from a provider and variations in the starting bar of the chart's history can cause repainting behavior that may be unavoidable in a script.

The first two types of repainting can be perfectly acceptable if:

1. You are aware of the behavior.
2. You can live with it, or
3. You can circumvent it.

It should now be clear that not **all** repainting behavior is wrong and requires avoiding at all costs. In many situations, some forms of repainting may be exactly what a script needs. What's important is to know when repainting behavior is **not** acceptable for one's needs. To avoid repainting that's not acceptable, it's important to understand how a tool works or how you should design the tools you build. If you publish scripts, ensure you mention any potentially misleading behavior along with the other limitations of your script in the publication's description.

For script users

One can decide to use repainting indicators if they understand the behavior, and whether that behavior meets their analysis requirements. Don't be one of those newcomers who slap "repaint" sentences on published scripts in an attempt to discredit them, as doing so reveals a lack of foundational knowledge on the subject.

Simply asking whether a script repaints is relatively meaningless, given that there are forms of repainting behavior that are perfectly acceptable in a script. Therefore, such a question will not beget a meaningful answer. One should instead ask *specific* questions about a script's potential repainting behavior, such as:

- Does the script calculate/display in the same way on historical and realtime bars?
- Do alerts from the script wait for the end of a realtime bar before triggering?
- Do signal markers shown by the script wait for the end of a realtime bar before showing?
- Does the script plot/draw values into the past?

- Does the strategy use `calc_on_every_tick = true`?
- Do the script's `request.security()` calls leak future information into the past on historical bars?

What's important is that you understand how the tools you use work, and whether their behavior is compatible with your objectives, repainting or not. As you will learn if you read this page, repainting is a complex matter. It has many faces and many causes. Even if you don't program in Pine Script™, this page will help you understand the array of causes that can lead to repainting, and hopefully enable more meaningful discussions with script authors.

For Pine Script™ programmers

As discussed above, not all forms of repainting behavior must be avoided at all costs, nor is all potential repainting behavior necessarily avoidable. We hope this page helps you better understand the dynamics at play so that you can design your trading tools with these behaviors in mind. This page's content should help make you aware of common coding mistakes that produce misleading repainting results.

Whatever your design decisions are, if you publish your script, explain the script to traders so they can understand how it behaves.

This page covers three broad categories of repainting causes:

- Historical vs realtime calculations
- Plotting in the past
- Dataset variations

Historical vs realtime calculations

Fluid data values

Historical data does not include records of intermediary price movements on bars; only open, high, low and close values (OHLC).

On realtime bars (bars running when the instrument's market is open), however, the high, low and close values are not fixed; they can change values many times before the realtime bar closes and its HLC values are fixed. They are *fluid*. This leads to a script sometimes working differently on historical data and in real time, where only the open price will not change during the bar.

Any script using values like high, low and close in realtime is subject to producing calculations that may not be repeatable on historical bars — thus repaint.

Let's look at this simple script. It detects crosses of the close value (in the realtime bar, this corresponds to the current price of the instrument) over and under an EMA:



Figure 215: image

```
//@version=6
indicator("Repainting", "", true)
ma = ta.ema(close, 5)
xUp = ta.crossover(close, ma)
xDn = ta.crossunder(close, ma)
plot(ma, "MA", color.black, 2)
bgcolor(xUp ? color.new(color.lime, 80) : xDn ? color.new(color.fuchsia, 80) : na)
```

Note that:

- The script uses bgcolor() to color the background green when close crosses over the EMA, and red on crosses under the EMA.
- The screen snapshot shows the script in realtime on a 30sec chart. A cross over the EMA has been detected, thus the background of the realtime bar is green.
- The problem here is that nothing guarantees this condition will hold true until the end of the realtime bar. The arrow points to the timer showing that 21 seconds remain in the realtime bar, and anything could happen until then.
- We are witnessing a repainting script.

To prevent this repainting, we must rewrite our script so that it does not use values that fluctuate during the realtime bar. This will require using values from a bar that has elapsed (typically the preceding bar), or the open price, which does not vary in realtime.

We can achieve this in many ways. This method adds a **and barstate.isconfirmed** condition to our cross detections, which requires the script to be executing on the bar's last iteration, when it closes and prices are confirmed. It is a simple way to avoid repainting:

```
//@version=6
indicator("Repainting", "", true)
ma = ta.ema(close, 5)
xUp = ta.crossover(close, ma) and barstate.isconfirmed
xDn = ta.crossunder(close, ma) and barstate.isconfirmed
plot(ma, "MA", color.black, 2)
bgcolor(xUp ? color.new(color.lime, 80) : xDn ? color.new(color.fuchsia, 80) : na)
```

This uses the crosses detected on the previous bar:

```
//@version=6
indicator("Repainting", "", true)
ma = ta.ema(close, 5)
xUp = ta.crossover(close, ma)[1]
xDn = ta.crossunder(close, ma)[1]
plot(ma, "MA", color.black, 2)
bgcolor(xUp ? color.new(color.lime, 80) : xDn ? color.new(color.fuchsia, 80) : na)
```

This uses only confirmed close and EMA values for its calculations:

```
//@version=6
indicator("Repainting", "", true)
ma = ta.ema(close[1], 5)
xUp = ta.crossover(close[1], ma)
xDn = ta.crossunder(close[1], ma)
plot(ma, "MA", color.black, 2)
bgcolor(xUp ? color.new(color.lime, 80) : xDn ? color.new(color.fuchsia, 80) : na)
```

This detects crosses between the realtime bar's open and the value of the EMA from the previous bars. Notice that the EMA is calculated using close, so it repaints. We must ensure we use a confirmed value to detect crosses, thus **ma[1]** in the cross detection logic:

```
//@version=6
indicator("Repainting", "", true)
ma = ta.ema(close, 5)
xUp = ta.crossover(open, ma[1])
xDn = ta.crossunder(open, ma[1])
plot(ma, "MA", color.black, 2)
bgcolor(xUp ? color.new(color.lime, 80) : xDn ? color.new(color.fuchsia, 80) : na)
```

All these methods have one thing in common: while they prevent repainting, they will also trigger signals later than repainting scripts. This is an inevitable compromise if one wants to avoid repainting. You can't have your cake and eat it too.

Repainting request.security() calls

The request.security() function behaves differently on historical and realtime bars. On historical bars, it only returns *confirmed* values from its requested context, whereas it can return *unconfirmed* values on realtime bars. When the script restarts its execution, the bars that had a realtime state become historical bars, and will therefore only contain the values it confirmed on those bars. If the values returned by request.security() fluctuate on realtime bars without confirmation from the context, the script will repaint them when it restarts its execution. See the Historical and realtime behavior section of the Other timeframes and data page for a detailed explanation.

One can ensure higher-timeframe data requests only return confirmed values on all bars, regardless of bar state, by offsetting the **expression** argument by at least one bar with the history-referencing operator `[]` and using `barmerge.lookahead_on` for the `lookahead` argument in the request.security() call, as explained here.

The script below demonstrates the difference between repainting and non-repainting HTF data requests. It contains two request.security() calls. The first function call requests close data from the **higherTimeframe** without additional specification, and the second call requests the same series with an offset and `barmerge.lookahead_on`.

As we see on all realtime bars (the ones with an orange background), the **repaintingClose** contains values that fluctuate without confirmation from the **higherTimeframe**, meaning it will *repaint* when the script restarts its execution. The **nonRepaintingClose**, on the other hand, behaves the same on realtime and historical bars, i.e., it only changes its value when new, confirmed data is available:



Figure 216: image

```
//@version=6
indicator("Repainting vs non-repainting `request.security()` demo", overlay = true)

//@variable The timeframe to request data from.
string higherTimeframe = input.timeframe("30", "Timeframe")

if timeframe.in_seconds() > timeframe.in_seconds(higherTimeframe)
    runtime.error("The 'Timeframe' input is smaller than the chart's timeframe. Choose a higher timeframe.")

//@variable The current `close` requested from the `higherTimeframe`. Fluctuates without confirmation on realt
float repaintingClose = request.security(syminfo.tickerid, higherTimeframe, close)
//@variable The last confirmed `close` requested from the `higherTimeframe`.
// Behaves the same on historical and realtime bars.
float nonRepaintingClose = request.security(
```

```

    syminfo.tickerid, higherTimeframe, close[1], lookahead = barmerge.lookahead_on
)

// Plot the values.
plot(repaintingClose, "Repainting close", color.new(color.purple, 50), 8)
plot(nonRepaintingClose, "Non-repainting close", color.teal, 3)
// Plot a shape when a new `higherTimeframe` starts.
plotshape(timeframe.change(higherTimeframe), "Timeframe change marker", shape.square, location.top, size = size)
// Color the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 60) : na, title = "Realtime bar highlight")

```

Note that:

- We used the `plotshape()` function to mark the chart when there's a change on the `higherTimeframe`.
- This script produces a runtime error if the `higherTimeframe` is lower than the chart's timeframe.
- On historical bars, the `repaintingClose` has a new value at the *end* of each timeframe, and the `nonRepaintingClose` has a new value at the *start* of each timeframe.

For the sake of easy reusability, below is a simple `noRepaintSecurity()` function that one can apply in their scripts to request non-repainting higher-timeframe values:

```

//@function Requests non-repainting `expression` values from the context of the `symbol` and `timeframe`.
noRepaintSecurity(symbol, timeframe, expression) =>
    request.security(symbol, timeframe, expression[1], lookahead = barmerge.lookahead_on)

```

Note that:

- The `[1]` offset to the series and the use of `lookahead = barmerge.lookahead_on` are interdependent. One **cannot** be removed without compromising the integrity of the function.
- Unlike a plain `request.security()` call, this wrapper function cannot accept tuple `expression` arguments. For multi-element use cases, one can pass a user-defined type whose fields contain the desired elements to request.

Using `request.security()` at lower timeframes

Some scripts use `request.security()` to request data from a timeframe **lower** than the chart's timeframe. This can be useful when functions specifically designed to handle intrabars at lower timeframes are sent down the timeframe. When this type of user-defined function requires the detection of the intrabars' first bar, as most do, the technique will only work on historical bars. This is due to the fact that realtime intrabars are not yet sorted. The impact of this is that such scripts cannot reproduce in real time their behavior on historical bars. Any logic generating alerts, for example, will be flawed, and constant refreshing will be required to recalculate elapsed realtime bars as historical bars.

When used at lower timeframes than the chart's without specialized functions able to distinguish between intrabars, `request.security()` will only return the value of the **last** intrabar in the dilation of the chart's bar, which is usually not useful, and will also not reproduce in real time, so lead to repainting.

For all these reasons, unless you understand the subtleties of using `request.security()` at lower timeframes than the chart's, it is best to avoid using the function at those timeframes. Higher-quality scripts will have logic to detect such anomalies and prevent the display of results which would be invalid when a lower timeframe is used.

For more reliable lower-timeframe data requests, use `request.security_lower_tf()`, as explained in this section of the Other timeframes and data page.

Future leak with `request.security()`

When `request.security()` is used with `lookahead = barmerge.lookahead_on` to fetch prices without offsetting the series by `[1]`, it will return data from the future on historical bars, which is dangerously misleading.

While historical bars will magically display future prices before they should be known, no lookahead is possible in realtime because the future there is unknown, as it should, so no future bars exist.

This is an example:

```

// FUTURE LEAK! DO NOT USE!
//@version=6
indicator("Future leak", "", true)
futureHigh = request.security(syminfo.tickerid, "1D", high, lookahead = barmerge.lookahead_on)
plot(futureHigh)

```



Figure 217: image

Note how the higher timeframe line is showing the timeframe's high value before it occurs. The solution to avoid this effect is to use the function as demonstrated in this section.

Using lookahead to produce misleading results is not allowed in script publications, as explained in the lookahead section of the Other timeframes and data page. Script publications that use this misleading technique **will be moderated**.

varip

Scripts using the varip declaration mode for variables (see our section on varip for more information) save information across realtime updates, which cannot be reproduced on historical bars where only OHLC information is available. Such scripts may be useful in realtime, including to generate alerts, but their logic cannot be backtested, nor can their plots on historical bars reflect calculations that will be done in realtime.

Bar state built-ins

Scripts using bar states may or may not repaint. As we have seen in the previous section, using barstate.isconfirmed is actually one way to **avoid** repainting that **will** reproduce on historical bars, which are always “confirmed”. Uses of other bar states such as barstate.isnew, however, will lead to repainting. The reason is that on historical bars, barstate.isnew is **true** on the bar's close, yet in realtime, it is **true** on the bar's open. Using the other bar state variables will usually cause some type of behavioral discrepancy between historical and realtime bars.

timenow

The timenow built-in returns the current time. Scripts using this variable cannot show consistent historical and realtime behavior, so they necessarily repaint.

Strategies

Strategies using `calc_on_every_tick = true` execute on each realtime update, while strategies run on the close of historical bars. They will most probably not generate the same order executions, and so repaint. Note that when this happens, it also invalidates backtesting results, as they are not representative of the strategy's behavior in realtime.

Plotting in the past

Scripts detecting pivots after 5 bars have elapsed will often go back in the past to plot pivot levels or values on the actual pivot, 5 bars in the past. This will often cause unsuspecting traders looking at plots on historical bars to infer that when the pivot happens in realtime, the same plots will appear on the pivot when it occurs, as opposed to when it is detected.

Let's look at a script showing the price of high pivots by placing the price in the past, 5 bars after the pivot was detected:

```
//@version=6
indicator("Plotting in the past", "", true)
pHi = ta.pivohigh(5, 5)
if not na(pHi)
    label.new(bar_index[5], na, str.tostring(pHi, format.mintick) + "\n", yloc = yloc.abovebar, style = label.new)
```



Figure 218: image

Note that:

- This script repaints because an elapsed realtime bar showing no price may get a price placed on it if it is identified as a pivot, 5 bars after the actual pivot occurs.
- The display looks great, but it can be misleading.

The best solution to this problem when developing script for others is to plot **without** an offset by default, but give the option for script users to turn on plotting in the past through inputs, so they are necessarily aware of what the script is doing, e.g.:

```
//@version=6
indicator("Plotting in the past", "", true)
plotInThePast = input(false, "Plot in the past")
pHi = ta.pivohigh(5, 5)
if not na(pHi)
    label.new(bar_index[plotInThePast ? 5 : 0], na, str.tostring(pHi, format.mintick) + "\n", yloc = yloc.abovebar, style = label.new)
```

Dataset variations

Starting points

Scripts begin executing on the chart's first historical bar, and then execute on each bar sequentially, as is explained in this manual's page on Pine Script™'s execution model. If the first bar changes, then the script will often not calculate the same way it did when the dataset began at a different point in time.

The following factors have an impact on the quantity of bars you see on your charts, and their *starting point*:

- The type of account you hold
- The historical data available from the data supplier
- The alignment requirements of the dataset, which determine its *starting point*

These are the account-specific bar limits:

- 40000 historical bars for the Ultimate plan.
- 25000 historical bars for the Expert plan.

- 20000 historical bars for the Premium plan.
- 10000 historical bars for Essential and Plus plans.
- 5000 historical bars for other plans.

Starting points are determined using the following rules, which depend on the chart’s timeframe:

- **Tick-based timeframes:** return the exact number of bars based on the plan.
- **Second-based timeframes:** aligns to the beginning of a day.
- **1 - 14 minutes:** aligns to the beginning of a week.
- **15 - 29 minutes:** aligns to the beginning of a month.
- **30 - 1439 minutes:** aligns to the beginning of a year.
- **1440 minutes and higher:** aligns to the first available historical data point.

As time goes by, these factors cause your chart’s history to start at different points in time. This often has an impact on your scripts calculations, because changes in calculation results in early bars can ripple through all the other bars in the dataset. Using functions like `ta.valuewhen()`, `ta.barssince()` or `ta.ema()`, for example, will yield results that vary with early history.

Revision of historical data

Historical and realtime bars are built using two different data feeds supplied by exchanges/brokers: historical data, and realtime data. When realtime bars elapse, exchanges/brokers sometimes make what are usually small adjustments to bar prices, which are then written to their historical data. When the chart is refreshed or the script is re-executed on those elapsed realtime bars, they will then be built and calculated using the historical data, which will contain those usually small price revisions, if any have been made.

Historical data may also be revised for other reasons, e.g., for stock splits.

[Previous

Plots](#plots)[Next

Sessions](#sessions) User Manual/Concepts/Sessions

Sessions

Introduction

Session information is usable in three different ways in Pine Script™:

1. **Session strings** containing from-to start times and day information that can be used in functions such as `time()` and `time_close()` to detect when bars are in a particular time period, with the option of limiting valid sessions to specific days. The `input.session()` function provides a way to allow script users to define session values through a script’s “Inputs” tab (see the Session input section for more information).
2. **Session states** built-in variables such as `session.ismarket` can identify which session a bar belongs to.
3. When fetching data with `request.security()` you can also choose to return data from *regular* sessions only or *extended* sessions. In this case, the definition of **regular and extended sessions** is that of the exchange. It is part of the instrument’s properties — not user-defined, as in point #1. This notion of *regular* and *extended* sessions is the same one used in the chart’s interface, in the “Chart Settings/Symbol/Session” field, for example.

The following sections cover both methods of using session information in Pine Script™.

Note that:

- Not all user accounts on TradingView have access to extended session information.
- There is no special “session” type in Pine Script™. Instead, session strings are of “string” type but must conform to the session string syntax.

Session strings

Session string specifications

Session strings used with `time()` and `time_close()` must have a specific format. Their syntax is:

`<time_period>:<days>`

Where:

- uses times in “hhmm” format, with “hh” in 24-hour format, so 1700 for 5PM.

The time periods are in the “hhmm-hhmm” format, and a comma can separate multiple time periods to specify combinations of discrete periods.

For example, - is a set of digits from 1 to 7 that specifies on which days the session is valid.

1 is Sunday, 7 is Saturday.

These are examples of session strings:

"24x7"

A 7-day, 24-hour session beginning at midnight.

"0000-0000:1234567"

Equivalent to the previous example.

"0000-0000"

Equivalent to the previous two examples because the default days are 1234567.

"0000-0000:23456"

The same as the previous example, but only Monday to Friday.

"2000-1630:1234567"

An overnight session that begins at 20:00 and ends at 16:30 the next day. It is valid on all days of the week.

"0930-1700:146"

A session that begins at 9:30 and ends at 17:00 on Sundays (1), Wednesdays (4), and Fridays (6).

"1700-1700:23456"

An *overnight session*. The Monday session starts Sunday at 17:00 and ends Monday at 17:00. It is valid Monday through Friday.

"1000-1001:26"

A weird session that lasts only one minute on Mondays (2) and Fridays (6).

"0900-1600,1700-2000"

A session that begins at 9:00, breaks from 16:00 to 17:00, and continues until 20:00. Applies to every day of the week.

Using session strings

Session properties defined with session strings are independent of the exchange-defined sessions determining when an instrument can be traded. Programmers have complete liberty in creating whatever session definitions suit their purpose, which is usually to detect when bars belong to specific time periods. This is accomplished in Pine Script™ by using one of the following two signatures of the `time()` function:

```
time(timeframe, session, timezone) → series int
time(timeframe, session) → series int
```

Here, we use `time()` with a `session` argument to display the market's opening high and low values on an intraday chart:

```
//@version=6
indicator("Opening high/low", overlay = true)

sessionInput = input.session("0930-0935")

sessionBegins(sess) =>
    t = time("", sess)
    timeframe.isintraday and (not barstate.isfirst) and na(t[1]) and not na(t)

var float hi = na
var float lo = na
if sessionBegins(sessionInput)
    hi := high
    lo := low
```



Figure 219: image

```
plot(lo, "lo", color.fuchsia, 2, plot.style_circles)
plot(hi, "hi", color.lime, 2, plot.style_circles)
```

Note that:

- We use a session input to allow users to specify the time they want to detect. We are only looking for the session's beginning time on bars, so we use a five-minute gap between the beginning and end time of our "0930-0935" default value.
- We create a `sessionBegins()` function to detect the beginning of a session. Its `time("", sess)` call uses an empty string for the function's `timeframe` parameter, which means it uses the chart's timeframe, whatever that is. The function returns `true` when:
 - The chart uses an intraday timeframe (seconds or minutes).
 - The script isn't on the chart's first bar, which we ensure with `(not barstate.isfirst)`. This check prevents the code from always detecting a session beginning on the first bar because `na(t[1])` and `not na(t)` is always `true` there.
 - The `time()` call has returned `na` on the previous bar because it wasn't in the session's time period, and it has returned a value that is not `na` on the current bar, which means the bar is **in** the session's time period.

Session states

Three built-in variables allow you to distinguish the type of session the current bar belongs to. They are only helpful on intraday timeframes:

- `session.ismarket` returns `true` when the bar belongs to regular trading hours.
- `session.ispremarket` returns `true` when the bar belongs to the extended session preceding regular trading hours.
- `session.ispostmarket` returns `true` when the bar belongs to the extended session following regular trading hours.

Using sessions with `request.security()`

When your TradingView account provides access to extended sessions, you can choose to see their bars with the "Settings/Symbol/Session" field. There are two types of sessions:

- **regular** (which does not include pre- and post-market data), and
- **extended** (which includes pre- and post-market data).

Scripts using the `request.security()` function to access data can return extended session data or not. This is an example where only regular session data is fetched:

```
//@version=6
indicator("Example 1: Regular Session Data")
regularSessionData = request.security("NASDAQ:AAPL", timeframe.period, close, barmerge.gaps_on)
plot(regularSessionData, style = plot.style_linebr)
```



Figure 220: image

If you want the `request.security()` call to return extended session data, you must first use the `ticker.new()` function to build the first argument of the `request.security()` call:



Figure 221: image

```
//@version=6
indicator("Example 2: Extended Session Data")
t = ticker.new("NASDAQ", "AAPL", session.extended)
extendedSessionData = request.security(t, timeframe.period, close, barmerge.gaps_on)
plot(extendedSessionData, style = plot.style_linebr)
```

Note that the previous chart's gaps in the script's plot are now filled. Also, keep in mind that our example scripts do not produce the background coloring on the chart; it is due to the chart's settings showing extended hours.

The `ticker.new()` function has the following signature:

`ticker.new(prefix, ticker, session, adjustment) → simple string`

Where:

- `prefix` is the exchange prefix, e.g., "NASDAQ"
- `ticker` is a symbol name, e.g., "AAPL"
- `session` can be `session.extended` or `session.regular`. Note that this is **not** a session string.
- `adjustment` adjusts prices using different criteria: `adjustment.none`, `adjustment.splits`, `adjustment.dividends`.

Our first example could be rewritten as:

```
//@version=6
indicator("Example 1: Regular Session Data")
t = ticker.new("NASDAQ", "AAPL", session.regular)
regularSessionData = request.security(t, timeframe.period, close, barmerge.gaps_on)
plot(regularSessionData, style = plot.style_linebr)
```

If you want to use the same session specifications used for the chart's main symbol, omit the third argument in `ticker.new()`; it is optional. If you want your code to declare your intention explicitly, use the `syminfo.session` built-in variable. It holds the session type of the chart's main symbol:

```
//@version=6
indicator("Example 1: Regular Session Data")
t = ticker.new("NASDAQ", "AAPL", syminfo.session)
regularSessionData = request.security(t, timeframe.period, close, barmerge.gaps_on)
plot(regularSessionData, style = plot.style_linebr)
```

[Previous

Repainting](#repainting)[Next

Strategies](#strategies) User Manual/Concepts/Strategies

Strategies

Introduction

Pine Script™ Strategies are specialized scripts that simulate trades across historical and realtime bars, allowing users to backtest and forward test their trading systems. Strategy scripts have many of the same capabilities as indicator scripts, and they provide the ability to place, modify, and cancel hypothetical orders and analyze performance results.

When a script uses the `strategy()` function as its declaration statement, it gains access to the `strategy.*` namespace, which features numerous functions and variables for simulating orders and retrieving essential strategy information. It also displays relevant information and simulated performance results in the dedicated Strategy Tester tab.

A simple strategy example

The following script is a simple strategy that simulates entering a long or short position when two moving averages cross. When the `fastMA` crosses above the `slowMA`, it places a “buy” market order to enter a long position. When the `fastMA` crosses below the `slowMA`, it places a “sell” market order to enter a short position:

```
//@version=6
strategy("Simple strategy demo", overlay = true, margin_long = 100, margin_short = 100)

//@variable The length of the `fastMA` and half the length of the `slowMA`.
int lengthInput = input.int(14, "Base length", 2)

// Calculate two moving averages with different lengths.
float fastMA = ta.sma(close, lengthInput)
float slowMA = ta.sma(close, lengthInput * 2)

// Place an order to enter a long position when `fastMA` crosses over `slowMA`.
if ta.crossover(fastMA, slowMA)
    strategy.entry("buy", strategy.long)

// Place an order to enter a short position when `fastMA` crosses under `slowMA`.
if ta.crossunder(fastMA, slowMA)
    strategy.entry("sell", strategy.short)

// Plot the moving averages.
plot(fastMA, "Fast MA", color.aqua)
plot(slowMA, "Slow MA", color.orange)
```

Note that:

- The `strategy()` function call declares that the script is a strategy named “Simple strategy demo” that displays visuals on the main chart pane.
- The `margin_long` and `margin_short` arguments in the `strategy()` call specify that the strategy must have 100% of a long or short trade’s amount available to allow the trade. See this section for more information.
- The `strategy.entry()` function is the command that the script uses to create entry orders and reverse positions. The “buy” entry order closes any short position and opens a new long position. The “sell” entry order closes any long position and opens a new short position.

Applying a strategy to a chart

To test a strategy, add it to the chart. Select a built-in or published strategy from the “Indicators, Metrics & Strategies” menu, or write a custom strategy in the Pine Editor and click the “Add to chart” option in the top-right corner:



Figure 222: image

The script plots trade markers on the main chart pane and displays simulated performance results inside the Strategy Tester tab:

Strategy Tester

The *Strategy Tester* visualizes the hypothetical performance of a strategy script and displays its properties. To use it, add a script declared with the `strategy()` function to the chart, then open the “Strategy Tester” tab. If two or more strategies are on the chart, specify which one to analyze by selecting its name in the top-left corner.

After the selected script executes across the chart’s data, the Strategy Tester populates the following four tabs with relevant strategy information:

- Overview
- Performance Summary
- List of Trades
- Properties

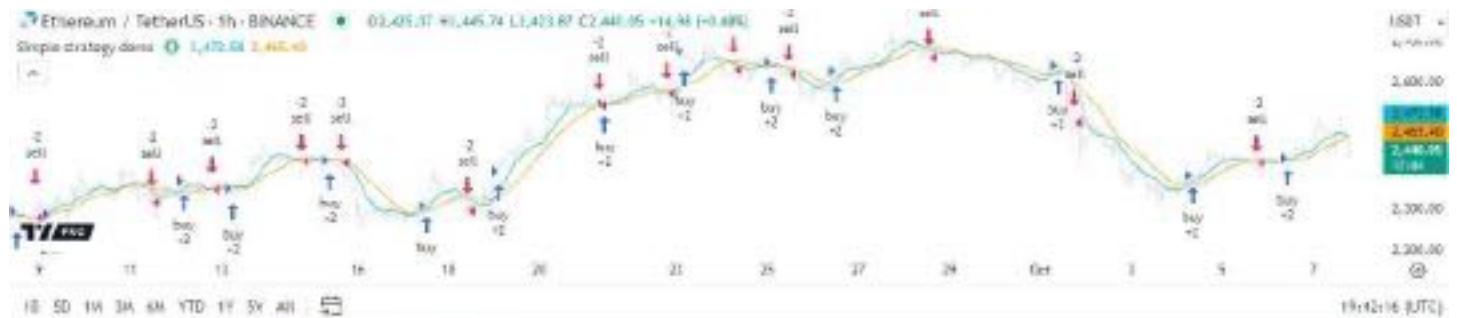
Overview

The Overview tab provides a quick look into a strategy’s performance over a sequence of simulated trades. This tab displays essential performance metrics and a chart with three helpful plots:

- The Equity baseline plot visualizes the strategy’s simulated equity across closed trades.
- The Drawdown column plot shows how far the strategy’s equity fell below its peak across trades.
- The Buy & hold equity plot shows the equity growth of a strategy that enters a single long position and holds that position throughout the testing range.

Note that:

- The chart has two separate vertical scales. The “Equity” and “Buy & hold equity” plots use the scale on the left, and the “Drawdown” plot uses the scale on the right. Users can toggle the plots and choose between absolute or percentage scales using the options at the bottom.



Form Screener - Pine Editor Strategy Tester Replay Trading Trading Panel

Simple strategy demo

Deep Backtesting

Overview Performance Summary List of Trades Properties

Net Profit 2,837.54 USDT 0.28% Total Closed Trades 1,665 Percent Profitable 36.21% Profit Factor 1.073 Max Drawdown 2,612.62 USDT -0.26% Avg Trade 1.70 USDT 0.18% Avg # Bars in Trades 26

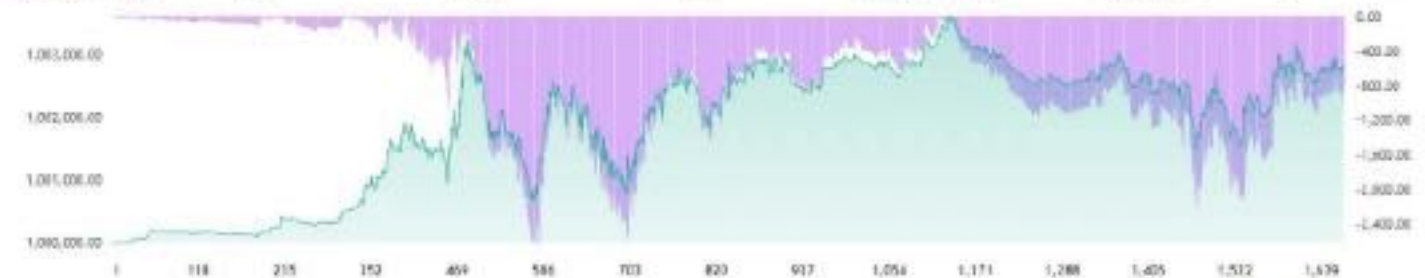


Figure 223: image

Simple strategy demo

Deep Backtesting

Overview Performance Summary List of Trades Properties

Net Profit 2,837.54 USDT 0.28% Total Closed Trades 1,665 Percent Profitable 36.21% Profit Factor 1.073 Max Drawdown 2,612.62 USDT -0.26% Avg Trade 1.70 USDT 0.18% Avg # Bars in Trades 26

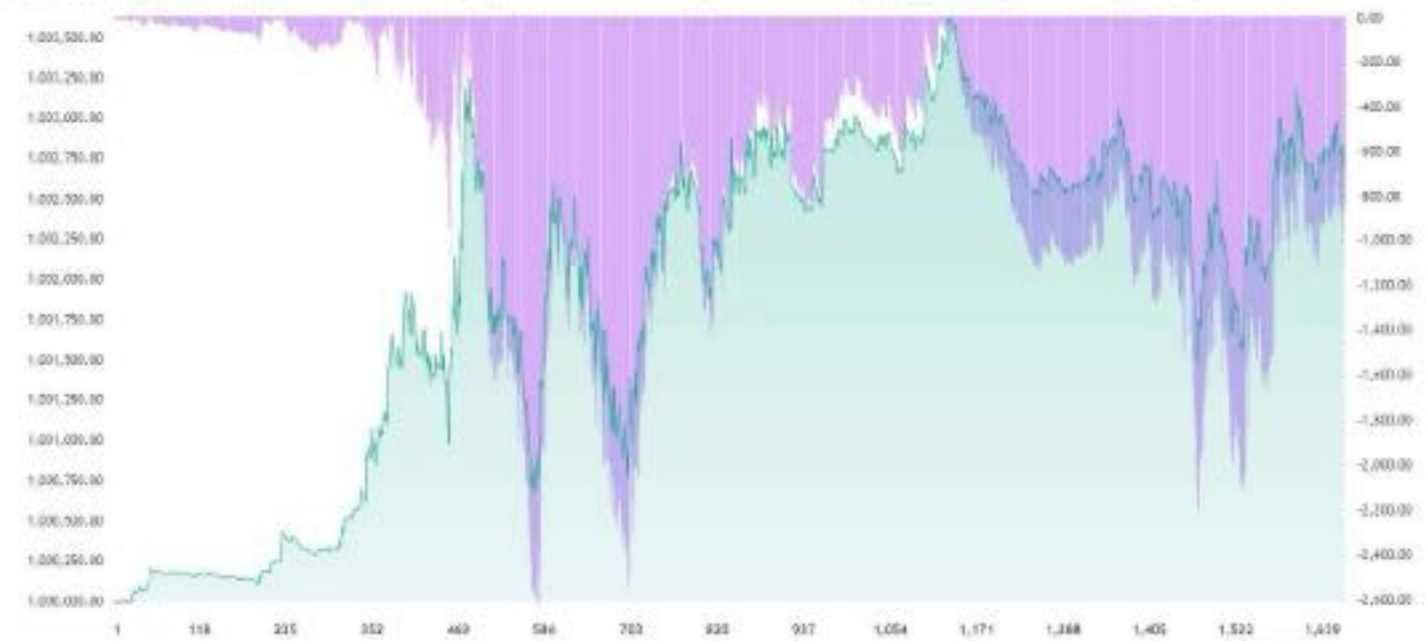


Figure 224: image

- When a user clicks on a point in this chart, the main chart scrolls to the corresponding bar where the trade closed and displays a tooltip containing the closing time.

Performance Summary

The Performance Summary tab presents an in-depth summary of a strategy's key performance metrics, organized into separate columns. The "All" column shows performance information for all simulated trades, and the "Long" and "Short" columns show relevant metrics separately for long and short trades. This view provides more detailed insights into a strategy's overall and directional trading performance:

	All	Long	Short
Net Profit	2,831.66 USDT 0.28%	2,562.12 USDT 0.16%	269.54 USDT 0.03%
Gross Profit	46,609.83 USDT 4.08%	20,989.86 USDT 2.10%	19,620.17 USDT 1.96%
Gross Loss	37,738.17 USDT 3.78%	18,427.34 USDT 1.84%	19,310.83 USDT 1.94%
Max Run-up	3,699.43 USDT 0.37%		
Max Drawdown	2,612.62 USDT 0.26%		
Buy & Hold Return	17,943,577.14 USDT 1,780.36%		
Sharpe Ratio	-3.36		
Sortino Ratio	-8.955		
Profit Factor	1.075	1.139	1.014
Max Contracts Held	1	8	5
Open P/L	24.68 USDT 0.08%		
Commission Paid	0 USDT	0 USDT	0 USDT
Total Closed Trades	1,645	834	835
Total Open Trades	1	8	9
Number Winning Trades	696	317	239
Number Losing Trades	1,062	515	596
Maximum Drawdown	36.09%	38.09%	34.60%

Figure 225: image

List of Trades

The List of Trades tab chronologically lists a strategy's simulated trades. Each item in the list displays vital information about a trade, including the dates and times of entry and exit orders, the names of the orders, the order prices, and the number of contracts/shares/lots/units. In addition, each item shows the trade's profit or loss and the strategy's cumulative profit, run-up, and drawdown:

Note that:

- Hovering the mouse over a list item's entry or exit information reveals a "Scroll to bar" button. Clicking that button navigates the main chart to the bar where the entry or exit occurred.
- The list shows each trade in *descending* order by default, with the latest trade at the top. Users can reverse this order by clicking the "Trade #" button above the list.

Properties

The "Properties" tab provides detailed information about a strategy's configuration and the dataset that it executes across, organized into four collapsible sections:

- The "Date Range" section shows the range of dates that had simulated trades, and the overall available backtesting range.
- The "Symbol Info" section displays the chart's symbol, timeframe, type, point value, currency, and tick size. It also includes the chart's specified precision setting.

Simple strategy demo    Deep Backtesting

Overview	Performance Summary	List of Trades	Properties						
Trade #	Type	Signal	Date/Time	Price	Contracts	Profit	Com. Profit	Run-up	Drawdown
1630	Exit Long	Open							
	Entry Long	buy	2024-10-06 10:00	2,422.81	1	24.51 USD 1.81%	2,954.17 USD 0.90%	98.99 USD 4.09%	4.34 USD 0.28%
1649	Exit Short	buy	2024-10-06 10:00	2,422.81	1	-11.85 USD -0.49%	2,851.64 USD 0.90%	20.11 USD 0.80%	57.94 USD 0.74%
	Entry Short	sell	2024-10-05 18:00	2,410.16					
1668	Exit Long	sell	2024-10-05 18:00	2,410.16	1	28.36 USD 1.20%	2,841.51 USD 0.90%	60.04 USD 2.52%	29.88 USD 1.23%
	Entry Long	buy	2024-10-04 08:00	2,381.60					
1667	Exit Short	buy	2024-10-04 08:00	2,381.60	1	624.39 USD 4.76%	2,814.95 USD 0.91%	195.99 USD 7.82%	11.75 USD 0.50%
	Entry Short	sell	2024-10-01 18:00	2,585.99					
1666	Exit Long	sell	2024-10-01 18:00	2,585.99	1	-135.00 USD -5.19%	2,696.36 USD 0.91%	4.48 USD 0.17%	271.85 USD 8.06%
	Entry Long	buy	2024-10-01 09:00	2,540.99					
1683	Exit Short	buy	2024-10-01 09:00	2,540.99	1	17.92 USD 0.77%	2,825.56 USD 0.90%	83.91 USD 3.16%	25.68 USD 0.96%
	Entry Short	sell	2024-09-28 13:00	2,558.91					
1664	Exit Long	sell	2024-09-28 13:00	2,558.91	1	30.12 USD 1.18%	2,807.64 USD 0.90%	99.81 USD 3.56%	29.87 USD 0.74%

Figure 226: image

- The “Strategy Inputs” section lists the names and values of all the inputs available in the strategy’s “Settings/Inputs” tab. This section only appears if the script includes `input*()` calls or specifies a nonzero `calc_bars_count` argument in the `strategy()` declaration statement.
- The “Strategy Properties” section provides an overview of the strategy’s properties, including the initial capital, account currency, order size, margin, pyramiding, commission, slippage, and other settings.

Simple strategy demo    Deep Backtesting

Overview	Performance Summary	List of Trades	Properties
Date range Trading range: 2020-01-02 07:00 – 2024-10-07 20:00, Backtesting range: 2020-01-01 08:00 – 2024-10-07 20:00 Symbol info Symbol: BINANCE:ETH/USD, Timeframe: 1 hour, Chart type: Candles, Currency: USD, Tick Size: 0.01, Precision: Default Strategy inputs Bar length: 64 Strategy properties Initial capital: 1000000 USD, Order size: 1 Contracts, Pyramiding: 1 orders, Commission: 0%, Slippage: 0 ticks			

Figure 227: image

Broker emulator

TradingView uses a *broker emulator* to simulate trades while running a strategy script. Unlike in real-world trading, the emulator fills a strategy’s orders exclusively using available *chart data* by default. Consequently, it executes orders on historical bars *after a bar closes*. Similarly, the earliest point that it can fill orders on realtime bars is after a new price tick. For more information about this behavior, see the Execution model page.

Because the broker emulator only uses price data from the chart by default, it makes *assumptions* about intrabar price movement when filling orders. The emulator analyzes the opening, high, low, and closing prices of chart bars to infer intrabar activity using the following logic:

- If the opening price of a bar is closer to the high than the low, the emulator assumes that the market price moved in this order: **open** → **high** → **low** → **close**.
- If the opening price of a bar is closer to the low than the high, the emulator assumes that the market price moved in this order: **open** → **low** → **high** → **close**.
- The emulator assumes *no gaps* exist between intrabars inside each chart bar, meaning it considers *any* value within a bar's high-low range as a valid price for order execution.
- When filling *price-based orders* (all orders except market orders), the emulator assumes intrabars **do not** exist within the gap between the previous bar's close and the current bar's open. If the market price crosses an order's price during the gap between two bars, the emulator fills the order at the current bar's *open* and not at the specified price.

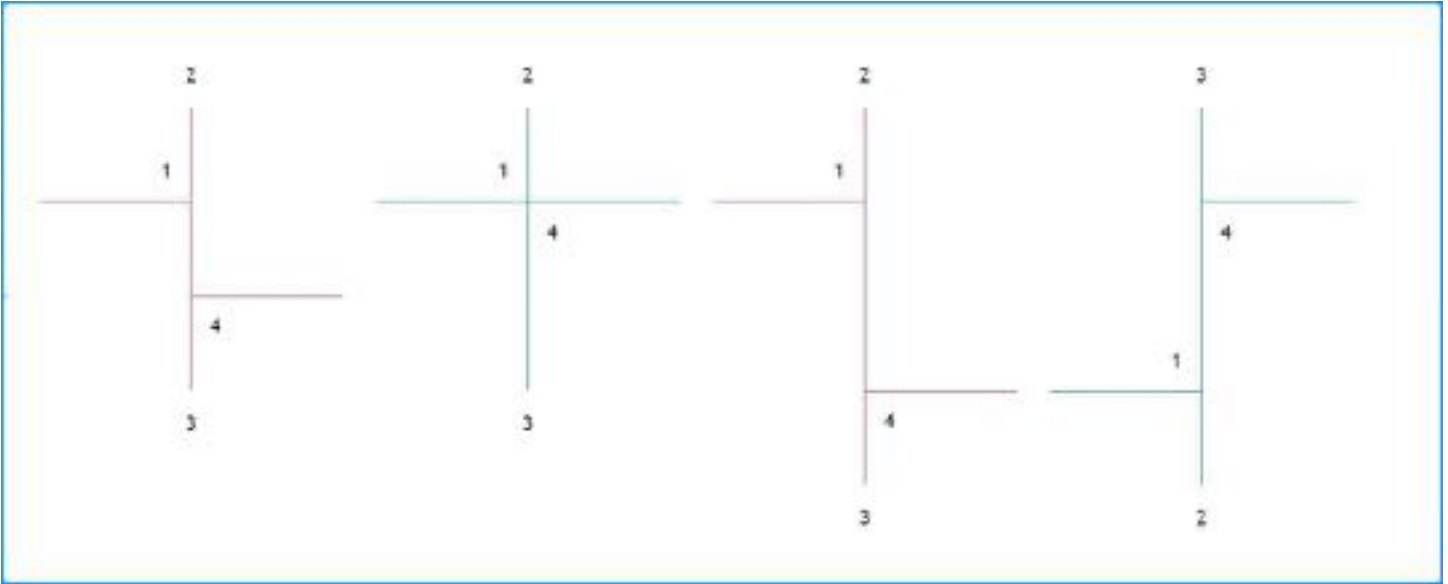


Figure 228: image

Bar magnifier

Users with Premium and higher-tier plans can override the broker emulator's default assumptions about intrabar prices by enabling the Bar Magnifier backtesting mode. In this mode, the emulator uses data from *lower timeframes* to obtain more granular information about price action within bars, allowing more precise order fills in the strategy's simulation.

To enable the Bar Magnifier mode, include `use_bar_magnifier = true` in the `strategy()` declaration statement, or select the "Using bar magnifier" option in the "Fill orders" section of the strategy's "Settings/Properties" tab.

The following example script illustrates how the Bar Magnifier can enhance order-fill behavior. When the time value crosses the defined `orderTime`, it creates "Buy" and "Exit" limit orders at the calculated `entryPrice` and `exitPrice`. For visual reference, the script colors the background orange when it places the orders, and it draws two horizontal lines at the order prices:

```
//@version=6
strategy("Bar Magnifier Demo", overlay = true, use_bar_magnifier = false)

//@variable The UNIX timestamp to place the order at.
int orderTime = timestamp("UTC", 2023, 3, 22, 18)

//@variable Is `color.orange` when `time` crosses the `orderTime`, false otherwise.
color orderColor = na

// Entry and exit prices.
float entryPrice = hl2 - (high - low)
float exitPrice = entryPrice + (high - low) * 0.25

// Entry and exit lines.
var line entryLine = na
var line exitLine = na
```



Figure 229: image

```

if ta.cross(time, orderTime)
    // Draw new entry and exit lines.
    entryLine := line.new(bar_index, entryPrice, bar_index + 1, entryPrice, color = color.green, width = 2)
    exitLine  := line.new(bar_index, exitPrice, bar_index + 1, exitPrice, color = color.red, width = 2)

    // Update order highlight color.
    orderColor := color.new(color.orange, 80)

    // Place limit orders at the `entryPrice` and `exitPrice`.
    strategy.entry("Buy", strategy.long, limit = entryPrice)
    strategy.exit("Exit", "Buy", limit = exitPrice)

// Update lines while the position is open.
else if strategy.position_size > 0.0
    entryLine.set_x2(bar_index + 1)
    exitLine.set_x2(bar_index + 1)

bgcolor(orderColor)

```

Because the script does not include a `use_bar_magnifier` argument in the `strategy()` function, the broker emulator uses the default assumptions when filling the orders: that the bar's price moved from open to high, high to low, and then low to close. Therefore, after filling the “Buy” order at the price indicated by the green line, the broker emulator inferred that the market price did not go back up to touch the red line and trigger the “Exit” order. In other words, the strategy *could not* enter and exit the position on the same bar according to the broker emulator's assumptions.

After we enable the Bar Magnifier mode, the broker emulator can access *10-minute* data on the 60-minute chart instead of relying on its assumptions about hourly bars. On this timeframe, the market price *did* move back up to the “Exit” order's price after reaching the “Buy” order's price in the same hour. Therefore, with the Bar Magnifier enabled in this scenario, both orders execute on the same hourly bar:



Figure 230: image

Orders and trades

Pine Script™ strategies use orders to make trades and manage positions, similar to real-world trading. In this context, an *order* is an instruction that a strategy sends to the broker emulator to perform a market action, and a *trade* is the resulting transaction after the emulator fills an order.

Let's take a closer look at how strategy orders work and how they become trades. Every 20 bars, the following script creates a long market order with `strategy.entry()` and draws a label. It calls `strategy.close_all()` on each bar from the global scope to generate a market order to close any open position:

```
//@version=6
strategy("Order execution demo", "My strategy", true, margin_long = 100, margin_short = 100)

//@function Displays the specified `txt` in a label at the `high` of the current bar.
debugLabel(string txt) =>
    label.new(
        bar_index, high, text = txt, color=color.lime, style = label.style_label_lower_right,
        textcolor = color.black, size = size.large
    )

//@variable Is `true` on every 20th bar, `false` otherwise.
bool longCondition = bar_index % 20 == 0

// Draw a label and place a long market order when `longCondition` occurs.
if longCondition
    debugLabel("Long entry order created")
    strategy.entry("My Long Entry Id", strategy.long)

// Place a closing market order whenever there is an open position.
strategy.close_all()
```

Note that:

- Although the script calls `strategy.close_all()` on every bar, the function only creates a new exit order when the strategy has an *open position*. If there is no open position, the function call has no effect.



Figure 231: image

The blue arrows on the above chart show where the strategy entered a long position, and the purple arrows mark the bars where the strategy closed the position. Notice that the label drawings appear one bar *before* the entry markers, and the entry markers appear one bar *before* the closing markers. This sequence illustrates order creation and execution in action.

By default, the earliest point the broker emulator fills an order is on the next available price tick, because creating and filling an order on the same tick is unrealistic. Since strategies recalculate after each bar closes by default, the next available tick where the emulator fills a generated order is at the *open* of the *following* bar. For example, when the `longCondition` occurs on bar 20, the script places an entry order to fill on the next tick, which is at the open of bar 21. When the strategy recalculates its values after bar 21 closes, it places an order to close the current position on the next tick, which is at the open of bar 22.

Order types

Pine Script™ strategies can simulate different order types to suit specific trading system needs. The main notable order types include market, limit, stop, and stop-limit.

Market orders

A *market order* is the simplest type of order, which most order placement commands generate by default. A market order is an instruction to buy or sell a security as soon as possible, irrespective of the price. As such, the broker emulator always executes a market order on the next available tick.

The example below alternates between placing a long and short market order once every `lengthInput` bars. When the `bar_index` is divisible by $2 * \text{lengthInput}$, the strategy generates a long market order. Otherwise, it places a short market order when the `bar_index` is divisible by the `lengthInput`:

```
//@version=6
strategy("Market order demo", overlay = true, margin_long = 100, margin_short = 100)

//@variable Number of bars between long and short entries.
int lengthInput = input.int(10, "Cycle length", 1)

//@function Displays the specified `txt` in a label on the current bar.
debugLabel(string txt, color lblColor) => label.new(
    bar_index, high, text = txt, color = lblColor, textcolor = color.white,
```



Figure 232: image

```

    style = label.style_label_lower_right, size = size.large
)

//@variable Is `true` every `2 * lengthInput` bars, `false` otherwise.
longCondition = bar_index % (2 * lengthInput) == 0
//@variable Is `true` every `lengthInput` bars, `false` otherwise.
shortCondition = bar_index % lengthInput == 0

// Generate a long market order with a `color.green` label on `longCondition`.
if longCondition
    debugLabel("Long market order created", color.green)
    strategy.entry("My Long Entry Id", strategy.long)
// Otherwise, generate a short market order with a `color.red` label on `shortCondition`.
else if shortCondition
    debugLabel("Short market order created", color.red)
    strategy.entry("My Short Entry Id", strategy.short)

```

Note that:

- The labels indicate the bars where the script generates the market orders. The broker emulator fills each order at the open of the following bar.
- The `strategy.entry()` command can automatically *reverse* an open position in the opposite direction. See this section below for more information.

Limit orders

A *limit order* is an instruction to buy or sell a security at a specific price or better (lower than specified for long orders, and higher than specified for short orders), irrespective of the time. To simulate a limit order in a strategy script, pass a *price* value to the `limit` parameter of an applicable order placement command.

When the market price reaches a limit order's value, or crosses it in the favorable direction, the broker emulator fills the order at that value or a better price. When a strategy generates a limit order at a *worse* value than the current market price (higher for long orders and lower for short orders), the emulator fills the order without waiting for the market price to reach that value.

For example, the following script generates a long limit order 800 ticks below the close of the bar 100 bars before the last

chart bar using the `strategy.entry()` command. It draws a label to signify the bar where the strategy created the order and a line to visualize the order's price:



Figure 233: image

```
//@version=6
strategy("Limit order demo", overlay = true, margin_long = 100, margin_short = 100)

//@function Displays text passed to `txt` and a horizontal line at `price` when called.
debugLabel(float price, string txt) =>
    label.new(
        bar_index, price, text = txt, color = color.teal, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    line.new(
        bar_index, price, bar_index + 1, price, color = color.teal, extend = extend.right,
        style = line.style_dashed
    )

// Generate a long limit order with a label and line 100 bars before the `last_bar_index`.
if last_bar_index - bar_index == 100
    limitPrice = close - syminfo.mintick * 800
    debugLabel(limitPrice, "Long Limit order created")
    strategy.entry("Long", strategy.long, limit = limitPrice)
```

Notice that in the chart above, the label and the start of the line occurred several bars before the “Long” entry marker. The broker emulator could not fill the order while the market price remained *above* the `limitPrice` because such a price is a *worse* value for the long trade. After the price fell and reached the `limitPrice`, the emulator filled the order mid-bar at that value.

If we set the `limitPrice` to a value *above* the bar’s close rather than *below*, the broker emulator fills the order at the open of the following bar because the closing price is already a more *favorable* value for the long trade. Here, we set the `limitPrice` in the script to 800 ticks above the bar’s close to demonstrate this effect:

```
//@version=6
strategy("Limit order demo", overlay = true, margin_long = 100, margin_short = 100)

//@function Displays text passed to `txt` and a horizontal line at `price` when called.
```



Figure 234: image

```
debugLabel(float price, string txt) =>
    label.new(
        bar_index, price, text = txt, color = color.teal, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    line.new(
        bar_index, price, bar_index + 1, price, color = color.teal, extend = extend.right,
        style = line.style_dashed
    )

// Generate a long limit order with a label and line 100 bars before the `last_bar_index`.
if last_bar_index - bar_index == 100
    limitPrice = close + syminfo.mintick * 800
    debugLabel(limitPrice, "Long Limit order created")
    strategy.entry("Long", strategy.long, limit = limitPrice)
```

Stop and stop-limit orders

A *stop order* is an instruction to activate a new market or limit order when the market price reaches a specific price or a worse value (higher than specified for long orders and lower than specified for short orders). To simulate a stop order, pass a price value to the `stop` parameter of an applicable order placement command.

When a strategy generates a stop order at a *better* value than the current market price, it activates the subsequent order without waiting for the market price to reach that value.

The following example calls `strategy.entry()` to place a stop order 800 ticks above the close 100 bars before the last historical chart bar. It also draws a label on the bar where it created the order and a line to display the stop price. As we see in the chart below, the strategy entered a long position immediately after the price crossed the stop level:

```
//@version=6
strategy("Stop order demo", overlay = true, margin_long = 100, margin_short = 100)

//@function Displays text passed to `txt` when called and shows the `price` level on the chart.
debugLabel(price, txt) =>
    label.new(
```



Figure 235: image

```

        bar_index, high, text = txt, color = color.teal, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    line.new(bar_index, high, bar_index, price, style = line.style_dotted, color = color.teal)
    line.new(
        bar_index, price, bar_index + 1, price, color = color.teal, extend = extend.right,
        style = line.style_dashed
    )

// Generate a long stop order with a label and lines 100 bars before the last bar.
if last_bar_index - bar_index == 100
    stopPrice = close + syminfo.mintick * 800
    debugLabel(stopPrice, "Long Stop order created")
    strategy.entry("Long", strategy.long, stop = stopPrice)

```

Note that:

- A basic stop order is essentially the opposite of a limit order in terms of its execution based on the market price. If we use a limit order instead of a stop order in this scenario, the order executes immediately on the next bar. See the previous section for an example.

When a `strategy.entry()` or `strategy.order()` call includes a `stopandlimit` argument, it creates a *stop-limit order*. Unlike a basic stop order, which triggers a market order when the current price is at the `stop` level or a worse value, a stop-limit order creates a subsequent limit order to fill at the specified `limit` price.

Below, we modified the previous script to simulate and visualize a stop-limit order. This script version includes the bar's low as the `limit` price in the `strategy.entry()` command. It also includes additional drawings to show where the strategy activated the subsequent limit order and to visualize the limit price.

In this example chart, notice how the market price reached the limit level on the next bar after the stop-limit order was created, but the strategy did not enter a position because the limit order was not yet active. After price later reached the stop level, the strategy placed the limit order, and then the broker emulator filled it after the market price dropped back down to the limit level:

```

//@version=6
strategy("Stop-Limit order demo", overlay = true, margin_long = 100, margin_short = 100)

```



Figure 236: image

```
//@function Displays text passed to `txt` when called and shows the `price` level on the chart.
debugLabel(price, txt, lblColor, lineWidth = 1) =>
    label.new(
        bar_index, high, text = txt, color = lblColor, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    line.new(bar_index, close, bar_index, price, style = line.style_dotted, color = lblColor, width = lineWidth)
    line.new(
        bar_index, price, bar_index + 1, price, color = lblColor, extend = extend.right,
        style = line.style_dashed, width = lineWidth
    )

var float stopPrice = na
var float limitPrice = na

// Generate a long stop-limit order with a label and lines 100 bars before the last bar.
if last_bar_index - bar_index == 100
    stopPrice := close + syminfo.mintick * 800
    limitPrice := low
    debugLabel(limitPrice, "", color.gray)
    debugLabel(stopPrice, "Long Stop-Limit order created", color.teal)
    strategy.entry("Long", strategy.long, stop = stopPrice, limit = limitPrice)

// Draw a line and label when the strategy activates the limit order.
if high >= stopPrice
    debugLabel(limitPrice, "Limit order activated", color.green, 2)
    stopPrice := na
```

Order placement and cancellation

The `strategy.*` namespace features the following five functions that simulate the placement of orders, known as *order placement commands*: `strategy.entry()`, `strategy.order()`, `strategy.exit()`, `strategy.close()`, and `strategy.close_all()`.

Additionally, the namespace includes the following two functions that cancel pending orders, known as *order cancellation commands*: `strategy.cancel()` and `strategy.cancel_all()`.

The segments below explain these commands, their unique characteristics, and how to use them.

`strategy.entry()`

The `strategy.entry()` command generates *entry orders*. Its unique features help simplify opening and managing positions. This order placement command generates market orders by default. It can also create limit, stop, and stop-limit orders with the `limit` and `stop` parameters, as explained in the Order types section above.

Reversing positions One of the `strategy.entry()` command's unique features is its ability to *reverse* an open position automatically. By default, when an order from `strategy.entry()` executes while there is an open position in the opposite direction, the command automatically *adds* the position's size to the new order's size. The added quantity allows the order to close the current position and open a new position for the specified number of contracts/lots/shares/units in the new direction.

For instance, if a strategy has an open position of 15 shares in the `strategy.long` direction and calls `strategy.entry()` to place a new market order in the `strategy.short` direction, the size of the resulting transaction is the specified entry size **plus** 15 shares.

The example below demonstrates this behavior in action. When the `buyCondition` occurs once every 100 bars, the script calls `strategy.entry()` with `qty = 15` to open a long position of 15 shares. Otherwise, when the `sellCondition` occurs on every 50th bar, the script calls `strategy.entry()` with `qty = 5` to enter a new short position of five shares. The script also highlights the chart's background on the bars where the `buyCondition` and `sellCondition` occurs:



Figure 237: image

```
//@version=6
strategy("Reversing positions demo", overlay = true)

//@variable Is `true` on every 100th bar, `false` otherwise.
bool buyCondition = bar_index % 100 == 0
//@variable Is `true` on every 50th bar, `false` otherwise.
bool sellCondition = bar_index % 50 == 0

if buyCondition
    // Place a "buy" market order to close the short position and enter a long position of 15 shares.
    strategy.entry("buy", strategy.long, qty = 15)
else if sellCondition
    // Place a "sell" market order to close the long position and enter a short position of 5 shares.
    strategy.entry("sell", strategy.short, qty = 5)
```

```
// Highlight the background when the `buyCondition` or `sellCondition` occurs.
bgcolor(buyCondition ? color.new(color.blue, 90) : sellCondition ? color.new(color.red, 90) : na)
```

The trade markers on the chart show the *transaction size*, not the size of the resulting position. The markers above show that the transaction size was *20 shares* on each order fill rather than 15 for long orders and five for short orders. Since `strategy.entry()` reverses a position in the opposite direction by default, each call *adds* the open position's size (e.g., 15 for long entries) to the new order's size (e.g., 5 for short entries), resulting in a quantity of 20 shares on each entry after the first. Although each of these *transactions* is 20 shares in size, the resulting positions are 5 shares for each short entry and 15 for each long entry.

Note that:

- The `strategy.risk.allow_entry_in()` function *overrides* the allowed direction for the `strategy.entry()` command. When a script specifies a trade direction with this risk management command, orders from `strategy.entry()` in the opposite direction *close* the open position without allowing a reversal.

Pyramiding Another unique characteristic of the `strategy.entry()` command is its connection to a strategy's *pyramiding* property. Pyramiding specifies the maximum number of *successive entries* a strategy allows in the same direction. Users can set this property by including a `pyramiding` argument in the `strategy()` declaration statement or by adjusting the "Pyramiding" input in the script's "Settings/Properties" tab. The default value is 1, meaning the strategy can open new positions but cannot add to them using orders from `strategy.entry()` calls.

The following example uses `strategy.entry()` to place a market order when the `entryCondition` occurs on every 25th bar. The direction of the orders changes once every 100 bars, meaning every 100-bar cycle includes *four* `strategy.entry()` calls with the same direction. For visual reference of the conditions, the script highlights the chart's background based on the current direction each time the `entryCondition` occurs:

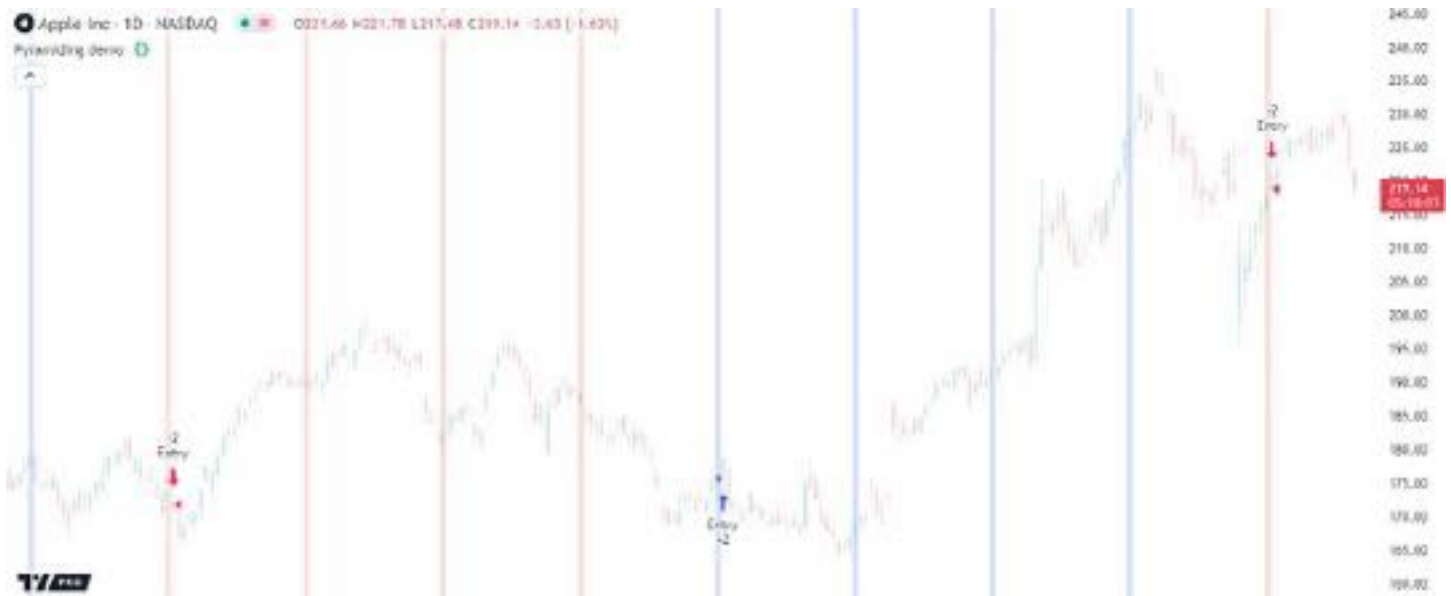


Figure 238: image

```
//@version=6
strategy("Pyramiding demo", overlay = true)

//@variable Represents the direction of the entry orders. A value of 1 means long, and -1 means short.
var int direction = 1
//@variable Is `true` once every 25 bars, `false` otherwise.
bool entryCondition = bar_index % 25 == 0

// Change the `direction` on every 100th bar.
if bar_index % 100 == 0
    direction *= -1
```

```
// Place a market order based on the current `direction` when the `entryCondition` occurs.
if entryCondition
    strategy.entry("Entry", direction == 1 ? strategy.long : strategy.short)
```

```
//@variable When the `entryCondition` occurs, is a blue color if the `direction` is 1 and a red color otherwise
color bgColor = entryCondition ? (direction == 1 ? color.new(color.blue, 80) : color.new(color.red, 80)) : na
// Highlight the chart's background using the `bgColor`.
bgcolor(bgColor, title = "Background highlight")
```

Notice that although the script calls `strategy.entry()` with the same direction four times within each 100-bar cycle, the strategy *does not* execute an order after every call. It cannot open more than one trade per position with `strategy.entry()` because it uses the default pyramiding value of 1.

Below, we modified the script by including `pyramiding = 4` in the `strategy()` declaration statement to allow up to four successive trades in the same direction. Now, an order fill occurs after every `strategy.entry()` call:



Figure 239: image

```
//@version=6
strategy("Pyramiding demo", overlay = true, pyramiding = 4)

//@variable Represents the direction of the entry orders. A value of 1 means long, and -1 means short.
var int direction = 1
//@variable Is `true` once every 25 bars, `false` otherwise.
bool entryCondition = bar_index % 25 == 0

// Change the `direction` on every 100th bar.
if bar_index % 100 == 0
    direction *= -1

// Place a market order based on the current `direction` when the `entryCondition` occurs.
if entryCondition
    strategy.entry("Entry", direction == 1 ? strategy.long : strategy.short)

//@variable When the `entryCondition` occurs, is a blue color if the `direction` is 1 and a red color otherwise
color bgColor = entryCondition ? (direction == 1 ? color.new(color.blue, 80) : color.new(color.red, 80)) : na
// Highlight the chart's background using the `bgColor`.
bgcolor(bgColor, title = "Background highlight")
```


strategy.order()

The `strategy.order()` command generates a *basic order*. Unlike other order placement commands, which can behave differently based on a strategy's properties and open trades, this command *ignores* most properties, such as pyramiding, and simply creates orders with the specified parameters. This command generates market orders by default. It can also create limit, stop, and stop-limit orders with the `limit` and `stop` parameters. Orders from `strategy.order()` can open new positions and modify or close existing ones. When a strategy executes an order from this command, the resulting market position is the *net sum* of the open position and the filled order quantity.

The following script uses `strategy.order()` calls to enter and exit positions. The strategy places a long market order for 15 units once every 100 bars. On every 25th bar that is not a multiple of 100, it places a short market order for five units. The script highlights the background to signify where the strategy places a “buy” or “sell” order:



Figure 240: image

```
//@version=6
strategy("`strategy.order()` demo", overlay = true)

//@variable Is `true` on every 100th bar, `false` otherwise.
bool buyCondition = bar_index % 100 == 0
//@variable Is `true` on every 25th bar, `false` otherwise.
bool sellCondition = bar_index % 25 == 0

if buyCondition
    // Place a "buy" market order to trade 15 units in the long direction.
    strategy.order("buy", strategy.long, qty = 15)
else if sellCondition
    // Place a "sell" market order to trade 5 units in the short direction.
    strategy.order("sell", strategy.short, qty = 5)

// Highlight the background when the `buyCondition` or `sellCondition` occurs.
bgcolor(buyCondition ? color.new(color.blue, 90) : sellCondition ? color.new(color.red, 90) : na)
```

This particular strategy never simulates a *short position*. Unlike the `strategy.entry()` command, `strategy.order()` *does not* automatically reverse open positions. After filling a “buy” order, the strategy has an open long position of 15 units. The three subsequent “sell” orders *reduce* the position by five units each, and $15 - 5 * 3 = 0$. In other words, the strategy opens a long position on every 100th bar and gradually reduces the size to 0 using three successive short orders. If we used `strategy.entry()` instead of the `strategy.order()` command in this example, the strategy would alternate between entering long

and short positions of 15 and five units, respectively.

`strategy.exit()`

The `strategy.exit()` command generates *exit orders*. It features several unique behaviors that link to open trades, helping to simplify closing market positions and creating multi-level exits with *take-profit*, *stop-loss*, and *trailing stop* orders.

Unlike other order placement commands, which can generate a *single order* per call, each call to `strategy.exit()` can produce *more than one* type of exit order, depending on its arguments. Additionally, a single call to this command can generate exit orders for *multiple entries*, depending on the specified `from_entry` value and the strategy's open trades.

Take-profit and stop-loss The most basic use of the `strategy.exit()` command is the placement of limit orders to trigger exits after earning enough money (take-profit), stop orders to trigger exits after losing too much money (stop-loss), or both (bracket).

Four parameters determine the prices of the command's take-profit and stop-loss orders:

- The **profit** and **loss** parameters accept *relative* values representing the number of *ticks* the market price must move away from the entry price to trigger an exit.
- The **limit** and **stop** parameters accept *absolute* values representing the specific *prices* that trigger an exit when the market price reaches them.

When a `strategy.exit()` call includes arguments for the relative *and* absolute parameters defining take-profit or stop-loss levels (**profit** and **limit** or **loss** and **stop**), it creates orders only at the levels expected to trigger exits *first*.

For instance, if the **profit** distance is 19 ticks and the **limit** level is 20 ticks past the entry price in the favorable direction, the `strategy.exit()` command places a take-profit order **profit** ticks past the entry price because the market price will move that distance before reaching the **limit** value. In contrast, if the **profit** distance is 20 ticks and the **limit** level is 19 ticks past the entry price in the favorable direction, the command places a take-profit order at the **limit** level because the price will reach that value first.

The following example creates exit bracket (take-profit and stop-loss) orders with the `strategy.exit()` command. When the `buyCondition` occurs, the script calls `strategy.entry()` to place a “buy” market order. It also calls `strategy.exit()` with **limit** and **stop** arguments to create a take-profit order at the `limitPrice` and a stop-loss order at the `stopPrice`. The script plots the `limitPrice` and `stopPrice` values on the chart to visualize the exit order prices:



Figure 241: image

```
//@version=6
strategy("Take-profit and stop-loss demo", overlay = true)

//@variable Is `true` on every 100th bar.
bool buyCondition = bar_index % 100 == 0

//@variable The current take-profit order price.
```

```

var float takeProfit = na
//@variable The current stop-loss order price.
var float stopLoss = na

if buyCondition
    // Update the `takeProfit` and `stopLoss` values.
    if strategy.opentrades == 0
        takeProfit := close * 1.01
        stopLoss    := close * 0.99
    // Place a long market order.
    strategy.entry("buy", strategy.long)
    // Place a take-profit order at the `takeProfit` price and a stop-loss order at the `stopLoss` price.
    strategy.exit("exit", "buy", limit = takeProfit, stop = stopLoss)

// Set `takeProfit` and `stopLoss` to `na` when the position closes.
if ta.change(strategy.closedtrades) > 0
    takeProfit := na
    stopLoss    := na

// Plot the `takeProfit` and `stopLoss` values.
plot(takeProfit, "TP", color.green, style = plot.style_circles)
plot(stopLoss, "SL", color.red, style = plot.style_circles)

```

Note that:

- We did not specify a `qty` or `qty_percent` argument in the `strategy.exit()` call, meaning it creates orders to exit 100% of the “buy” order’s size.
- The `strategy.exit()` command’s exit orders *do not* necessarily execute at the specified prices. Strategies can fill limit orders at *better* prices and stop orders at *worse* prices, depending on the range of values available to the broker emulator.

When a `strategy.exit()` call includes a `from_entry` argument, the resulting exit orders only apply to existing entry orders that have a matching ID. If the specified `from_entry` value does not match the ID of any entry in the current position, the command *does not* create any exit orders.

Below, we changed the `from_entry` argument of the `strategy.exit()` call in our previous script to “buy2”, which means it creates exit orders only for open trades with the “buy2” entry ID. This version does not place *any* exit orders because it does not create any entry orders with the “buy2” ID:

```

//@version=6
strategy("Invalid `from_entry` ID demo", overlay = true)

//@variable Is `true` on every 100th bar.
bool buyCondition = bar_index % 100 == 0

//@variable The current take-profit order price.
var float takeProfit = na
//@variable The current stop-loss order price.
var float stopLoss = na

if buyCondition
    // Update the `takeProfit` and `stopLoss` values before entering the trade.
    if strategy.opentrades == 0
        takeProfit := close * 1.01
        stopLoss    := close * 0.99
    // Place a long market order.
    strategy.entry("buy", strategy.long)
    // Attempt to place an exit bracket for "buy2" entries.
    // This call has no effect because the strategy does not create entry orders with the "buy2" ID.
    strategy.exit("exit", "buy2", limit = takeProfit, stop = stopLoss)

// Set `takeProfit` and `stopLoss` to `na` when the position closes.
if ta.change(strategy.closedtrades) > 0

```

```

takeProfit := na
stopLoss   := na

// Plot the `takeProfit` and `stopLoss` values.
plot(takeProfit, "TP", color.green, style = plot.style_circles)
plot(stopLoss, "SL", color.red, style = plot.style_circles)

```

Note that:

- When a `strategy.exit()` call *does not* include a `from_entry` argument, it creates exit orders for *all* the position's open trades, regardless of their entry IDs. See the Exits for multiple entries section below to learn more.

Partial and multi-level exits Strategies can use more than one call to `strategy.exit()` to create successive *partial* exit orders for the same entry ID, helping to simplify the formation of multi-level exit strategies. To use multiple `strategy.exit()` calls to exit from an open trade, include a `qty` or `qty_percent` argument in each call to specify how much of the traded quantity to close. If the sum of the exit order sizes exceeds the open position, the strategy automatically *reduces* their sizes to match the position.

Note that:

- When a `strategy.exit()` call includes *both* `qty` and `qty_percent` arguments, the command uses the `qty` value to size the order and ignores the `qty_percent` value.

This example demonstrates a simple strategy that creates two partial exit order brackets for an entry ID. When the `buyCondition` occurs, the script places a “buy” market order for two shares with `strategy.entry()`, and it creates “exit1” and “exit2” brackets using two calls to `strategy.exit()`. The first call uses a `qty` of 1, and the second uses a `qty` of 3:



Figure 242: image

```

//@version=6
strategy("Multi-level exit demo", "test", overlay = true)

//@variable Is `true` on every 100th bar.
bool buyCondition = bar_index % 100 == 0

//@variable The take-profit price for "exit1" orders.
var float takeProfit1 = na
//@variable The take-profit price for "exit2" orders.
var float takeProfit2 = na

```

```

//@variable The stop-loss price for "exit1" orders.
var float stopLoss1 = na
//@variable The stop-loss price for "exit2" orders.
var float stopLoss2 = na

if buyCondition
    // Update the `takeProfit*` and `stopLoss*` values before entering the trade.
    if strategy.opentrades == 0
        takeProfit1 := close * 1.01
        takeProfit2 := close * 1.02
        stopLoss1 := close * 0.99
        stopLoss2 := close * 0.98
    // Place a long market order with a `qty` of 2.
    strategy.entry("buy", strategy.long, qty = 2)
    // Place an "exit1" bracket with a `qty` of 1 at the `takeProfit1` and `stopLoss1` prices.
    strategy.exit("exit1", "buy", limit = takeProfit1, stop = stopLoss1, qty = 1)
    // Place an "exit2" bracket with a `qty` of 3 at the `takeProfit1` and `stopLoss1` prices.
    // The size of the resulting orders decreases to match the open position.
    strategy.exit("exit2", "buy", limit = takeProfit2, stop = stopLoss2, qty = 3)

// Set `takeProfit1` and `stopLoss1` to `na` when the price touches either value.
if high >= takeProfit1 or low <= stopLoss1
    takeProfit1 := na
    stopLoss1 := na
// Set `takeProfit2` and `stopLoss2` to `na` when the price touches either value.
if high >= takeProfit2 or low <= stopLoss2
    takeProfit2 := na
    stopLoss2 := na

// Plot the `takeProfit*` and `stopLoss*` values.
plot(takeProfit1, "TP1", color.green, style = plot.style_circles)
plot(takeProfit2, "TP2", color.green, style = plot.style_circles)
plot(stopLoss1, "SL1", color.red, style = plot.style_circles)
plot(stopLoss2, "SL2", color.red, style = plot.style_circles)

```

As we can see from the trade markers on the chart above, the strategy first executes the “exit1” take-profit or stop-loss order to reduce the open position by one share, leaving one remaining share in the position. However, we specified a size of *three shares* for the “exit2” order bracket, which exceeds the remaining position. Rather than using this specified quantity, the strategy automatically *reduces* the “exit2” orders to one share, allowing it to close the position successfully.

Note that:

- This strategy only fills **one** exit order from the “exit1” bracket, **not both**. When a strategy.exit() call generates more than one exit order type for an entry ID, the strategy fills the only the *first* triggered one and automatically cancels the others.
- The strategy reduced the “exit2” orders because all orders from the strategy.exit() calls automatically belong to the same strategy.oca.reduce group by default. Learn more about OCA groups below.

When creating multiple exit orders with *different* strategy.exit() calls, it’s crucial to note that the orders from each call *reserve* a portion of the open position. The orders from one strategy.exit() call *cannot* exit the portion of a position that a previous call already reserved.

For example, this script generates a “buy” entry order for 20 shares with a strategy.entry() call and “limit” and “stop” exit orders with two separate calls to strategy.exit() 100 bars before the last chart bar. We specified a quantity of 19 shares for the “limit” order and 20 for the “stop” order:

```

//@version=6
strategy("Reserved exit demo", "test", overlay = true)

//@variable The price of the "limit" exit order.
var float limitPrice = na
//@variable The price of the "stop" exit order.
var float stopPrice = na

```

```
//@variable Is `true` 100 bars before the last chart bar.
bool longCondition = last_bar_index - bar_index == 100

if longCondition
    // Update the `limitPrice` and `stopPrice`.
    limitPrice := close * 1.01
    stopPrice := close * 0.99
    // Place a long market order for 20 shares.
    strategy.entry("buy", strategy.long, 20)
    // Create a take-profit order for 19 shares at the `limitPrice`.
    strategy.exit("limit", limit = limitPrice, qty = 19)
    // Create a stop-loss order at the `stopPrice`. Although this call specifies a `qty` of 20, the previous
    // `strategy.exit()` call reserved 19, meaning this call creates an exit order for only 1 share.
    strategy.exit("stop", stop = stopPrice, qty = 20)

//@variable Is `true` when the strategy has an open position, `false` otherwise.
bool showPlot = strategy.opentrades == 1

// Plot the `limitPrice` and `stopPrice` when `showPlot` is `true`.
plot(showPlot ? limitPrice : na, "Limit (take-profit) price", color.green, 2, plot.style_linebr)
plot(showPlot ? stopPrice : na, "Stop (stop-loss) price", color.red, 2, plot.style_linebr)
```

Users unfamiliar with the `strategy.exit()` command's unique behaviors might expect this strategy to close the entire market position if it fills the “stop” order before the “limit” order. However, the trade markers in the chart below show that the “stop” order only reduces the position by **one share**. The `strategy.exit()` call for the “limit” order executes first in the code, reserving 19 shares of the open position for closure with that order. This reservation leaves only one share available for the “stop” order to close, regardless of when the strategy fills it:



Figure 243: image

Trailing stops One of the `strategy.exit()` command's key features is its ability to create *trailing stops*, i.e., stop-loss orders that trail behind the market price by a specified amount whenever it moves to a better value in the favorable direction (upward for long positions and downward for short positions).

This type of exit order has two components: an *activation level* and a *trail offset*. The activation level is the value the market price must cross to activate the trailing stop calculation, and the trail offset is the distance the activated stop follows behind the price as it reaches successively better values.

Three `strategy.exit()` parameters determine the activation level and trail offset of a trailing stop order:

- The `trail_price` parameter accepts an *absolute price value* for the trailing stop's activation level.
- The `trail_points` parameter is an alternative way to specify the activation level. Its value represents the *tick distance* from the entry price required to activate the trailing stop.
- The `trail_offset` parameter accepts a value representing the order's trail offset as a specified number of ticks.

To create and activate a trailing stop order, a `strategy.exit()` call must specify a `trail_offset` argument and either a `trail_price` or `trail_points` argument. If the call contains both `trail_price` and `trail_points` arguments, the command uses the level expected to activate the stop *first*. For instance, if the `trail_points` distance is 50 ticks and the `trail_price` value is 51 ticks past the entry price in the favorable direction, the `strategy.exit()` command uses the `trail_points` value to set the activation level because the market price will move that distance *before* reaching the `trail_price` level.

The example below demonstrates how a trailing stop order works in detail. The strategy places a "Long" market order with the `strategy.entry()` command 100 bars before the last chart bar, and it calls `strategy.exit()` with `trail_price` and `trail_offset` arguments on the following bar to create a trailing stop. The script uses lines, labels, and a plot to visualize the trailing stop's behavior.

The green line on the chart shows the level the market price must reach to activate the trailing stop order. After the price reaches this level from below, the script uses a blue plot to display the trailing stop's price. Each time the market price reaches a new high after activating the trailing stop, the stop's price *increases* to maintain a distance of `trailOffsetInput` ticks from the best value. The exit order *does not* change its price level when the price decreases or does not reach a new high. Eventually, the market price crosses below the trailing stop, triggering an exit:



Figure 244: image

```
//@version=6
strategy("Trailing stop order demo", overlay = true, margin_long = 100, margin_short = 100)

//@variable The distance from the entry price required to activate the trailing stop.
int activationOffsetInput = input.int(1000, "Activation level offset (in ticks)", 0)
//@variable The distance the stop follows behind the highest `high` after activation.
int trailOffsetInput = input.int(2000, "Trailing stop offset (in ticks)", 0)

//@variable Draws a label and an optional line at the specified `price`.
debugDrawings(float price, string txt, color drawingColor, bool drawLine = false) =>
    // Draw a label showing the `txt` at the `price` on the current bar.
    label.new(
```



```

        bar_index, price, text = txt, color = drawingColor, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    // Draw a horizontal line at the `price` starting from the current bar when `drawLine` is `true`.
    line.new(
        bar_index, price, bar_index + 1, price, color = drawingColor, extend = extend.right,
        style = line.style_dashed
    )

    //@variable The level required to activate the trailing stop.
    var float activationLevel = na
    //@variable The price of the trailing stop.
    var float trailingStop = na
    //@variable The value that the trailing stop would have if it was currently active.
    float theoreticalStopPrice = high - trailOffsetInput * syminfo.mintick

    // Place a long market order 100 bars before the last historical bar.
    if last_bar_index - bar_index == 100
        strategy.entry("Long", strategy.long)

    // Create and visualize the exit order on the next bar.
    if last_bar_index - bar_index == 99
        // Update the `activationLevel`.
        activationLevel := open + syminfo.mintick * activationOffsetInput
        // Create the trailing stop order that activates at the `activationLevel` and trails behind the `high` by
        // `trailOffsetInput` ticks.
        strategy.exit(
            "Trailing Stop", from_entry = "Long", trail_price = activationLevel,
            trail_offset = trailOffsetInput
        )
        // Create drawings to signify the activation level.
        debugDrawings(activationLevel, "Trailing Stop Activation Level", color.green, true)

    // Visualize the trailing stop's levels while the position is open.
    if strategy.opentrades == 1
        // Create drawings when the `high` is above the `activationLevel` for the first time to show when the
        // stop activates.
        if na(trailingStop) and high >= activationLevel
            debugDrawings(activationLevel, "Activation level crossed", color.green)
            trailingStop := theoreticalStopPrice
            debugDrawings(trailingStop, "Trailing Stop Activated", color.blue)
        // Otherwise, update the `trailingStop` value when the `theoreticalStopPrice` reaches a new high.
        else if theoreticalStopPrice > trailingStop
            trailingStop := theoreticalStopPrice

    // Plot the `trailingStop` value to visualize the trailing price movement.
    plot(trailingStop, "Trailing Stop")

```

Exits for multiple entries A single call to the `strategy.exit()` command can generate exit orders for *more than one* entry in an open position, depending on the call's `from_entry` value.

If an open position consists of two or more entries with the same ID, a single call to `strategy.exit()` with that ID as the `from_entry` argument places exit orders for each corresponding entry created before or on the bar where the call occurs.

For example, this script periodically calls `strategy.entry()` on two consecutive bars to enter and add to a long position. Both calls use “buy” as the `id` argument. After creating the second entry, the script calls `strategy.exit()` once with “buy” as its `from_entry` argument to generate separate exit orders for each entry with that ID. When the market price reaches the `takeProfit` or `stopLoss` value, the broker emulator fills *two* exit orders and closes the position:

```

//@version=6
strategy("Exits for entries with the same ID demo", overlay = true, pyramiding = 2)

```



Figure 245: image

```
//@variable Take-profit price for exit commands.
var float takeProfit = na
//@variable Stop-loss price for exit commands.
var float stopLoss = na

//@variable Is `true` on two consecutive bars in 100-bar cycles.
bool buyCondition = math.min(bar_index % 100, math.max(bar_index - 1, 0) % 100) == 0

if buyCondition
    // Place a "buy" market order to enter a trade.
    strategy.entry("buy", strategy.long)
    // Calculate exits on the second order.
    if strategy.opentrades == 1
        // Update the `takeProfit` and `stopLoss`.
        takeProfit := close * 1.01
        stopLoss := close * 0.99
        // Place exit orders for both "buy" entries.
        strategy.exit("exit", "buy", limit = takeProfit, stop = stopLoss)

// Set `takeProfit` and `stopLoss` to `na` when both trades close.
if ta.change(strategy.closedtrades) == 2
    takeProfit := na
    stopLoss := na

// Plot the `takeProfit` and `stopLoss` values.
plot(takeProfit, "TP", color.green, style = plot.style_circles)
plot(stopLoss, "SL", color.red, style = plot.style_circles)
```

A single `strategy.exit()` call can also generate exit orders for *all* entries in an open position, irrespective of entry ID, when it does not include a `from_entry` argument.

Here, we changed the `strategy.entry()` instance in the above script to create an entry order with a distinct ID on each call, and we removed the `from_entry` argument from the `strategy.exit()` call. Since this version does not specify which entries the exit orders apply to, the `strategy.exit()` call creates orders for *every* entry in the position:

```
//@version=6
strategy("Exits for entries with different IDs demo", overlay = true, pyramiding = 2)

//@variable Take-profit price for exit commands.
var float takeProfit = na
```



Figure 246: image

```
//@variable Stop-loss price for exit commands.
var float stopLoss = na

//@variable Is `true` on two consecutive bars in 100-bar cycles.
bool buyCondition = math.min(bar_index % 100, math.max(bar_index - 1, 0) % 100) == 0

if buyCondition
    // Place a long market order with a unique ID.
    strategy.entry("buy" + str.tostring(strategy.opentrades + strategy.closedtrades), strategy.long)
    // Calculate exits on the second order.
    if strategy.opentrades == 1
        // Update the `takeProfit` and `stopLoss`.
        takeProfit := close * 1.01
        stopLoss := close * 0.99
        // Place exit orders for ALL entries in the position, irrespective of ID.
        strategy.exit("exit", limit = takeProfit, stop = stopLoss)

// Set `takeProfit` and `stopLoss` to `na` when both trades close.
if ta.change(strategy.closedtrades) == 2
    takeProfit := na
    stopLoss := na

// Plot the `takeProfit` and `stopLoss` values.
plot(takeProfit, "TP", color.green, style = plot.style_circles)
plot(stopLoss, "SL", color.red, style = plot.style_circles)
```

It's crucial to note that a call to `strategy.exit()` without a `from_entry` argument *persists* and creates exit orders for all open trades in a position, regardless of *when* the entries occur. This behavior can affect strategies that manage positions with multiple entries or exits. When a strategy has an open position and calls `strategy.exit()` on any bar without specifying a `from_entry` ID, it generates exit orders for each entry created *before* or on that bar, and it continues to generate exit orders for subsequent entries *after* that bar until the position closes.

Let's explore this behavior and how it works. The script below creates a long entry order with `strategy.entry()` on each bar within a user-specified time range, and it calls `strategy.exit()` without a `from_entry` argument on *one bar* within that range to generate exit orders for *every* entry in the open position. The exit command uses a `loss` value of 0, which means an exit order fills each time the market price is not above an entry order's price.

The script prompts users to select three points before it starts its calculations. The first point specifies when order creation begins, the second determines when the single `strategy.exit()` call occurs, and the third specifies when order creation stops:

```
//@version=6
```



Figure 247: image

```
strategy("Exit persist demo", overlay = true, margin_long = 100, margin_short = 100, pyramiding = 100)
```

```
//@variable The time when order creation starts.
int entryStartTime = input.time(0, "Start time for entries", confirm = true)
//@variable The time when the `strategy.exit()` call occurs.
int exitCallTime = input.time(0, "Exit call time", confirm = true)
//@variable The time when order creation stops.
int entryEndTime = input.time(0, "End time for entries", confirm = true)

// Raise a runtime error if incorrect timestamps are chosen.
if exitCallTime <= entryStartTime or entryEndTime <= exitCallTime or entryEndTime <= entryStartTime
    runtime.error("The input timestamps must follow this condition: entryStartTime < exitCallTime < entryEndTime")

// Create variables to track entry and exit conditions.
bool entriesStart = time == entryStartTime
bool callExit     = time == exitCallTime
bool entriesEnd   = time == entryEndTime
bool callEntry    = time >= entryStartTime and time < entryEndTime

// Place a long entry order when `callEntry` is `true`.
if callEntry
    strategy.entry("Entry", strategy.long)

// Call `strategy.exit()` when `callExit` is `true`, which occurs only once.
// This single call persists and creates exit orders for EVERY entry in the position because it does not
// specify a `from_entry` ID.
if callExit
    strategy.exit("Exit", loss = 0)

// Draw labels to signify when entries start, when the `strategy.exit()` call occurs, and when order placement
switch
    entriesStart => label.new(
        bar_index, high, "Start placing entry orders.", color = color.green, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    callExit => label.new(
        bar_index, high, "Call `strategy.exit()` once.", color = color.blue, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
```

```

    )
    entriesEnd => label.new(
        bar_index, high, "Stop placing orders.", color = color.red, textcolor = color.white,
        style = label.style_label_lower_left, size = size.large
    )

// Create a line and label to visualize the lowest entry price, i.e., the price required to close the position
var line lowestLine = line.new(
    entryStartTime + 1000, na, entryEndTime, na, xloc.bar_time, extend.right, color.orange, width = 2
)
var lowestLabel = label.new(
    entryStartTime + 1000, na, "Lowest entry price", color = color.orange,
    style = label.style_label_upper_right, xloc = xloc.bar_time
)

// Update the price values of the `lowestLine` and `lowestLabel` after each new entry.
if callEntry[1]
    var float lowestPrice = strategy.opentrades.entry_price(0)
    float entryPrice = strategy.opentrades.entry_price(strategy.opentrades - 1)
    if not na(entryPrice)
        lowestPrice := math.min(lowestPrice, entryPrice)
        lowestLine.set_y1(lowestPrice)
        lowestLine.set_y2(lowestPrice)
        lowestLabel.set_y(lowestPrice)

// Highlight the background when `entriesStart`, `callExit`, and `entriesEnd` occurs.
bgcolor(entriesStart ? color.new(color.green, 80) : na, title = "Entries start highlight")
bgcolor(callExit ? color.new(color.blue, 80) : na, title = "Exit call highlight")
bgcolor(entriesEnd ? color.new(color.red, 80) : na, title = "Entries end highlight")

```

Note that:

- We included `pyramiding = 100` in the `strategy()` declaration statement, which allows the position to have up to 100 open entries from `strategy.entry()`.
- The script uses labels and `bgcolor()` to signify when order placement starts and stops and when the `strategy.exit()` call occurs.
- The script draws a line and a label at the lowest entry price to show the value the market price must reach to close the position.

We can observe the unique `strategy.exit()` behavior in this example by comparing the code itself with the script's chart outputs. The script calls `strategy.exit()` *one time*, only on the bar with the blue label. However, this single call placed exit orders for every entry **before** or on that bar and continued placing exit orders for all entries **after** that bar. This behavior occurs because `strategy.exit()` has no way to determine when to stop placing orders if it does not link to entries with a specific ID. In this case, the command only ceases to create new exit orders after the position fully closes.

The above script would exhibit different behavior if we included a `from_entry` argument in the `strategy.exit()` call. When a call to this command specifies a `from_entry` ID, it only applies to entries with that ID which the strategy created *before* or *on* the bar of the call. The command does not place exit orders for subsequent entries created *after* that bar in that case, even ones with the same ID.

Here, we added `from_entry = "Entry"` to our script's `strategy.exit()` call, meaning it only produces exit orders for entries with the "Entry" ID. Only 17 exits occur this time, each corresponding to an entry order created before or on the bar with the blue label. The call does not affect any entries that the strategy creates *after* that bar:

```

//@version=6
strategy("Exit persist demo", overlay = true, margin_long = 100, margin_short = 100, pyramiding = 100)

//@variable The time when order creation starts.
int entryStartTime = input.time(0, "Start time for entries", confirm = true)
//@variable The time when the `strategy.exit()` call occurs.
int exitCallTime = input.time(0, "Exit call time", confirm = true)
//@variable The time when order creation stops.
int entryEndTime = input.time(0, "End time for entries", confirm = true)

```



Figure 248: image

```
// Raise a runtime error if incorrect timestamps are chosen.
if exitCallTime <= entryStartTime or entryEndTime <= exitCallTime or entryEndTime <= entryStartTime
    runtime.error("The input timestamps must follow this condition: entryStartTime < exitCallTime < entryEndTime")

// Create variables to track entry and exit conditions.
bool entriesStart = time == entryStartTime
bool callExit     = time == exitCallTime
bool entriesEnd   = time == entryEndTime
bool callEntry    = time >= entryStartTime and time < entryEndTime

// Place a long entry order when `callEntry` is `true`.
if callEntry
    strategy.entry("Entry", strategy.long)

// Call `strategy.exit()` when `callExit` is `true`, which occurs only once.
// This single call only places exit orders for all entries with the "Entry" ID created before or on the bar w
// `callExit` occurs. It DOES NOT affect any subsequent entries created after that bar.
if callExit
    strategy.exit("Exit", from_entry = "Entry", loss = 0)

// Draw labels to signify when entries start, when the `strategy.exit()` call occurs, and when order placement
switch
    entriesStart => label.new(
        bar_index, high, "Start placing entry orders.", color = color.green, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    callExit => label.new(
        bar_index, high, "Call `strategy.exit()` once.", color = color.blue, textcolor = color.white,
        style = label.style_label_lower_right, size = size.large
    )
    entriesEnd => label.new(
        bar_index, high, "Stop placing orders.", color = color.red, textcolor = color.white,
        style = label.style_label_lower_left, size = size.large
    )

// Create a line and label to visualize the lowest entry price, i.e., the price required to close the position
var line lowestLine = line.new(
```

```

    entryStartTime + 1000, na, entryEndTime, na, xloc.bar_time, extend.right, color.orange, width = 2
)
var lowestLabel = label.new(
    entryStartTime + 1000, na, "Lowest entry price", color = color.orange,
    style = label.style_label_upper_right, xloc = xloc.bar_time
)

// Update the price values of the `lowestLine` and `lowestLabel` after each new entry.
if callEntry[1]
    var float lowestPrice = strategy.opentrades.entry_price(0)
    float entryPrice = strategy.opentrades.entry_price(strategy.opentrades - 1)
    if not na(entryPrice)
        lowestPrice := math.min(lowestPrice, entryPrice)
        lowestLine.set_y1(lowestPrice)
        lowestLine.set_y2(lowestPrice)
        lowestLabel.set_y(lowestPrice)

// Highlight the background when `entriesStart`, `callExit`, and `entriesEnd` occurs.
bgcolor(entriesStart ? color.new(color.green, 80) : na, title = "Entries start highlight")
bgcolor(callExit ? color.new(color.blue, 80) : na, title = "Exit call highlight")
bgcolor(entriesEnd ? color.new(color.red, 80) : na, title = "Entries end highlight")

```

strategy.close() and strategy.close_all()

The `strategy.close()` and `strategy.close_all()` commands generate orders to exit from an open position. Unlike `strategy.exit()`, which creates *price-based* exit orders (e.g., stop-loss), these commands generate market orders that the broker emulator fills on the next available tick, irrespective of the price.

The example below demonstrates a simple strategy that places a “buy” entry order with `strategy.entry()` once every 50 bars and a market order to close the long position with `strategy.close()` 25 bars afterward:



Figure 249: image

```

//@version=6
strategy("Close demo", "test", overlay = true)

//@variable Is `true` on every 50th bar.

```



```

buyCond = bar_index % 50 == 0
//@variable Is `true` on every 25th bar except for those that are divisible by 50.
sellCond = bar_index % 25 == 0 and not buyCond

if buyCond
    strategy.entry("buy", strategy.long)
if sellCond
    strategy.close("buy")

bgcolor(buyCond ? color.new(color.blue, 90) : na)
bgcolor(sellCond ? color.new(color.red, 90) : na)

```

Notice that the `strategy.close()` call in this script uses “buy” as its required `id` argument. Unlike `strategy.exit()`, this command’s `id` parameter specifies the *entry ID* of an open trade. It **does not** represent the ID of the resulting exit order. If a market position consists of multiple open trades with the same entry ID, a single `strategy.close()` call with that ID as its `id` argument generates a single market order to exit from all of them.

The following script creates a “buy” order with `strategy.entry()` once every 25 bars, and it calls `strategy.close()` with “buy” as its `id` argument to close all open trades with that entry ID once every 100 bars. The market order from `strategy.close()` closes the entire position in this case because every open trade has the same “buy” entry ID:



Figure 250: image

```

//@version=6
strategy("Multiple close demo", "test", overlay = true, pyramiding = 3)

//@variable Is `true` on every 100th bar.
sellCond = bar_index % 100 == 0
//@variable Is `true` on every 25th bar except for those that are divisible by 100.
buyCond = bar_index % 25 == 0 and not sellCond

if buyCond
    strategy.entry("buy", strategy.long)
if sellCond
    strategy.close("buy")

bgcolor(buyCond ? color.new(color.blue, 90) : na)

```

```
bgcolor(sellCond ? color.new(color.red, 90) : na)
```

Note that:

- We included `pyramiding = 3` in the `strategy()` declaration statement, allowing the script to generate up to three entries per position with `strategy.entry()` calls.

The `strategy.close_all()` command generates a market order to exit from the open position that *does not* link to any specific entry ID. This command is helpful when a strategy needs to exit as soon as possible from a position consisting of multiple open trades with different entry IDs.

The script below places “A”, “B”, and “C” entry orders sequentially based on the number of open trades, and then it calls `strategy.close_all()` to create a single order that closes the entire position on the next bar:

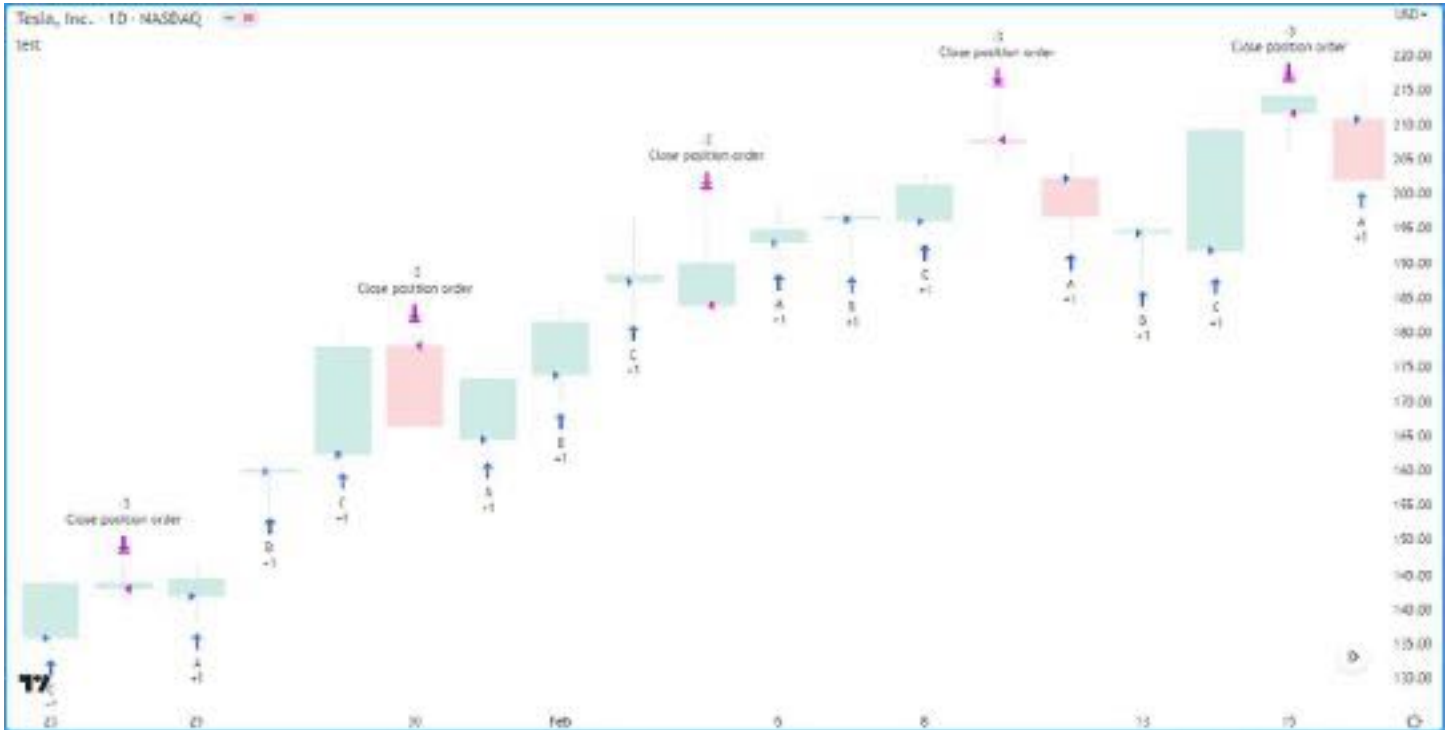


Figure 251: image

```
//@version=6
strategy("Close multiple ID demo", "test", overlay = true, pyramiding = 3)

switch strategy.opentrades
  0 => strategy.entry("A", strategy.long)
  1 => strategy.entry("B", strategy.long)
  2 => strategy.entry("C", strategy.long)
  3 => strategy.close_all()
```

`strategy.cancel()` and `strategy.cancel_all()`

The `strategy.cancel()` and `strategy.cancel_all()` commands allow strategies to cancel *unfilled* orders before the broker emulator processes them. These order cancellation commands are most helpful when working with *price-based orders*, including all orders from `strategy.exit()` calls and the orders from `strategy.entry()` and `strategy.order()` calls that use `limit` or `stop` arguments.

The `strategy.cancel()` command has a required `id` parameter, which specifies the ID of the entry or exit orders to cancel. The `strategy.cancel_all()` command does not have such a parameter because it cancels *all* unfilled orders, regardless of ID.

The following strategy places a “buy” limit order 500 ticks below the closing price 100 bars before the last chart bar with `strategy.entry()`, and it cancels the order on the next bar with `strategy.cancel()`. The script highlights the chart’s background to signify when it places and cancels the “buy” order, and it draws a horizontal line at the order’s price. As we see below, our example chart shows no entry marker when the market price crosses the horizontal line because the strategy already cancels the order (when the chart’s background is orange) before it reaches that level:



Figure 252: image

```
//@version=6
strategy("Cancel demo", "test", overlay = true)

//@variable Draws a horizontal line at the `limit` price of the "buy" order.
var line limitLine = na

//@variable Is `color.green` when the strategy places the "buy" order, `color.orange` when it cancels the order.
color bgColor = na

if last_bar_index - bar_index == 100
    float limitPrice = close - syminfo.mintick * 500
    strategy.entry("buy", strategy.long, limit = limitPrice)
    limitLine := line.new(bar_index, limitPrice, bar_index + 1, limitPrice, extend = extend.right)
    bgColor := color.new(color.green, 50)

if last_bar_index - bar_index == 99
    strategy.cancel("buy")
    bgColor := color.new(color.orange, 50)

bgcolor(bgColor)
```

The `strategy.cancel()` command affects *all* unfilled orders with a specified ID. It does nothing if the specified `id` represents the ID of an order that does not exist. When there is more than one unfilled order with the specified ID, the command cancels *all* of them at once.

Below, we've modified the previous script to place a “buy” limit order on three consecutive bars, starting 100 bars before the last chart bar. After placing all three orders, the strategy cancels them using `strategy.cancel()` with “buy” as the `id` argument, resulting in nothing happening when the market price reaches any of the order prices (horizontal lines):

```
//@version=6
strategy("Multiple cancel demo", "test", overlay = true, pyramiding = 3)

//@variable Draws a horizontal line at the `limit` price of the "buy" order.
var line limitLine = na
```



Figure 253: image

```
//@variable Is `color.green` when the strategy places the "buy" order, `color.orange` when it cancels the order
color bgColor = na
```

```
if last_bar_index - bar_index <= 100 and last_bar_index - bar_index >= 98
    float limitPrice = close - syminfo.mintick * 500
    strategy.entry("buy", strategy.long, limit = limitPrice)
    limitLine := line.new(bar_index, limitPrice, bar_index + 1, limitPrice, extend = extend.right)
    bgColor := color.new(color.green, 50)
```

```
if last_bar_index - bar_index == 97
    strategy.cancel("buy")
    bgColor := color.new(color.orange, 50)
```

```
bgcolor(bgColor)
```

Note that:

- We included `pyramiding = 3` in the `strategy()` declaration statement, allowing three successive entries from `strategy.entry()` per position. The script would also achieve the same result without this setting if it called `strategy.order()` instead because *pyramiding does not* affect orders from that command.

The `strategy.cancel()` and `strategy.cancel_all()` commands can cancel orders of any type, including market orders. However, it is important to note that either command can cancel a market order only if its call occurs on the *same* script execution as the order placement command. If the call happens after that point, it has *no effect* because the broker emulator fills market orders on the *next available tick*.

This example places a “buy” market order 100 bars before the last chart bar with `strategy.entry()`, then it attempts to cancel the order on the next bar with `strategy.cancel_all()`. The cancellation command *does not* affect the “buy” order because the broker emulator fills the order on the next bar’s *opening tick*, which occurs *before* the script evaluates the `strategy.cancel_all()` call:

```
//@version=6
strategy("Cancel market demo", "test", overlay = true)
```

```
//@variable Is `color.green` when the strategy places the "buy" order, `color.orange` when it tries to cancel
color bgColor = na
```



Figure 254: image

```

if last_bar_index - bar_index == 100
    strategy.entry("buy", strategy.long)
    bgColor := color.new(color.green, 50)

if last_bar_index - bar_index == 99
    strategy.cancel_all()
    bgColor := color.new(color.orange, 50)

bgcolor(bgColor)

```

Position sizing

Pine Script™ strategies feature two ways to control the sizes of the orders that open and manage positions:

- Set a default *fixed* quantity type and value for the orders. Programmers can specify defaults for these properties by including `default_qty_type` and `default_qty_value` arguments in the `strategy()` declaration statement. Script users can adjust these values with the “Order size” inputs in the “Settings/Properties” tab.
- Include a *non-naqty* argument in the `strategy.entry()` or `strategy.order()` call. When a call to either of these commands specifies a non-na qty value, that call ignores the strategy’s default quantity type and value and places an order for qty contracts/shares/lots/units instead.

The following example uses `strategy.entry()` calls with different `qty` values for long and short trades. When the current bar’s low equals the `lowest` value, the script places a “Buy” order to enter a long position of `longAmount` units. Otherwise, when the high equals the `highest` value, it places a “Sell” order to enter a short position of `shortAmount` units:

```

//@version=6
strategy("Buy low, sell high", overlay = true, default_qty_type = strategy.cash, default_qty_value = 5000)

int length = input.int(20, "Length", 1)
float longAmount = input.float(4.0, "Long Amount", 0.0)
float shortAmount = input.float(2.0, "Short Amount", 0.0)

float highest = ta.highest(length)
float lowest = ta.lowest(length)

switch
    low == lowest => strategy.entry("Buy", strategy.long, longAmount)
    high == highest => strategy.entry("Sell", strategy.short, shortAmount)

```

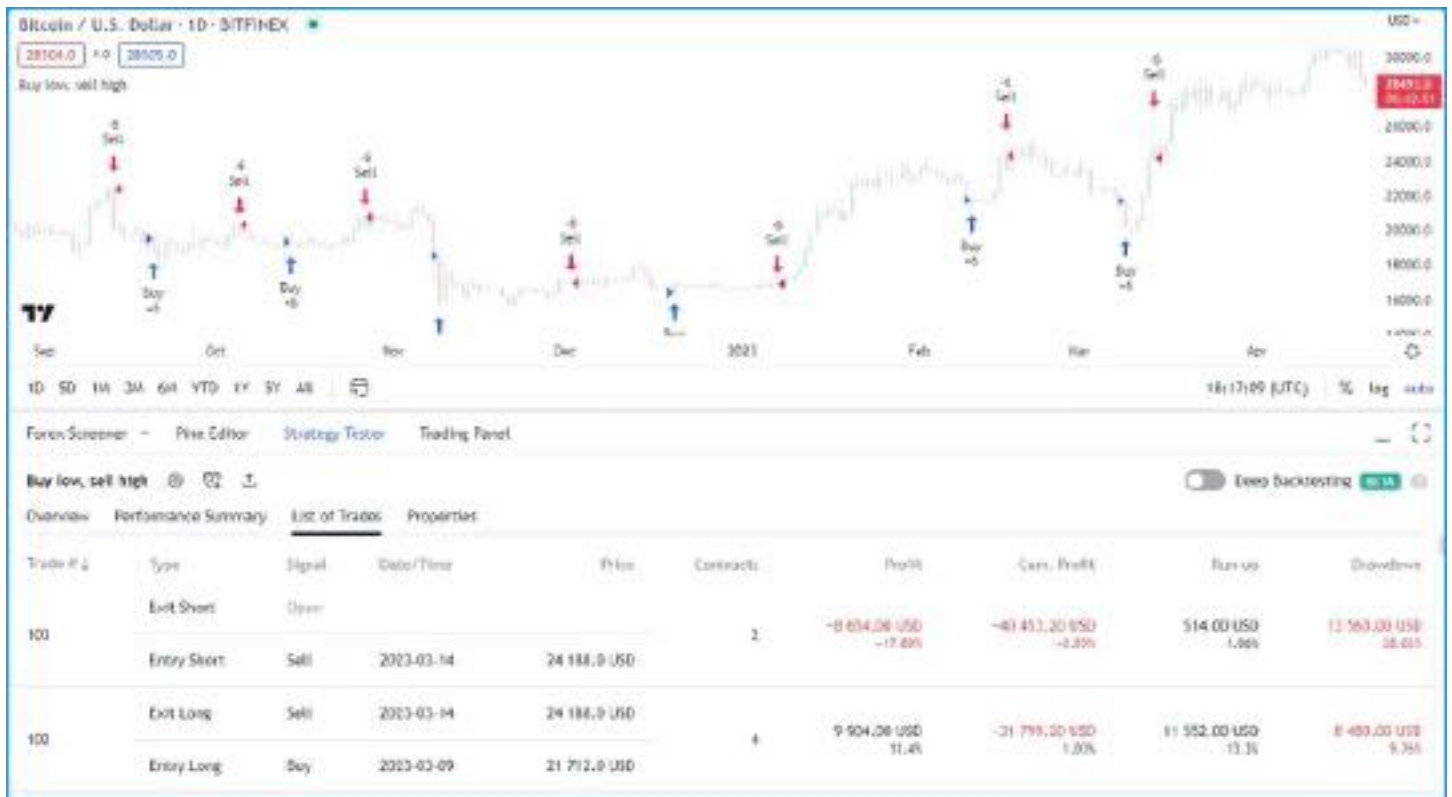


Figure 255: image

Notice that although we've included `default_qty_type` and `default_qty_value` arguments in the `strategy()` declaration statement, the strategy *does not* use this default setting to size its orders because the specified `qty` in the entry commands takes precedence. If we want to use the default size, we must *remove* the `qty` arguments from the `strategy.entry()` calls or set their values to `na`.

Here, we edited the previous script by including ternary expressions for the `qty` arguments in both `strategy.entry()` calls that replace input values of 0 with `na`. If the specified `longAmount` or `shortAmount` is 0, which is what we set as the new default, the corresponding entry orders use the strategy's default order size instead, as we see below:

```
//@version=6
strategy("Buy low, sell high", overlay = true, default_qty_type = strategy.cash, default_qty_value = 5000)

int length = input.int(20, "Length", 1)
float longAmount = input.float(0.0, "Long Amount", 0.0)
float shortAmount = input.float(0.0, "Short Amount", 0.0)

float highest = ta.highest(length)
float lowest = ta.lowest(length)

switch
  low == lowest => strategy.entry("Buy", strategy.long, longAmount == 0.0 ? na : longAmount)
  high == highest => strategy.entry("Sell", strategy.short, shortAmount == 0.0 ? na : shortAmount)
```

Closing a market position

By default, strategies close a market position using the *First In, First Out (FIFO)* method, which means that any exit order closes or reduces the position starting with the *first* open trade, even if the exit command specifies the entry ID of a *different* open trade. To override this default behavior, include `close_entries_rule = "ANY"` in the `strategy()` declaration statement.

The following example places “Buy1” and “Buy2” entry orders sequentially, starting 100 bars before the latest chart bar. When the position size is 0, it calls `strategy.entry()` to place the “Buy1” order for five units. After the strategy's position size matches the size of that order, it uses `strategy.entry()` to place the “Buy2” order for ten units. The strategy then creates



Figure 256: image

“bracket” exit orders for both entries using a single `strategy.exit()` call without a `from_entry` argument. For visual reference, the script plots the `strategy.position_size` value in a separate pane:

```
//@version=6
strategy("Exit Demo", pyramiding = 2)

float positionSize = strategy.position_size

if positionSize == 0 and last_bar_index - bar_index <= 100
    strategy.entry("Buy1", strategy.long, 5)
else if positionSize == 5
    strategy.entry("Buy2", strategy.long, 10)
else if positionSize == 15
    strategy.exit("bracket", loss = 10, profit = 10)

plot(positionSize == 0 ? na : positionSize, "Position Size", color.lime, 4, plot.style_histogram)
```

Note that:

- We included `pyramiding = 2` in the `strategy()` declaration statement, allowing two successive entries from `strategy.entry()` per position.

Each time the market price triggers an exit order, the above script exits from the open position, starting with the *oldest* open trade. This FIFO behavior applies even if we explicitly specify an exit from “Buy2” before “Buy1” in the code.

The script version below calls `strategy.close()` with “Buy2” as its `id` argument, and it includes “Buy1” as the `from_entry` argument in the `strategy.exit()` call. The market order from `strategy.close()` executes on the next available tick, meaning the broker emulator fills it *before* the take-profit and stop-loss orders from `strategy.exit()`:

```
//@version=6
strategy("Exit Demo", pyramiding = 2)

float positionSize = strategy.position_size
```

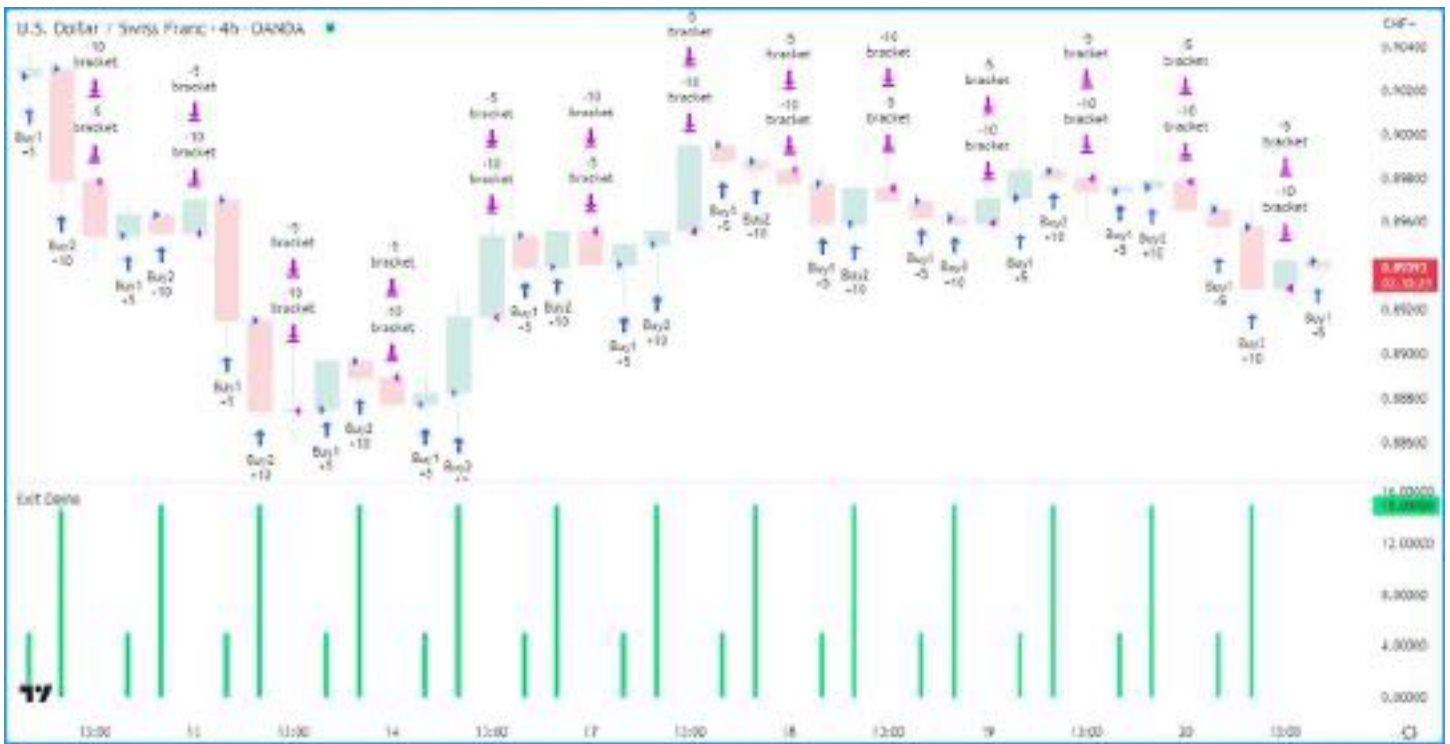



Figure 257: image

```

if positionSize == 0 and last_bar_index - bar_index <= 100
    strategy.entry("Buy1", strategy.long, 5)
else if positionSize == 5
    strategy.entry("Buy2", strategy.long, 10)
else if positionSize == 15
    strategy.close("Buy2")
    strategy.exit("bracket", "Buy1", loss = 10, profit = 10)

```

```

plot(positionSize == 0 ? na : positionSize, "Position Size", color.lime, 4, plot.style_histogram)

```

The market order from the script's `strategy.close()` call is for 10 units because it links to the open trade with the “Buy2” entry ID. A user might expect this strategy to close that trade completely when the order executes. However, the “List of Trades” tab shows that five units of the order go toward closing the “Buy1” trade *first* because it is the oldest, and the remaining five units close *half* of the “Buy2” trade. After that, the “bracket” orders from the `strategy.exit()` call close the rest of the position:

Note that:

- If we included `close_entries_rule = "ANY"` in the `strategy()` declaration statement, the market order from `strategy.close()` would close the open trade with the “Buy2” entry ID *first*, and then the “bracket” orders from `strategy.exit()` would close the trade with the “Buy1” entry ID.

OCA groups

One-Cancels-All (OCA) groups allow a strategy to fully or partially *cancel* specific orders when the broker emulator executes another order from the same group. To assign an order to an OCA group, include an `oca_name` argument in the call to the order placement command. The `strategy.entry()` and `strategy.order()` commands also allow programmers to specify an *OCA type*, which defines whether a strategy cancels, reduces, or does not modify the order after executing other orders.

`strategy.oca.cancel`

When an order placement command uses `strategy.oca.cancel` as its `oca_type` argument, the strategy completely *cancels* the resulting order if another order from the same OCA group executes first.

To demonstrate how this OCA type impacts a strategy's orders, consider the following script, which places orders when the `ma1` value crosses the `ma2` value. If the `strategy.position_size` is 0 when the cross occurs, the strategy places two stop orders

Exit Demo ⊞ 🔍 📄 🔧 Deep Backtesting 95%									
Overview	Performance Summary		List of Trades	Properties					
Trade # ↓	Type	Signal	Date/Time	Price	Contracts	Profit	Com. Profit	Run-up	Drawdown
100	Exit Long	Open							
	Entry Long	Buy1	2023-04-20 17:00	0.89423 CHF	5	-0.00 CHF -0.00%	-0.16 CHF 0%	0.00 CHF 0.01%	0.00 CHF 0.00%
99	Exit Long	Bracket	2023-04-20 11:00	0.89300 CHF	5	-0.01 CHF -0.31%	-0.15 CHF 0%	0.01 CHF 0.11%	0.31 CHF 0.34%
	Entry Long	Buy2	2023-04-20 09:00	0.89582 CHF					
98	Exit Long	Close entry(s) order Buy2	2023-04-20 11:00	0.89300 CHF	5	-0.01 CHF -0.31%	-0.16 CHF 0%	0.01 CHF 0.11%	0.32 CHF 0.35%
	Entry Long	Buy2	2023-04-20 09:00	0.89582 CHF					
97	Exit Long	Close entry(s) order Buy2	2023-04-20 11:00	0.89300 CHF	5	-0.01 CHF -0.4%	-0.15 CHF 0%	0.00 CHF 0.06%	0.31 CHF 0.48%
	Entry Long	Buy1	2023-04-20 05:00	0.89560 CHF					

Figure 258: image

with `strategy.order()` calls. The first is a long order at the bar's high, and the second is a short order at the bar's low. If the strategy already has an open position during the cross, it calls `strategy.close_all()` to close the position with a market order:

```
//@version=6
strategy("OCA Cancel Demo", overlay=true)

float ma1 = ta.sma(close, 5)
float ma2 = ta.sma(close, 9)

if ta.cross(ma1, ma2)
    if strategy.position_size == 0
        strategy.order("Long", strategy.long, stop = high)
        strategy.order("Short", strategy.short, stop = low)
    else
        strategy.close_all()

plot(ma1, "Fast MA", color.aqua)
plot(ma2, "Slow MA", color.orange)
```

Depending on the price action, the strategy might fill *both* stop orders before creating the closing market order. In that case, the strategy exits the position without evaluating `strategy.close_all()` because both orders have the same size. We see this behavior in the chart below, where the strategy alternated between executing “Long” and “Short” orders a few times without executing an order from `strategy.close_all()`:



Figure 259: image